



Bare-Metal Ada on the ESP32-S3

A practical guide to the runtime, the peripheral HAL,
and a pure-Ada ext4 filesystem

Written in Ada, with a little Xtensa assembly.

github.com/rowsail/ada_esp32s3

Contents

1	Why Ada for Microcontrollers	1
1.1	The case for Ada	1
1.2	What this book covers	2
2	Getting Started	3
2.1	What you need	3
2.2	Get the code	3
2.3	The ./x dispatcher	3
2.4	Choosing a profile	4
2.5	Board configuration	4
2.6	Debugging	4
2.7	Where to go next	4
3	Anatomy of an Ada Application	7
3.1	The layers, top to bottom	7
3.2	What runs before Main	8
3.3	A minimal application	8
4	The Runtime Profiles	9
4.1	Choosing a profile	9
5	Building and Running on the ESP32-S3	11
5.1	Prerequisites	11
5.2	The ./x helper	11
5.3	What a build does	11
5.4	Packaging and flashing: two Ada host tools	12
5.4.1	esp_elf2image — ELF to flash image	12
5.4.2	esp_flash — writing it to the chip	12
5.5	Starting your own project	13
5.6	Console output	13
6	Debugging on the Chip	15
6.1	Interactive debugging: OpenOCD + GDB	15
6.2	Editor integration: VS Code and Vim	15
6.2.1	VS Code	16
6.2.2	Vim and Neovim	16
6.3	Post-mortem: decoding a Guru Meditation	18
6.4	The runtime’s own diagnostics	18
7	Ada Language Techniques	19
7.1	Terminology	19
7.2	Writing numbers: bases and underscores	21
7.3	Subtypes and modular types	21

7.3.1	Range subtypes: only the legal values	21
7.3.2	Static-predicate subtypes: a non-contiguous valid set	22
7.3.3	Modular types: wrap-around and bits	22
7.3.4	And beyond	22
7.4	Arrays and arbitrary index ranges	22
7.5	The others choice: filling in “everything else”	23
7.6	Fixed-point arithmetic	24
7.7	Trusting external data: the 'Valid attribute	25
7.8	Unchecked_Conversion: deliberate reinterpretation	25
7.9	Access types: where you need a pointer, and where you don't	25
7.9.1	Where you can avoid them	25
7.9.2	Where you genuinely need one	26
7.9.3	Why the pointer you do use is safer	26
7.10	Contracts for hardware-safe APIs	26
7.11	Exceptions and error handling	27
7.12	Restricting the language: pragma Restrictions and profiles	28
7.13	Controlling elaboration: Pure, Preelaborate, No_Elaboration_Code	29
8	Talking to Hardware	31
8.1	Register Addressing in Ada	31
8.1.1	An object at a fixed address	31
8.1.2	Generated register packages	32
8.1.3	Array overlays	32
8.2	Bit Fields and Record Representation	32
8.2.1	A register as a record	32
8.2.2	Enumerations with explicit codes	33
8.2.3	Scalar storage order: decoding on-disk and on-wire data	33
8.3	Volatile, Atomic, and full-word access	33
8.4	Packing: pragma Pack and packed arrays	34
8.5	Placing objects in memory	34
8.6	Controlling memory: bounded allocation	35
8.7	Interfacing with C, ROM, and assembly	35
8.8	The thread through it all	35
9	Encapsulation, Ownership and Concurrency	37
9.1	private types: hiding the representation	37
9.2	private child packages: sealing the unsafe layer	38
9.3	limited types: values that cannot be copied	38
9.4	controlled types and RAI: deterministic teardown	38
9.5	protected types and objects: race-free sharing	39
9.6	The four together: a misuse-proof driver	39
9.7	Related keywords	40
10	The Peripheral HAL	43
10.1	Conventions used by every driver	43
10.2	GPIO — digital input and output	44
10.3	RNG — hardware random numbers	45
10.4	Temperature — the on-die sensor	46
10.5	SPI — full-duplex master	46
10.6	I2C — master	48
10.7	UART — asynchronous serial	49

11	DMA, PWM, Audio, Pulse and Analog Peripherals	53
11.1	GDMA — general DMA	53
11.2	MCPWM — motor-control PWM	54
11.3	I2S — digital audio over GDMA	55
11.3.1	PDM mode: the hardware PCM ↔ PDM converters	56
11.4	LEDC — LED / general PWM	57
11.5	RMT — remote-control pulse TX/RX (IR, WS2812)	57
11.6	PCNT — pulse / edge counter	58
11.7	SDM — sigma-delta density output	59
11.8	TWAI — CAN 2.0	60
11.9	Timer — general-purpose timers	62
11.10	LCD — 8-bit i80 parallel master	62
11.11	ADC — SAR analog input	63
12	Low-Power, Touch and Crypto	65
12.1	RTC — deep sleep and retained memory	65
12.2	RTC_IO — pad hold and RTC pulls	66
12.3	Touch — capacitive sensing	67
12.4	SHA — hardware hashing	67
12.5	AES — hardware block cipher	68
13	Writing a Task-Safe Peripheral Driver	71
13.1	The problem	71
13.2	The ownership model	71
13.3	Three layers	72
13.4	Layer 1 — the Engine (the only code that sees the registers)	72
13.5	Layer 2 — ownership: the Session and the Guard	72
13.6	Layer 3 — the gateway that cannot be bypassed	73
13.7	The limited-handle variant	75
13.8	Why it is built this way	75
13.9	The benefits	75
13.10	When the rule applies (and when it does not)	76
14	Tasking and Concurrency	77
14.1	Tasks and interrupt handlers	77
14.1.1	Creating and starting a task	77
14.1.2	Creating and attaching an interrupt handler	78
14.1.3	Worked example: counting falling edges on GPIO0	78
14.2	Real-time tasks: periodic and time-driven	80
14.3	Waiting for an event — and timing out	81
14.3.1	Polling (any profile)	81
14.3.2	Blocking until an interrupt signals you (any profile)	81
14.3.3	Timing out: three profiles, three approaches	82
14.4	Concurrency patterns	84
14.5	Watchdogs and recovery	85
14.6	Dual-core (SMP) programming	86
15	Mass Storage: SD Cards and the ext4 Filesystem	87
15.1	SD_SPI — an SD card over the SPI master	87
15.2	SDMMC — the native SD/MMC host	88
15.3	The ext4 filesystem	88
15.3.1	Connecting a block device	88

15.3.2	Mounting and reading	89
15.3.3	Writing	90
15.3.4	Crash-safe transactions	90
15.3.5	Errors	90
15.3.6	How it is validated	91
16	Conformance at Scale: A Standalone ACATS Sweep	93
16.1	Two ways to run the suite	93
16.2	Getting the grade off the chip	93
16.3	The grader must not trust the test id	94
16.4	Phase 1 — parallel build	94
16.5	Phase 2 — parallel run and grade	95
16.6	Results	95
16.6.1	A wider net: the maximal list across seven boards	96
16.7	The embedded profile	96
16.8	Closing a won't-build category: the Ada.Calendar family	97
16.9	Why this matters	98
17	Building and Porting the Runtime	99
17.1	The idea: a board on top of bb-runtimes	99
17.2	gen_runtime.sh: from board to archive	99
17.3	Three profiles, three system.ads	100
17.4	The full profile: overlay plus patches	100
17.5	Register packages from the SVD	101
17.6	The stale-cache trap	101
17.7	Porting to another board	102
18	The Boot Path: Bootloader and PSRAM	103
18.1	Three things run before Main	103
18.2	The 2nd-stage bootloader	103
18.3	PSRAM: octal bring-up	104
18.4	start.S: from reset state to Ada	104
18.5	The memory map	105
19	The Context Switch	107
19.1	Where it lives	107
19.2	The data structures	107
19.3	The windowed-ABI premise	108
19.4	Two ways in	108
19.5	Dual-format dispatch	109
19.6	Walkthrough: a cooperative switch	109
19.7	Walkthrough: a preemptive switch	109
19.8	Starting a task that has never run	110
19.9	What state is saved — and what is not	110
19.10	Verifying the hard parts	110
20	Interrupts	113
20.1	The Xtensa interrupt levels	113
20.2	Priorities and the masking model	113
20.3	From silicon to handler	114
20.4	Attaching a handler: two axes	114
20.4.1	Axis 1 — API layer (high level vs low level)	114

20.4.2	Axis 2 — static vs dynamic (within the high-level layer)	115
20.5	Why the profiles differ	117
20.5.1	The bare-board full <code>System.Interrupts</code>	117
20.6	Exercising the vectors	118
21	Full-Profile Limitations (and Why They Rarely Bite)	119
21.1	Toolchain-bound: asynchronous select with a relative delay	119
21.2	RM-permitted: aborting a pure CPU loop	119
21.3	<code>Configurable_Run_Time</code> finalization	120
21.4	<code>Ada.Calendar</code> year range stops at 2099	120
21.5	Per-task heap reclamation under task churn	120
21.6	Minor robustness edges	121
21.7	Architectural: <code>Interrupt_Priority'Last</code> and the kernel tick	121
21.8	The common case is clean	121
A	Quick Reference	123
A.1	Usable GPIO pins	123
A.2	Runtime profiles at a glance	123
A.3	Example index	123
B	Glossary	125
C	Further Reading	129
C.1	The Ada language and its profiles	129
C.2	The runtime and the toolchain	129
C.3	The silicon	129
C.4	Conformance and high-integrity Ada	130
C.5	This project	130

Chapter 1

Why Ada for Microcontrollers

Microcontroller firmware lives in a hostile place: a few hundred kilobytes of RAM, no operating system to catch mistakes, interrupts that fire between any two instructions, and hardware registers whose exact bit layout is law. A bug is not a stack trace on a screen — it is a board that silently locks up in the field. Ada was designed for exactly this kind of software, and the language gives the firmware author tools that C simply does not have.

1.1 The case for Ada

Strong, range-checked types.

In Ada a “GPIO pin” need not be an `int`. It can be a type that admits *only* the pins the silicon actually exposes; passing a flash-mapped pad becomes a compile error, not a hung chip. Numeric types carry ranges, and the compiler (or a run-time check) enforces them.

Representation control.

Ada lets you state, in the language, the precise bit and byte layout of a record and the exact address of an object. A hardware register is just an Ada record at a fixed address — no casts, no macros, no guessing about padding (Sections 8.1 and 8.2 of the Talking to Hardware chapter).

Packages and strong interfaces.

Drivers are packages with a visible specification and a hidden body. The unsafe register pokes can be sealed inside a *private child* that application code literally cannot name.

Tasking and real-time in the language.

Tasks, protected objects (data guarded by a monitor), `delay until`, and priorities are part of Ada itself — not a bolted-on RTOS API. On this point they are implemented by a native Ada runtime that runs directly on the silicon.

Restriction profiles.

With one pragma you can forbid features that do not belong in certified or tiny firmware (no heap, no recursion in the runtime, no unbounded constructs) and have the compiler *prove* you stayed within the subset — the Ravenscar and Jorvik profiles (Chapter 4).

Contracts.

Preconditions, postconditions and type predicates let you state intent (“this key must be 128 or 256 bits”) and have it checked.

The result is firmware where a large class of mistakes — wrong-width register writes, out-of-range pins, aliasing of a peripheral by two tasks, forgetting to release a DMA channel — are caught by the compiler or by a cheap run-time check, long before they reach silicon.

1.2 What this book covers

This book documents a complete bare-metal stack for the dual-core ESP32-S3 (Xtensa LX7) written almost entirely in Ada, running directly on the silicon with its own runtime, boot path and tooling:

1. the Ada language techniques that make register and bit-field programming safe and pleasant (the Ada Language Techniques and Talking to Hardware chapters, Chapters [7](#) and [8](#));
2. the anatomy of an Ada application, from `Main` down through the runtime to the boot ROM, and the three selectable runtime profiles (Chapters [3–5](#));
3. the reusable peripheral HAL — twenty-plus task-safe drivers — with the API and worked examples for each (Chapters [10–12](#));
4. mass storage: the SD-card block drivers and the pure-Ada ext2/3/4 filesystem (Chapter [15](#)).

Ada terms that may be unfamiliar to a reader coming from C — *library-level*, *elaboration*, *aspect*, *protected object* and the like — are defined in the Terminology section of the Ada Language Techniques chapter (§[7.1](#)).

Chapter 2

Getting Started

This chapter takes you from a fresh clone to an LED blinking under your own Ada, in a few commands. The later chapters explain the runtime, the HAL and the language techniques; this one just gets you running.

2.1 What you need

An ESP32-S3 board.

Any devkit with the built-in USB-Serial-JTAG port (most of them). One USB cable is enough — it carries the console *and* the debugger.

Alire (alr).

The Ada package manager fetches the toolchains the build needs: the `gnat_xtensa_esp32_elf` cross-compiler, a native `gnat_native` (for the host tools), and `gprbuild`. Install Alire, then make sure its toolchains are on your `PATH` for the shell you build from.

Serial-port permission.

On Linux, add yourself to the `dialout` group (and install the `udev` rule for the USB-Serial-JTAG, see the debugging chapter) so you can open `/dev/ttyACM0` without `sudo`.

2.2 Get the code

The runtime depends on two git submodules (the `bb-runtimes` fork that carries the ESP32-S3 bareboard runtime, and `xtensa-dynconfig`), so clone *recursively*:

```
git clone --recurse-submodules \
    https://github.com/rowsail/ada_esp32s3.git
cd ada_esp32s3
# (if you already cloned without submodules:)
git submodule update --init --recursive
```

2.3 The `./x` dispatcher

Everything goes through one script at the repository root. List the examples, then build, flash and watch one:

```
./x list                                # the available examples + their profiles
./x run esp32s3_gpio0_blink -p /dev/ttyACM0 # build + flash + serial monitor
```

`./x run` builds the example, packages it (the Ada `esp_elf2image` host tool), writes it to the board (the Ada `esp_flash` tool, over the chip’s ROM bootloader), resets, and streams the console. The first build also generates the runtime for the chosen profile, so it takes a little longer; later ones are incremental. The blink example toggles an LED and prints over the console — your first Ada running bare-metal on the S3.

The pieces are also separate commands when you want them:

```
./x build   esp32s3_gpio0_blink           # cross-compile + link + package app.bin
./x flash   esp32s3_gpio0_blink -p /dev/ttyACM0
./x monitor -p /dev/ttyACM0               # just the serial console (115200)
./x clean   esp32s3_gpio0_blink
```

2.4 Choosing a profile

Most examples build under more than one runtime profile (Chapter 4). Pass `-profile` to pick one; `auto` (the default) uses the example’s own:

```
./x run esp32s3_heartbeat --profile embedded -p /dev/ttyACM0
```

`light-tasking` is the smallest, `embedded` adds exceptions, RAII handles, a heap and the full HAL (the usual choice), and `full` adds the complete Ada tasking model.

2.5 Board configuration

Each project owns a `config/board.ads` (flash and PSRAM sizes) that sizes the image and the second-stage bootloader. Edit it directly, or:

```
./x config esp32s3_gpio0_blink show
./x config esp32s3_gpio0_blink flash-size 4MB
```

2.6 Debugging

On-chip debugging needs the pinned OpenOCD + S3 GDB, fetched once; then `./x debug` (or F5 in VS Code) halts at your `Main`:

```
./x get-debug-tools
./x debug esp32s3_gpio0_blink           # build, flash, OpenOCD + GDB, rest at
Main
```

The Debugging chapter (Chapter 6) covers GDB, decoding a Guru Meditation, and the editor integration in full.

2.7 Where to go next

- The anatomy of an application and the runtime profiles — Chapters 3–5.
- The peripheral HAL — start at Chapter 10; each driver has worked examples you can run with `./x run`.
- The Ada techniques behind it all — the Ada Language Techniques and Talking to Hardware chapters (Chapters 7 and 8), including a Terminology section (§7.1) for unfamiliar terms.

To start your *own* project rather than an in-repo example, the `esp32-ada` launcher scaffolds one in any empty folder (see the tooling notes); it gives you the same `board.ads` and build flow without copying the runtime.

Chapter 3

Anatomy of an Ada Application

An Ada program for the ESP32-S3 is not a flat blob. It is a stack of layers, each written in Ada (with a thin sliver of Xtensa assembly at the very bottom). Knowing the layers explains what links into your binary, what runs before `Main`, and where each feature comes from.

3.1 The layers, top to bottom

Your application.

One or more Ada packages and a main procedure (`Main`). It withs the peripheral HAL (`ESP32S3.*`) and, if it uses tasks or protected objects, those language features directly.

The HAL (`libs/esp32s3_hal`).

Hand-written driver packages over a generated register layer (`ESP32S3_Registers.*`). Optional — a program can poke registers itself — but it is where all the device knowledge lives.

The GNAT binder output.

`gnatbind` generates a package (`b__main`) that elaborates every unit in the right order and calls `Main`. Its `adainit/adafinal` bracket your program.

The Ada runtime (RTS).

The `System.*` and `Ada.*` bodies: secondary stack, exceptions (if enabled), `Ada.Text_IO` over the USB-serial console, and — crucially — the tasking kernel (GNARL/GNARLI): tasks, protected objects, delays, scheduling.

Board support.

The silicon-specific bottom of the runtime: the Xtensa windowed *context switch*, *interrupt entry/exit* vectors, the *clock/alarm tick*, SMP scheduling and the inter-core interrupt. This is Ada plus a little assembly (`start.S`, `highint5.S`, `context.S`). This is the layer an off-the-shelf RTOS would otherwise provide — here it is ours.

The 2nd-stage bootloader.

Before any of the above runs, our own minimal loader (a ZFP-Ada `boot_main` + an assembly prologue + a thin C PSRAM shim) maps the flash cache/MMU and brings up the octal PSRAM, then jumps to the application image. It replaces the chip vendor's stock second-stage bootloader.

The mask ROM.

The chip's on-chip ROM loads the 2nd-stage bootloader from flash at reset.

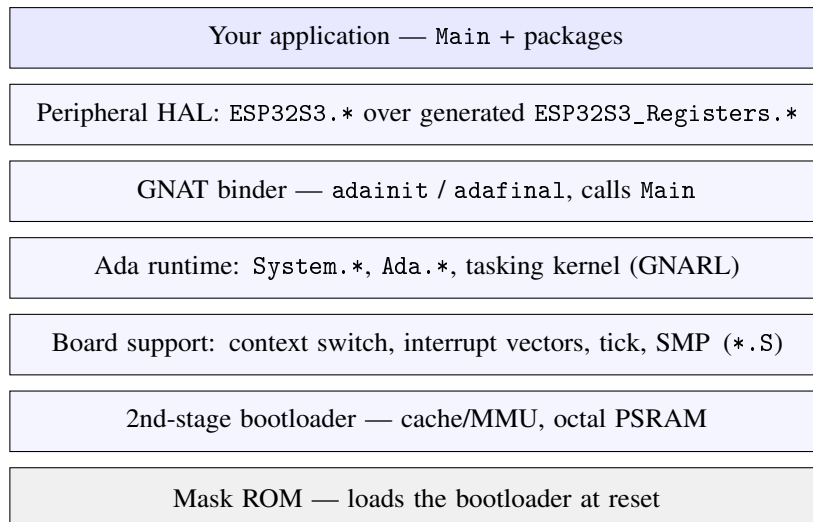


Figure 3.1: The layers of an Ada application, from your code down to the chip’s ROM. Everything above the ROM is Ada (with a little Xtensa assembly in board support).

3.2 What runs before Main

At power-on: ROM → our 2nd-stage bootloader (cache/MMU + PSRAM) → `start.S` (selects the 240 MHz PLL, sets up the stack and the interrupt vectors, and on the SMP examples releases the second core) → the runtime’s startup, which calls `adainit` (elaboration) → your `Main`. There is no vendor startup shim and no third-party RTOS scheduler — the Ada runtime is the only scheduler, and it is linked from source.

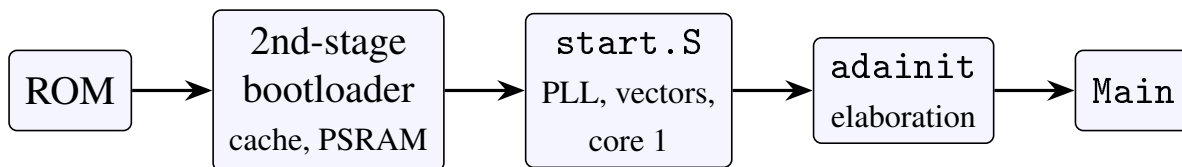


Figure 3.2: Reset to Main.

3.3 A minimal application

```

with Ada.Real_Time; use Ada.Real_Time;
with ESP32S3.GPIO;

procedure Main is
  Led : constant ESP32S3.GPIO.Pin_Id := 2;
begin
  ESP32S3.GPIO.Configure (Led, Mode => ESP32S3.GPIO.Output);
  loop
    ESP32S3.GPIO.Toggle (Led);
    delay until Clock + Milliseconds (250);
  end loop;
end Main;

```

`delay until` is served by the board-support tick; `ESP32S3.GPIO` is the HAL; everything links against the runtime crate.

Chapter 4

The Runtime Profiles

The runtime is offered in three *profiles*, selected by the environment variable `ESP32S3_RTS_PROFILE` when the crate is built. They trade language features against footprint and certifiability. The same application source can often build under more than one; the HAL drivers that use finalization are restricted to the richer two.

Profile	What it gives you
light-tasking	The Ravenscar/Jorvik subset (pragma Profile (Jorvik)). Tasks, protected objects, delay until, priorities — but with restrictions the compiler enforces: <code>No_Finalization</code> , <code>No_Exception_Propagation</code> , <code>No_Secondary_Stack</code> , and a heap-less bump allocator. Smallest and most analysable; a raised exception is fatal (resets the board). HAL drivers that need none of those (GPIO, RNG, Temperature, SDMMC, Touch, RTC, crypto) build here.
embedded	Still Jorvik, but exception-capable: full exception propagation with messages, controlled-type <i>finalization</i> (RAII), a secondary stack, and a real first-fit heap. This is the default target for most of the HAL and for the ext4 filesystem. The RAII driver handles (SPI/I2C/UART sessions, GDMA/LEDC/RMT/... channels) live here.
full	Complete Ada tasking with no Jorvik restrictions: dynamic task creation, task hierarchies, selective accept/rendezvous, abort and asynchronous transfer of control. Built on a fully ported GNARL kernel. Experimental (work in progress), but real rendezvous and abort run on the silicon.

4.1 Choosing a profile

- Certifiable, deterministic, minimal-footprint firmware, or code that must never allocate: `light-tasking`.
- Most applications — you want exceptions, RAII handles, the secondary stack, a heap, and the full HAL including the filesystem: `embedded`.
- Code that genuinely needs dynamic tasks, rendezvous or abort: `full`.

The selection happens at build time. An example's `build.sh` sets it:

```
export ESP32S3_RTS_PROFILE=embedded # or light-tasking / full
```

Because the HAL keys its object directory by profile, the same shared driver tree is safely reused across profiles without stale objects.

Chapter 5

Building and Running on the ESP32-S3

5.1 Prerequisites

- The Alire-managed GNAT cross-toolchain for the target (`gnat_xtensa_esp32_elf`) and `gprbuild`; these are fetched by Alire and discovered automatically by the build scripts.
- A USB cable to the board's built-in USB-serial-JTAG port (it appears as `/dev/ttyACM0`).
- Nothing else: no external SDK or build system. An Ada `esp_flash` host tool (built with the Alire `gnat_native` toolchain) packages and writes the image.

5.2 The `./x` helper

From the repository root, the `./x` script builds, flashes, monitors and debugs any example by name:

```
./x build esp32s3_gpio0_blink          # cross-compile + link + package app.bin
./x flash esp32s3_gpio0_blink -p /dev/ttyACM0
./x run   esp32s3_gpio0_blink -p /dev/ttyACM0 # build + flash + serial monitor
```

5.3 What a build does

`build.sh` sets the profile and heap size, then `bare_build.sh`:

1. regenerates/selects the runtime crate for the chosen profile (`gen_runtime.sh`; how that works, and how to port it, is Chapter 17);
2. compiles the Ada application against the pinned runtime with `gprbuild`, producing a relocatable `ada_app.o` (Main renamed to `_ada_main`);
3. compiles the shared bare boot (`start.S`, the interrupt vectors, the heap and a freestanding libc) and the example's C console glue;
4. links everything against the linker scripts into `app.elf`; and
5. packages `app.bin` (image header + segments).

`flash.sh` writes the 2nd-stage bootloader, the partition table and `app.bin` over USB-serial.

The last two steps — turning the linked `app.elf` into a flashable `app.bin`, and writing it to the chip — are handled by two small *host* tools written in native Ada (built with the Alire `gnat_native` toolchain), so the whole build/flash path needs only the Alire toolchain and the committed vendored blobs.

5.4 Packaging and flashing: two Ada host tools

5.4.1 esp_elf2image — ELF to flash image

examples/common/bare/elf2image converts app.elf into the ESP32-S3 flash image app.bin — the exact on-flash format the chip’s ROM and our second-stage bootloader expect (flash mode dio, freq 80m, size 2MB). The generated image boots unmodified on hardware across every example. The packaging is more than a flat copy:

- **Segments** are the ELF’s allocated PROGBITS sections, merged where contiguous and each padded to a multiple of four.
- A **24-byte header** — the 8-byte common part (magic 16#E9#, segment count, the dio/2MB/80m flash byte, the entry point) plus a 16-byte extended part (chip_id=9, wp_pin=16#EE#, hash_appended=1).
- **Flash (IROM/DROM) segments aligned to 64 KB**, with the RAM segments written *interleaved as the alignment padding* (then a zero PADDING segment for the remainder) — the exact scheme the bootloader’s MMU mapping relies on.
- An **XOR checksum** (seed 16#EF#) as the last byte of a 16-aligned block, and finally a **SHA-256** of the whole image appended (computed by an in-tree sha256.adb).

The flash parameters (dio/80m/2MB, chip = ESP32-S3) are fixed to match the bare boot; they are constants at the top of the source.

5.4.2 esp_flash — writing it to the chip

examples/common/bare/espflash writes the image over the chip’s ROM serial bootloader. It is 100 % Ada: the OS serial interface (open, termios raw mode, the TIOCM* DTR/RTS modem lines, poll, read, write) is bound directly to libc through Interfaces.C — there is no C source. (GNAT.Serial_Communications would cover the byte I/O but not the DTR/RTS control the reset needs, so the tool binds ioctl(TIOCM* DTR/RTS) itself.) The sequence is the standard ROM protocol, with no RAM stub and no compression:

1. **Reset into download mode** — the USB-JTAG DTR/RTS pulse.
2. **SYNC** — the 16#07# 16#07# 16#12# 16#20# + 32×16#55# handshake until the ROM answers.
3. **SPI attach** and **set-params** (the flash geometry).
4. Per file, **flash_begin** (erase) then **flash_data** in 16#400#-byte blocks (the last padded with 16#FF#), each carrying the ROM XOR checksum (seed 16#EF#).
5. **flash_end**, then a **hard reset to run** (suppressible with -no-reset for capture workflows like the ACATS sweep, which does its own reset).

Every command is a <direction, op, length, checksum> packet in SLIP framing (16#C0# delimiters, 16#DB# escaping) over the serial port. A typical invocation writes the three images at their flash offsets:

```
esp_flash -p /dev/ttyACM0 --flash-size 2MB \
0x0      bootloader.bin \
0x8000   partition-table.bin \
0x10000  app.bin
```

Note that `esp_flash` writes *flash* only; the external PSRAM is not flashed — the 2nd-stage bootloader maps it at runtime. On hardware a ~225 KB image flashes in about three seconds (ROM speed, no stub).

5.5 Starting your own project

The `esp32-ada` launcher scaffolds a fresh application in any empty directory (no runtime sources are copied — it pins the crate):

```
. export.sh                # put esp32-ada on PATH
mkdir my_app && cd my_app
esp32-ada new               # scaffold build.sh, the .gpr, src/main.adb, board.ads
# edit src/main.adb ...
esp32-ada run -p /dev/ttyACM0
```

Each project owns a `config/board.ads` that states the flash and PSRAM sizes (used to size the image header and build/select the bootloader):

```
package Board is
  Flash_Size : constant := 2 * 1024 * 1024;    -- 2 MB
  PSRAM_Size  : constant := 2 * 1024 * 1024;    -- mapped at 16#3D00_0000#
end Board;
```

5.6 Console output

The runtime wires `Ada.Text_IO` to the built-in USB-serial-JTAG console, so `Put_Line` reaches `/dev/ttyACM0`. The examples additionally use a tiny C glue layer calling the ROM `esp_rom_printf` for formatted status lines. Either way, `./x run` opens a monitor so you see the output live.

Chapter 6

Debugging on the Chip

When something misbehaves on the silicon there are three complementary tools: an interactive debugger over JTAG, the post-mortem backtrace, and the runtime’s own fault diagnostics. Each works against the bare application image directly, using only the pinned cross-toolchain.

6.1 Interactive debugging: OpenOCD + GDB

The ESP32-S3’s built-in USB-Serial-JTAG port is both the console *and* a JTAG debug interface, so a single USB cable is enough. The project pins its own copies of the debug tools; one command fetches them:

```
./x get-debug-tools      # downloads OpenOCD + xtensa-esp32s3-elf-gdb into tools/  
./x debug blink          # build, flash, start OpenOCD, attach GDB, rest at Main
```

One subtlety is worth knowing: the S3 needs the *S3-specific* xtensa-esp32s3-elf-gdb; the Alire crate’s esp32 (LX6) GDB silently fails on it. `./x debug` selects the right one. From there it is ordinary GDB on `app.elf` — breakpoints, `step`, `print`, `backtrace`. Two options matter on this target:

- `-smp` exposes *both* LX7 cores as GDB threads (`info threads`), so you can see what each core is doing — essential for a cross-core hang.
- `-attach` halts the board *in place* without resetting it, so you can examine a live hang or crash exactly where it stopped (a post-mortem).

The command line and both editors (next section) drive this same OpenOCD + GDB path; when finished, `./x kill-openocd` releases the USB-JTAG port.

6.2 Editor integration: VS Code and Vim

The same build, flash and debug actions are wired into both VS Code and Vim, so you need never leave the editor. Both drive the top-level `./x` dispatcher and both use the project’s pinned OpenOCD + s3 GDB for on-chip debugging.

Two prerequisites are shared by either editor. On-chip debugging first needs the tools fetched once:

```
./x get-debug-tools      # pinned, SHA-256-verified OpenOCD + xtensa-esp32s3-elf-gdb
```

And *code intelligence* — completion, go-to-definition, diagnostics, hover — comes from the **Ada Language Server**, which ships inside the AdaCore toolchain (nothing to install separately). It only needs to be pointed at the example’s `.gpr` project file, shown for each editor below.

6.2.1 VS Code

There are two complementary pieces. The first is the committed workspace configuration in `.vscode/` (`tasks.json`, `launch.json`, `settings.json`), which wires the editor to `./x` and needs two extensions:

Ada & SPARK (`AdaCore.ad`).

The Ada Language Server — required, for all code intelligence.

Native Debug (`webfreak.debug`).

A GDB frontend, needed only for F5 debugging. `cppdbg` cannot walk the Xtensa register-window stack (it connects but shows no call stack or source); Native Debug simply relays GDB’s own frames, which unwind correctly.

Install both from a terminal (or from the Extensions view, which also offers them as workspace recommendations):

```
code --install-extension AdaCore.ad
code --install-extension webfreak.debug
```

Then point the language server at the example you are working on by editing `ada.projectFile` in `.vscode/settings.json` and reloading the window (`Ctrl-Shift-P` → *Developer: Reload Window*):

```
"ada.projectFile": "examples/esp32s3_gpio0_blink/gpio_blink.gpr"
```

Build, flash and monitor are tasks: `Ctrl-Shift-B` runs the default Ada: build, and *Tasks: Run Task* offers flash, monitor, run and clean. Each prompts for the example, serial port and runtime profile and routes to `./x`; build errors land in the **Problems** panel via a GNAT problem matcher.

To **debug** (F5, “Ada: debug (OpenOCD + GDB)”): build *and flash* the example first, so the chip runs the same image whose `app.elf` you debug — the launch only *attaches*, so a stale or different-profile image leaves the breakpoint addresses mismatched. It starts OpenOCD and halts at `app_main` (the shared boot-glue entry; these GNAT images have no C main); set breakpoints in the Ada source and continue/step.

The second piece is an optional **status-bar extension** (`ide/vscode-ada-esp32`) that adds clickable build / flash / monitor / debug icons plus persistent port and profile selectors. A prebuilt `.vsix` is committed to the repository, so installing needs only the code CLI on PATH:

```
./x install-ide          # installs the committed vscode-ada-esp32.vsix
```

Reload the window afterwards; re-run it to update after a `git pull` (a repo-hosted `.vsix` does not auto-update). Its debug icon uses the same OpenOCD + GDB launch described above.

6.2.2 Vim and Neovim

The `ide/vim-ada-esp32` plugin gives the same actions as Vim commands. It is pure Vimscript — nothing to compile — and needs **Vim 8.1+ with +terminal** (which bundles the `termdebug` GDB frontend); Neovim works too. Install it with one command, which symlinks the plugin into Vim’s and Neovim’s native package directories; because it is a symlink *into* the repo, a `git pull` updates it in place:

```
./x install-vim
```

Or wire it up by hand — it has the standard plugin/ layout:

```
" vim-plug
Plug '/path/to/ada_esp32s3/ide/vim-ada-esp32'
" or, with no plugin manager:
set runtimepath^=/path/to/ada_esp32s3/ide/vim-ada-esp32
```

The commands auto-detect context by searching upward for `./x`, so they work from anywhere inside the repo:

```
:AdaEsp32Build [example]
    build via ./x build.

:AdaEsp32Flash [example]
    flash to the selected port.

:AdaEsp32BuildFlash [example]
    build then flash.

:AdaEsp32Run [example]
    build + flash + monitor.

:AdaEsp32Monitor
    serial console at 115200.

:AdaEsp32Debug [example]
    OpenOCD + termdebug (below).

:AdaEsp32Port [port] / :AdaEsp32Profile [prof]
    set the serial port / runtime profile; both persist across later commands.
```

The example argument accepts the short name (`gpio0_blink`) or the full directory (`esp32s3_gpio0_blink`), with tab-completion; the chosen example, port and profile persist (in `g:ada_esp32_example` and friends) so later commands reuse them without re-prompting.

Debugging uses Vim’s built-in `termdebug` — the Vim analog of the Native Debug setup, relaying GDB’s own frames so it handles Xtensa correctly. After a build:

```
:AdaEsp32Build gpio0_blink
:AdaEsp32Debug gpio0_blink
```

This starts OpenOCD in the background, waits for its GDB server, then opens `termdebug` reset and halted with a breakpoint armed at `app_main`. From there `:Continue` runs to it, `:Break src/foo.adb:53` (or `:Break` on the cursor line) sets an Ada breakpoint, and `:Step/Over/Finish` are the step controls; OpenOCD is stopped automatically when you quit the session. One inherent caveat: a `:Over` across a *blocking* statement (a delay, an entry call) can step into the hand-written Xtensa assembly (the interrupt vectors or context switch), which carries no debug bounds — GDB then reports “cannot find bounds of current function.” Set a breakpoint past the statement and `:Continue` instead.

Code intelligence again comes from the Ada Language Server through your LSP client, pointed at the example’s `.gpr`. For example, with `coc.nvim` in `coc-settings.json`:

```
{ "ada.projectFile": "examples/esp32s3_gpio0_blink/gpio_blink.gpr" }
```

or register `ada_language_server` for the `ada` filetype with native LSP / `vim-lsp` and set the same `ada.projectFile` setting.

6.3 Post-mortem: decoding a Guru Meditation

An unhandled fault prints a register dump and a backtrace of program-counter values over the console (a “Guru Meditation”). You do not need a live debugger to read it — turn the addresses back into source with `addr2line` against the ELF:

```
xtensa-esp32s3-elf-addr2line -fe build/app.elf 0x4200abcd 0x4200ef01
```

In the dump, `EXCVADDR` is the faulting data address and `EXCCAUSE` the reason — a `LoadStoreError` (a bad data access), or an `InstructionFetchError` (a wild jump, often a corrupted stack or a stray pointer). Flash, reproduce, decode: that loop localises most faults without ever attaching a debugger.

6.4 The runtime’s own diagnostics

The runtime helps before you reach for a tool at all:

- **Precise stack-overflow detection.** Each task arms a hardware watchpoint near its stack limit, so an overflow faults *precisely* (a debug exception) instead of silently corrupting a neighbour — and on the full profile the runtime can turn it into a catchable `Storage_Error`.
- **A fault safe-state hook.** The `esp32s3_safe_state` example wraps the panic handler to classify a fault — stack overflow versus other — against the running task’s stack window, latch a safe state, and print an explicit `*** STACK OVERFLOW ***` verdict before the board resets.
- **The last-chance handler.** Any exception that escapes every handler ends up in the runtime’s last-chance handler; on the leanest profile, where exceptions cannot propagate across calls, that is your fault report — a message and a controlled reset.

A note on console output while debugging. `Ada.Text_IO` reaches `/dev/ttyACM0`; for terse status from interrupt or early-boot context the examples call the ROM `esp_rom_printf`, which is always available. When two cores — or an interrupt handler and a task — might print at once, funnel output through one task: interleaved single-character writes have themselves been mistaken for bugs more than once.

Chapter 7

Ada Language Techniques

A handful of Ada features come up again and again in embedded code: writing literals clearly, constraining values to only what is legal, validating data that arrives from outside, reinterpreting bits deliberately, attaching checkable contracts, and handling — or forbidding — exceptions. This chapter collects the language-level techniques; the next, *Talking to Hardware*, applies them to registers and memory. Together they underpin the HAL and the filesystem, so a few terms are worth learning before the driver chapters that use them.

7.1 Terminology

A few Ada words recur throughout the book and can trip up a reader coming from C. They are defined once here.

Access type / 'Access.

Ada's typed pointer; `X'Access` yields a pointer to `X`. Strongly typed, with accessibility rules that guard against dangling references.

Activation.

The moment a declared task begins executing: after its master's declarations, or, for a library-level task, as part of elaboration before `Main`.

Aspect.

A property attached to a declaration with the word `with`: `with Volatile, with Address => ..., with Pre => ...`. Comparable to a GCC `__attribute__`, but standardised and checked.

Attribute.

A built-in query or operation written with an apostrophe: `X'Image`, `T'Last`, `Obj'Address`, `P'Access`. There is no C equivalent.

Ceiling priority.

The priority at which a protected object's operations run (the priority-ceiling protocol); for an interrupt handler it is the interrupt's hardware level.

Constraint check / Constraint_Error.

The run-time check that a value is in range or valid; a failure raises `Constraint_Error`. The checks can be switched off in a release build.

Elaboration.

The once-only, ordered set-up that runs *before* `Main`: evaluating initial values, executing the statements in package bodies, and activating library-level tasks. Like C's static initialisers, but ordered and able to run arbitrary code.

Entry / barrier.

A protected (or task) operation guarded by a Boolean condition, the *barrier*; a caller blocks until the barrier is open. This is how condition synchronisation is expressed.

Generic.

A compile-time template — a package or subprogram parameterised by types or values — instantiated to produce a concrete unit, much like a C++ template.

Library unit / library-level.

A unit (package or subprogram) declared at the outermost level, *not* nested inside a subprogram or block. A library-level object or task exists for the whole run of the program. C has only file scope and block scope; Ada nests arbitrarily, so “library-level” names the top, program-lifetime level — and the Jorvik profiles require tasks (and some other entities) to live there.

Package (specification / body).

A named module. The *specification* is the visible interface (roughly a C header) and the *body* is the implementation (roughly the `.c`) — but they are one logical unit and the body’s internals are genuinely hidden by the compiler, not merely by convention.

Pragma.

A directive to the compiler — `pragma Pack`, `pragma Attach_Handler`, `pragma Restrictions`. Like `#pragma`, but part of the language with defined meanings.

Profile (Ravenscar / Jorvik).

A named bundle of Restrictions defining an analysable tasking subset. Distinct from this project’s runtime *build* profiles, which choose *which* such subset the runtime supports.

Protected object.

Data plus operations with *automatic* mutual exclusion — a monitor. Access is race-free without a manual lock; the compiler generates the locking.

Rendezvous.

A synchronous call from one task to another task’s *entry*: the two meet, exchange parameters, the accept body runs, then both continue (the `full` profile).

Representation clause.

A clause that fixes a type’s machine layout — bit positions, size, byte order: `for T use record ... end record`. It is how a record is mapped onto a hardware register.

Secondary stack.

A runtime area used to return a result whose size the caller does not know in advance (e.g. a function returning `String`). C has no equivalent; the leanest profile omits it.

Task.

A thread of control defined in the language. It *activates* (starts) automatically — there is no explicit start call.

Type vs. subtype.

A *type* is a distinct set of values and operations; two distinct types never mix without an explicit conversion. A *subtype* is the same type plus a constraint (a range or predicate) — fully compatible with its base, just narrower.

Unconstrained type.

A type whose size is settled only when an object is created — `String`, `array (Positive range <>)`, a record with defaulted discriminants.

7.2 Writing numbers: bases and underscores

Embedded code is full of specific numbers — addresses, bit masks, clock dividers, register fields. Ada lets you write each literal in whatever base makes it clearest and group its digits for readability, with no runtime cost: the base and the underscores are purely lexical, resolved entirely by the compiler.

Base 10 is the default. A plain numeric literal is decimal:

```
Count : constant := 42;
Clk    : constant := 240000000;    -- 240 MHz, but hard to read at a glance
```

Other bases use base#digits#. The base — itself written in decimal, and any value from 2 to 16 — precedes a value delimited by #. The four literals below are *the same number*, 42, just written in different bases:

```
D : constant := 42;           -- base 10 (the default)
B : constant := 2#101010#;    -- binary      (also 42)
O : constant := 8#52#;       -- octal       (also 42)
H : constant := 16#2A#;      -- hexadecimal (also 42)
```

In a hexadecimal literal the letter digits A–F may be upper- or lower-case (16#2a# and 16#2A# are identical).

Underscores group digits. An underscore may appear *between* digits (never leading, trailing, doubled, or next to a #); the compiler ignores it completely. It exists only so a human can read — and proofread — a long literal. Group however suits the base: thousands for decimal, nibbles or bytes for hexadecimal and binary.

```
Clock_Hz : constant := 240_000_000;    -- 240 MHz, grouped in thousands
Base      : constant := 16#6000_4000#;  -- a peripheral base address
Mask      : constant := 2#1111_0000_1111_0000#; -- a bit mask, by nibble
All_Ones  : constant := 16#FFFF_FFFF#;  -- 32 set bits, by 16-bit halves
```

The grouping is what makes a transcription slip obvious: a missing digit in 16#6000_4000# throws the nibble columns out of alignment, whereas 16#6004000# hides it. This is why the register and bit-field code throughout this book reads the way it does — addresses are underscore-grouped hexadecimal (16#6003_8000#), and masks and field positions are binary or 2 ** n.

7.3 Subtypes and modular types

The other way the type carries intent is by constraining which values are legal at all. Two facilities earn their keep constantly in embedded code: subtypes that narrow a value to a valid range or set, and modular types that give wrap-around arithmetic and bit operations.

7.3.1 Range subtypes: only the legal values

A subtype attaches a constraint to an existing type. The everyday embedded use is to make an out-of-range value impossible:

```
subtype Channel is Natural range 0 .. 7;    -- a peripheral with 8 channels
subtype Percent is Natural range 0 .. 100;  -- a duty cycle, say
```

A value known at compile time that violates the range is a compile error; a value computed at run time is caught by a constraint check (wherever checks are enabled). Either way “channel 9” never reaches the hardware, and the range is documented in the type rather than in a comment.

7.3.2 Static-predicate subtypes: a non-contiguous valid set

When the legal values are not one contiguous range, a *static predicate* states the exact set. The HAL's pin type is the real example: of the 0..48 pad numbers, some are absent on the package and some are bonded to the in-package SPI flash and octal PSRAM — driving any of those hangs the chip:

```
type Pad_Number is range -1 .. 48;           -- -1 is the "no pin" sentinel

subtype Pin_Id is Pad_Number range 0 .. 48 with
  Static_Predicate => Pin_Id in 0 .. 21 | 38 .. 48;  -- 22..37 reserved
```

Because the predicate is *static*, naming a reserved pad as a constant of type `Pin_Id` is rejected at compile time (*static expression fails static predicate check*); a pin computed at run time is verified by the predicate under `-gnata`. Every driver names its pins with `Pin_Id`, so a flash/PSRAM pad cannot be wired by accident.

From C / ESP-IDF: a pin is an int or a #define, and wiring a flash pad is a run-time hang you debug with an oscilloscope. `Pin_Id` turns a bad constant into a *compile* error and a bad computed value into a checked one.

7.3.3 Modular types: wrap-around and bits

A modular type — `mod 2**n` — holds the values $0..2^n - 1$, and its arithmetic *wraps* instead of overflowing. That is exactly the behaviour you want for a register word, a free-running counter, or a checksum; modular types also support the logical operators `and`, `or`, `xor` and `not`:

```
type Byte is mod 2 ** 8 with Size => 8;      -- 0 .. 255, wraps at 256

Sum : Byte := 0;
...
Sum  := Sum + B;                             -- no overflow error; wraps (a checksum)
Flags := Flags or 2#0000_0100#;              -- set a bit
Flags := Flags and not 2#0000_0100#;         -- clear a bit
```

The predefined `Interfaces.Unsigned_8/16/32/64` are the usual choice for register-sized words — they are modular and add `Shift_Left / Shift_Right / Rotate_*`. A plain signed integer would instead raise `Constraint_Error` on overflow and offers no bit operators. So the rule of thumb is: registers, masks and counters are *modular*, while quantities that must never silently wrap — an index, a length — stay *range-constrained*.

7.3.4 And beyond

Two more worth keeping in mind:

- **Fixed-point types** (`type Q8 is delta 1.0 / 256.0 range 0.0 .. 1.0`) give fractional values with exact, integer-backed arithmetic — a duty cycle or a scaled sensor reading without dragging in floating point.
- **Derived types** make otherwise-identical numbers *incompatible*: with `type Hz is new Natural;` and `type Millis is new Natural;`, a frequency can never be passed where a delay was meant — the mix-up is a compile error, not a bug.

7.4 Arrays and arbitrary index ranges

An Ada array type fixes not just its element type but its *index range*, and that range is yours to choose — any discrete range at all. The lower bound need not be 0 or 1; it can be any value (even negative), and the index can be an enumeration type rather than an integer. The range is part of the type:

```

type Sine_Table is array (0 .. 255) of Integer;      -- 0-based lookup table
type CAN_Data   is array (1 .. 8)   of Unsigned_8;  -- 1-based, eight bytes
type Window     is array (-3 .. 3)   of Float;      -- centred on zero
type Channel     is (Red, Green, Blue);
type Mix         is array (Channel)  of Unsigned_8;  -- indexed by an enum

```

The point is not novelty: it lets the index *match the thing it models*, so the code reads like the datasheet or the protocol — with no mental “subtract one” and none of the off-by-one bugs that come with it.

Iterating without knowing the bounds. Every array value carries its own bounds, available as attributes: A’First, A’Last, A’Length and A’Range (which is A’First .. A’Last). The idiomatic loop ranges over ’Range, so it is correct whatever the bounds are — and stays correct if you later change them:

```

Total : Natural := 0;
for I in Buf'Range loop      -- I takes each actual index value in turn
    Total := Total + Natural (Buf (I));
end loop;

```

Because the loop names Buf’Range, the same code works for a 0 .. 255 buffer and a 1 .. 8 one; there is no literal bound to get wrong. (When you do not need the index itself, for E of Buf loop iterates the elements directly.)

A worked example: J1939 CAN payloads, indexed from 1. A Classic CAN frame carries up to eight data bytes. SAE J1939 — the protocol on commercial vehicles — numbers those bytes *1 through 8* in its specifications (“byte 1,” “bytes 4–5,” and so on). Declaring the payload with a 1 .. 8 index makes the Ada line up with the document exactly:

```

type J1939_Payload is array (1 .. 8) of Interfaces.Unsigned_8;

-- PGN 61444 (EEC1): engine speed is SPN 190, in BYTES 4-5,
-- little-endian, scaled 0.125 rpm/bit. The byte numbers here are
-- taken verbatim from the J1939 spec -- no off-by-one translation.
function Engine_RPM (D : J1939_Payload) return Float is
    Raw : constant Interfaces.Unsigned_16 :=
        Interfaces.Unsigned_16 (D (4)) +      -- byte 4: low
        Interfaces.Unsigned_16 (D (5)) * 256;  -- byte 5: high
begin
    return Float (Raw) * 0.125;
end Engine_RPM;

```

D (4) is “byte 4” — the same number a J1939 reference uses — so the code can be checked against the standard line by line, and the loop idiom still applies: for I in D’Range loop walks bytes 1 .. 8.

From C / ESP-IDF: In C the eight bytes are data[0 .. 7], so every J1939 byte number from the spec has to be mentally decremented as you read or write the code (spec “byte 4” becomes data[3]). An arbitrary lower bound removes that whole translation step.

7.5 The others choice: filling in “everything else”

others is a catch-all *choice* — it stands for “every case not named so far,” and it must always come *last*. Ada uses it wherever you enumerate possibilities: array and record aggregates, case statements, and exception handlers. It is everywhere in embedded Ada (it fills the reserved fields of the register writes in the next chapter), so it is worth a close look.

In array aggregates — fill the rest. An aggregate must give a value for *every* element; others supplies the ones you did not name.

```
Zeros  : Buffer := (others => 0);           -- every element 0
Header : Buffer := (0 => 16#AA#, 1 => 16#55#, others => 0); -- first two, rest 0
```

In record aggregates — and the <> “box”. A record aggregate must also set every component. Writing `others => <>` means “give each remaining component its *default* value” — the default from the type’s own declaration. This is the central idiom for writing a hardware register: set the one or two fields you mean, and let every reserved or don’t-care field take its declared default.

```
-- A register record whose fields mostly default to 0 / False:
R.TX_CONF := (TX_START => True, others => <>); -- set one bit, default the rest
```

Use `others => <>` rather than `others => 0` for a record whose remaining components are of *different* types: each takes its own default, so the types need not agree. `others => 0` only works when everything it covers is the same type (e.g. an all-zero byte buffer).

In case — cover the remaining values. A case must be exhaustive; when `others` handles whatever you did not list.

```
case Mode is
  when Standard => ...;
  when others   => ...;    -- every other value of the type
end case;
```

In exception handlers — catch the rest. when `others` catches any exception not named earlier in the handler.

```
begin
  ...
exception
  when Constraint_Error => ...; -- a specific one
  when others           => ...; -- anything else
end;
```

Two properties tie these uses together: `others` is always the *last* choice, and it covers *exactly* the cases not already named — so adding a component, value or exception elsewhere silently changes what “everything else” means. For a register write that is precisely what you want (a newly-defined reserved bit still defaults). For a case over an enumeration it is often the opposite: omitting `others` is safer, because then *adding* a value to the type forces the compiler to make you handle it.

7.6 Fixed-point arithmetic

Microcontroller code is full of fractional quantities — a duty cycle, a gain, a scaled sensor reading — that do not justify (or cannot afford) floating point. A *fixed-point* type gives fractions with exact, integer-backed arithmetic: you state the resolution with `delta`, and the compiler represents values as integers scaled by that `'Small`.

```
type Duty is delta 1.0 / 256.0 range 0.0 .. 1.0; -- 8-bit fraction, 0 .. 1
D : Duty := 0.75;                               -- stored as the integer 192 (= 0.75 * 256)

type Volts is delta 0.001 range 0.0 .. 3.3;      -- millivolt resolution
```

Arithmetic on Duty is integer arithmetic underneath — no FPU, no rounding surprises — yet the source reads in real-world units. The PWM and ADC drivers scale this way: a duty cycle or a voltage is a fixed-point value the driver converts to the register's raw counts.

7.7 Trusting external data: the 'Valid attribute

Anything that enters from outside the program — a value read from a register, a field unpacked from an SD sector, a message off the wire — might not be a legal value of its type. Reading it into a constrained type and then using it is *erroneous* if the bits are out of range. X'Valid asks “is the object actually a valid value of its (sub)type?” and answers with a plain Boolean — no exception, safe to ask even of garbage:

```
Raw : Parity with Address => Reg'Address, Volatile; -- enum from hardware
begin
  if Raw'Valid then
    Use_Parity (Raw);           -- known to be None/Even/Odd
  else
    Handle_Corrupt;           -- some other bit pattern
  end if;
```

This is the safe way to consume an enumeration with reserved codes, a range field that hardware might leave uninitialised, or a structure read from possibly-corrupt storage — validate first, trust after.

7.8 Unchecked_Conversion: deliberate reinterpretation

Occasionally you must view the same bits as two different types — serialise a record to bytes, decode a fat pointer, read a float's bit pattern. Ada.Unchecked_Conversion is the explicit, greppable way to do it; unlike a C cast it is a named instantiation, so every reinterpretation in the codebase is searchable.

```
function To_Fat is new Ada.Unchecked_Conversion (Handler_Access, Fat_Ptr);
```

The runtime uses exactly this to inspect an `access protected procedure` as a `(object, code)` pair when matching a registered interrupt handler. The discipline: prefer a representation clause or an address overlay when the two views have a defined relationship; keep `Unchecked_Conversion` for the genuine bit-reinterpretation cases, where its name documents the hazard.

7.9 Access types: where you need a pointer, and where you don't

Ada has pointers — *access types* — but a great many of the places a C programmer reaches for one do not need it. The embedded habit is to lead with the pointer-free option and use an access type only where the problem genuinely demands it.

7.9.1 Where you can avoid them

Parameters are already passed efficiently. You do not pass `&thing` and dereference. A parameter mode — `in`, `out`, `in out` — says how it is used, and the compiler passes a large object by reference automatically:

```
procedure Process (Frame : in out Packet); -- no pointer; Frame is the object
```

Whole arrays and records are values. You can assign, return and pass entire structures; there is no need for a pointer just to “return a struct.”

```
R := S; -- copies a whole record
function Identity return Matrix; -- returns a whole array by value
```

Unconstrained types size themselves. A String or an array (Positive range <>) takes its bounds from its value, so a function can return one without malloc and without a pointer (the secondary stack carries it):

```
function Pin_Name (P : Pin_Id) return String; -- no heap, no pointer
```

A register is an address overlay, not a pointer. As §8.1 showed, you place a typed object *at* the register's address; you never hold a volatile uint32_t * and dereference it.

A resource is a handle, not a raw pointer. The limited-controlled handles of Chapter 9 own a peripheral by scope, not by a pointer you must remember to free.

7.9.2 Where you genuinely need one

- **Dynamic or linked data built at run time** — a list, a tree, a variable-length buffer pool: type Node_Ref is access Node; and new Node.
- **Callbacks and interrupt handlers** — an access-to-subprogram value lets you register code to be called later. This is the On_Edge'Access from the GPIO counter (§14.1) and the handler registration in the Interrupts chapter.
- **Hardware structures that reference a buffer** — a DMA descriptor holds the *address* of the data to move, so the buffer is aliased and the descriptor stores its 'Access (§8.5).
- **Talking to C** — C interfaces are expressed in pointers, so System.Address and access types cross that boundary (§8.7).

7.9.3 Why the pointer you do use is safer

Even when you need one, an Ada access type is far less hazardous than a C pointer:

- it is **named and typed** — a Node_Ref is not a Byte_Ref and neither is a void *; the compiler keeps them apart;
- there is **no pointer arithmetic** — you index an array, you do not increment a pointer past its end;
- **accessibility rules** reject Local'Access returned from the subprogram that owns Local, so a dangling reference is a compile error, not a crash;
- not null access subtypes exclude the null case at the type level;
- and for T'Storage_Size use 0; (§8.6) forbids new on an access type outright — a compile-time promise that this pointer type never touches a heap, which is how the light-tasking profile stays heap-less.

So the rule of thumb: prefer value semantics and address overlays; reach for an access type for the genuinely dynamic structure, the callback, or the hardware-referenced buffer — and even then it is typed, arithmetic-free, and, if you wish, provably heap-free.

7.10 Contracts for hardware-safe APIs

Finally, a driver's *interface* can carry its rules as checkable contracts, so misuse is caught at the call site rather than producing a wrong register write. Preconditions and postconditions state what must hold on

entry and exit; type invariants and predicates state what must always hold of a value. The AES driver, for instance, accepts only a legal key length:

```
function Supported_Key (Key : Byte_Array) return Boolean is
  (Key'Length in 16 | 32);           -- AES-128 or AES-256 only

procedure Set_Key (Key : Byte_Array)
  with Pre => Supported_Key (Key);    -- a 24-byte key is rejected at the call
```

The cost is tunable: pragma Assertion_Policy (and the -gnata switch) turns the run-time checks on for development and *off* for a release build, where the contracts remain as machine-checked documentation. Combined with the range and predicate subtypes of §7.3, the effect is that a large class of driver-misuse bugs — a bad pin, a wrong-width key, an out-of-range channel — cannot reach the hardware.

7.11 Exceptions and error handling

An Ada *exception* is a named error condition: `raise` signals it and a handler catches it. The mechanics are familiar:

```
begin
  Read_Sector (N, Buf);
exception
  when Storage_Error =>           -- out of memory / stack
    Recover;
  when E : others =>              -- anything else; E names it
    Log (Ada.Exceptions.Exception_Message (E));
end;
```

A `raise` propagates up the call chain to the nearest matching handler, running finalizers (Chapter 9) on the way — which is why a controlled handle is released even when an exception unwinds past it. If no handler matches, it keeps propagating; if it escapes the whole program, the runtime's *last-chance handler* runs (below).

Predefined and user-defined exceptions. The language defines a handful the runtime raises for you: `Constraint_Error` (a failed range, index, null or overflow check — what a bad `Pin_Id` or an out-of-range index produces), `Program_Error` (an illegal runtime situation), `Storage_Error` (out of stack or heap), and `Tasking_Error`. You declare your own just as easily, and `raise` it with a message:

```
Not_Owned : exception;           -- a named error condition of your own
...
if not Holding then
  raise Not_Owned with "use the bus without holding it";
end if;
```

A handler can inspect the exception through `Ada.Exceptions` — `Exception_Name`, `Exception_Message`, `Exception_Information` — and a bare `raise`; inside a handler *re-raises* the current one, so you can clean up locally and still let an outer scope deal with it.

How this project uses them. The ownership-checked HAL gateways raise `Not_Owned` / `Not_Initialized` when a session is used without being held or set up; the AES driver's `Supported_Key` precondition (§7.10) rejects a wrong-length key; the range and predicate subtypes (§7.3) raise `Constraint_Error` on a bad value under -gnata; and the ext4 filesystem reports I/O and corruption errors by raising. The lock-free drivers instead report failure through a `Status` enum or an out Boolean — precisely so they work on *every* profile, which is the next point.

The cost — and even the availability — depends on the profile. This is the embedded-specific part, and it differs across all three:

- `light-tasking` is `No_Exception_Propagation`: there is no unwinder, so a `raise` not handled *in the same subprogram* cannot propagate — in practice it reaches the last-chance handler and resets the board. The range and predicate checks still *detect* errors; you simply cannot catch and recover across calls. Design for “detect, log, reset,” and report an expected failure with a status result, not an exception.
- `embedded` adds *zero-cost* exceptions (ZCX): full propagation, handlers, `Ada.Exceptions` names and messages, and finalization on the way out (so an RAII handle releases during an unwind). A handler that is not entered costs nothing on the happy path — the cost lives in static tables, not in instructions. This is the minimum profile for the RAII drivers and the filesystem; here you catch and recover normally.
- `full` has the *same* exception model as `embedded`, plus the complete tasking runtime, so exceptions interoperate with dynamic tasks, rendezvous and abort — and a stack overflow caught by the runtime’s guard can be delivered as a catchable `Storage_Error` instead of a reset (see the Debugging chapter).

When an exception escapes everything — always on `light-tasking`, and as a final backstop elsewhere — the runtime’s *last-chance handler* runs; on this port it reports the fault and resets, and the `esp32s3_safe_state` example shows a fault being classified there (see the Debugging chapter).

Strategy on a microcontroller. Prefer the cheap, local mechanisms to *prevent* bad values — range and predicate subtypes (§7.3), `'Valid` (§7.7), contracts (§7.10); use a status result for an expected, recoverable condition (a NACK, a CRC mismatch); and reserve exceptions for the genuinely exceptional, remembering that on the leanest profile “handling” one means a controlled reset, not a resumption.

A runnable demonstration. The `esp32s3_exceptions` example exercises all four outcomes on the chip — local handling, propagation across a call, a re-raise to an outer handler, and an unhandled exception reaching the last-chance handler (it installs a custom one, exported as `__gnat_last_chance_handler`, that prints the exception and halts).

7.12 Restricting the language: `pragma Restrictions` and profiles

The flip side of choosing features is forbidding them. `pragma Restrictions` lets you ban constructs that have no place in tiny or certifiable firmware and have the compiler *prove* you stayed within the subset:

```
pragma Restrictions (No_Dependence => Ada.Unchecked_Deallocation);
pragma Restrictions (No_Recursion, No_Secondary_Stack);
pragma Restrictions (No_Exceptions);           -- if a raise would be fatal anyway
```

A named bundle of these is a *profile*: `pragma Profile` (`Ravenscar`) — and its superset `Jorvik` — selects the analysable tasking subset (no dynamic task creation, no entry queues longer than one, a single delay form), which is what the `light-tasking` and `embedded` runtimes are built against. The Runtime Profiles chapter covers the runtime side; the point here is that the restriction is a *language* statement the compiler enforces unit by unit, so a violation is a build error, not a surprise on silicon.

7.13 Controlling elaboration: Pure, Preelaborate, No_Elaboration_Code

Elaboration (§7.1) is the code that runs before Main to bring declarations to life. On a microcontroller you usually want *less* of it: less start-up code, and data that can live in flash rather than be built in RAM at boot. Category pragmas let you state — and have the compiler enforce — how little a unit needs:

- `pragma Pure` — the unit has no state and no elaboration at all (only pure functions and constants); the strongest promise, and its data can sit in flash.
- `pragma Preelaborate` — the unit may be elaborated without running any code: all its data has static initial values, so there is no boot-time cost and nothing is built at start-up.
- `pragma Elaborate_Body` — forces the body to be elaborated immediately after the spec, a sequencing guarantee that keeps elaboration order predictable.
- `pragma Restrictions (No_Elaboration_Code)` — the compiler *rejects* any unit that would emit elaboration code: a hard guarantee that start-up is free.

```
package Sensor_Tables is
  pragma Pure;                                -- pure: a flash-resident lookup table
  Lut : constant array (0 .. 255) of Integer := ( ... );
end Sensor_Tables;

package Ring is
  pragma Preelaborate;                         -- no code runs to elaborate this unit
  ...
end Ring;
```

Embedded payoff. A Pure or Preelaborate unit contributes nothing to boot time and keeps its constants out of RAM; the runtime itself builds its lowest layers with No_Elaboration_Code so the kernel has no hidden start-up. The pragmas turn “this should be cheap to start” into a checked property.

Chapter 8

Talking to Hardware

A peripheral register is a fixed memory location with side effects; a structure read off storage or the wire is a foreign byte layout. Ada lets the *type* carry both — the exact address, the bit positions, the volatility, the endianness — so once a register or structure is described correctly it is safe at every use, with no pointer casts or shift-and-mask macros. This chapter is those techniques; they build directly on the language features of Chapter 7.

8.1 Register Addressing in Ada

A peripheral register is a fixed memory location with side effects. To program it in Ada you describe *three* things: its layout (a type), its location (an address), and the fact that touching it is observable (volatility). You never need a pointer cast.

8.1.1 An object at a fixed address

The simplest register is a 32-bit word. Place an Ada object at its address and mark it `Volatile` so the compiler never caches, reorders, or elides accesses:

```
with System;
with Interfaces; use Interfaces;

procedure Toggle_Raw is
  -- The GPIO "output set" and "output clear" W1TS/W1TC registers.
  GPIO_OUT_W1TS : Unsigned_32
    with Volatile, Address => System'To_Address (16#6000_4008#);
  GPIO_OUT_W1TC : Unsigned_32
    with Volatile, Address => System'To_Address (16#6000_400C#);
begin
  GPIO_OUT_W1TS := 2 ** 2;  -- set GPIO2 (writing a 1 sets that bit)
  GPIO_OUT_W1TC := 2 ** 2;  -- clear GPIO2 (writing a 1 clears it)
end Toggle_Raw;
```

`System'To_Address` turns a numeric literal into an address; the `Address` aspect binds the object there. Because the type is exactly `Unsigned_32`, the write is exactly one 32-bit store — the compiler cannot “optimise” it into a byte write, which a peripheral might not accept.

From C / ESP-IDF: this replaces `*(volatile uint32_t *)0x60004008 = x;` — but with no cast, and with the access width fixed by the type, so the compiler cannot silently split or merge the access.

8.1.2 Generated register packages

Writing every register by hand is error-prone, so the whole ESP32-S3 register map is generated from the vendor SVD file into typed packages under `ESP32S3_Registers.*` (one package per peripheral). Each register is a record with a representation clause, and each peripheral is one big record placed at its base address. A driver then writes fields by name:

```
with ESP32S3_Registers.GPIO; use ESP32S3_Registers.GPIO;

-- GPIO_Periph is the peripheral record, already placed at 16#6000_4000#.
GPIO_Periph.OUT_W1TS.OUT_W1TS := 2 ** 2;    -- set GPIO2, by field name
```

The application almost never touches these directly — it calls the hand-written HAL (`ESP32S3.GPIO` and friends) — but the same techniques underpin every driver.

8.1.3 Array overlays

Sometimes the SVD flattens a regular bank of registers (a FIFO, a descriptor table) into separate fields. You can overlay your own array on top of them with `Address`, recovering clean indexing:

```
type Reg_Bank is array (0 .. 7) of Interfaces.Unsigned_32 with Volatile;
Bank : Reg_Bank with Import, Volatile, Address => Some_Periph.First_Reg'Address;
-- Bank (3) is now the fourth 32-bit register, addressed normally.
```

The `Import` aspect says “this object already exists at that address; do not allocate or initialise it.” This idiom appears throughout the HAL where the register layout is regular but the generator did not capture it as an array.

8.2 Bit Fields and Record Representation

Hardware registers pack many fields into one word: a 3-bit mode here, a single enable bit there, a 12-bit divider somewhere else. In C this means shift-and-mask macros that are easy to get wrong. In Ada you declare a record whose components have the right types and then a *representation clause* that nails each component to specific bits. The compiler does the shifting and masking for you, correctly, every time.

8.2.1 A register as a record

```
type Mode_Bits is mod 2 ** 3;      -- a 3-bit field
type Divider   is mod 2 ** 12;     -- a 12-bit field

type Control_Register is record
  Enable   : Boolean;              -- 1 bit
  Mode     : Mode_Bits;            -- 3 bits
  Reserved : Boolean := False;
  Div      : Divider;             -- 12 bits
end record
with Volatile_Full_Access, Size => 32, Bit_Order => System.Low_Order_First;

for Control_Register use record
  Enable at 0 range 0 .. 0;
  Mode   at 0 range 1 .. 3;
  Reserved at 0 range 4 .. 4;
  Div     at 0 range 5 .. 16;
end record;
```

Now you assign fields by name and the layout is guaranteed:

```
Ctrl := (Enable => True, Mode => 2, Div => 1023, others => <>);
```

at 0 range L .. H means “bits *L* through *H* of the byte at offset 0.” `Volatile_Full_Access` guarantees the whole 32-bit word is read or written in one access — essential for registers that latch on a full-word write. This is precisely how the generated `ESP32S3_Registers` packages describe every register, and how the HAL builds register values:

```
-- From the real SPI driver: build a USER register value by field, in one write.
Regs.USER := (DOU_TDIN => True, USR_MOSI => True, USR_MISO => True,
              CK_OUT_EDGE => Out_Edge, others => <>);
```

8.2.2 Enumerations with explicit codes

When a field’s values are an enumeration, give the enumeration an explicit representation so the symbolic names map to the exact hardware codes:

```
type Parity is (None, Even, Odd);
for Parity use (None => 0, Even => 2, Odd => 3);
```

8.2.3 Scalar storage order: decoding on-disk and on-wire data

The same machinery decodes *external* byte layouts, not just registers. A record can declare its `Scalar_Storage_Order`, and GNAT will read each field with the correct endianness automatically — invaluable for filesystem and protocol structures. The ext4 filesystem in Chapter 15 relies on exactly this idea (though it ultimately uses explicit little-/big-endian readers for clarity), and JBD2’s big-endian journal records show why being able to say “this structure is big-endian” in the type is so useful.

8.3 Volatile, Atomic, and full-word access

Placing an object at a register’s address is only half the story; you must also tell the compiler how that location behaves, because the optimiser otherwise assumes ordinary memory — caching reads, dropping “dead” writes, reordering. A small family of aspects states the truth:

- `Volatile` — every read and write happens, in program order, and none is cached or elided. The baseline for any register.
- `Atomic` — additionally, each access is indivisible (a single load or store). Use it for a flag shared with an interrupt handler or the other core, where a torn half-update would be a bug.
- `Volatile_Full_Access` — the whole object is read or written in *one* access of its size. Essential for a register that latches on a full-word write or clears on read: updating one bit-field still emits a single 32-bit access, not a byte poke.
- `Volatile_Components / Atomic_Components` — the same, applied to each element of an array. The runtime’s per-core `Switch_Pending` and `In_Native_Int` flags are `Volatile_Components` arrays for exactly this reason.

```
Ready : Boolean with Atomic, Address => Status_Reg'Address; -- ISR-shared flag
-- Control_Register from the bit-field section is Volatile_Full_Access:
-- Ctrl.Enable := True emits one 32-bit store, not a read-modify-byte-write.
```

Pick the weakest that is correct: `Volatile` for most registers, `Volatile_Full_Access` for latch-on-write ones, `Atomic` only where indivisibility actually matters (it can be more expensive).

8.4 Packing: pragma Pack and packed arrays

The bit-field record nails *named* fields to bits. When you instead have a homogeneous run of small components — a bit vector, a table of 2-bit codes — `pragma Pack` (or a `Component_Size` clause) tells the compiler to store them at their natural bit width instead of rounding each up to a byte or word:

```
type Pin_Mask is array (0 .. 31) of Boolean;
pragma Pack (Pin_Mask);           -- 32 bits total, one bit per pin
-- or, explicitly:
type Codes is array (0 .. 15) of Mode_Bits with Component_Size => 2; -- 32 bits
```

A packed array (...) of Boolean is the idiomatic Ada bitmap: you index it by position (`Mask (Pin) := True`) and the compiler does the shift-and-mask, while the whole thing still occupies one word you can overlay on a register. Reach for the named record when the fields differ; reach for a packed array when they are uniform and indexed.

8.5 Placing objects in memory

On a microcontroller *where* an object lives is part of its meaning: some data must sit in fast IRAM, some in external PSRAM, some in RTC memory that survives deep sleep, and some must *not* be cleared at startup. Two aspects cover it:

- `Linker_Section => "..."` places an object in a named output section the linker script positions (e.g. an IRAM or `.noinit` section).
- an `Address` overlay (§8.1) pins it at a fixed location — which is how the RTC-retained boot counter in the deep-sleep example lives in RTC slow memory and survives a reset:

```
Boot_Count : Unsigned_32 with Volatile,
  Address => System'To_Address (16#5000_0000#); -- RTC slow RAM, retained

IRAM_Buf : Buffer with Linker_Section => ".iram1.data";
```

Alignment matters too: with `Alignment => 64` forces DMA buffers onto a cache-line boundary, and the GDMA driver relies on aligned, internal-RAM (not PSRAM) descriptors — placement and alignment together.

Base	Top	Region
16#3C00_0000#	16#3D00_0000#	Flash → DROM — cached read-only data
16#3D00_0000#	16#3E00_0000#	External PSRAM — mapped by the bootloader
16#3FC8_8000#	16#3FD0_0000#	Internal SRAM (DRAM view) — data, stacks, heap
16#4037_0000#	16#403E_0000#	Internal SRAM (IRAM view) — code, vectors, DMA
16#4200_0000#	16#4400_0000#	Flash → IROM — cached code
16#5000_0000#	16#5000_2000#	RTC slow RAM — retained across deep sleep

Table 8.1: Principal address-space regions, by ascending base address. Flash (DROM) and external PSRAM are not separate windows: they are two sub-ranges of one 32 MB external data-bus window (16#3C00_0000#–16#3E00_0000#) that the cache MMU partitions — this project maps flash from the bottom and PSRAM at 16#3D00_0000#, the PSRAM’s actual extent set by the fitted device. Placement pins objects where they belong: RTC memory for state that survives sleep, internal SRAM for DMA buffers (PSRAM cannot be a DMA source), flash for constants.

8.6 Controlling memory: bounded allocation

Embedded firmware usually wants allocation to be *bounded* or absent. Ada makes the size explicit rather than leaving it to a global heap:

- for `T'Storage_Size` use `0`; on an access type forbids `new` of `T` — a compile-time guarantee that pointer type never heap-allocates.
- A task can cap its own stack: `task body Worker` with `pragma Storage_Size` (or the `Storage_Size` aspect) sizes the primary stack precisely.
- The binder caps the secondary stacks (`-D<n>`, `-Q<n>`), and the `light-tasking` profile removes the heap and secondary stack entirely (a bump allocator only).

```
type Node_Ref is access Node with Storage_Size => 0;  -- new Node => compile
error
task Sampler with Storage_Size => 4 * 1024;           -- a 4 KB primary stack
```

Sizing memory in the source — not discovering it when the heap is exhausted at 3 a.m. in the field — is the embedded mindset the language supports directly.

8.7 Interfacing with C, ROM, and assembly

A bare-metal stack still meets non-Ada code: a ROM routine, a C glue function, a few instructions only assembly can express. Ada's interfacing pragmas make those boundaries explicit and typed.

Calling out — `pragma Import` (or the `Import` aspect) with `Convention => C` binds an Ada declaration to an external symbol; no header, no wrapper:

```
procedure Rom_Printf (Fmt : System.Address)
  with Import, Convention => C, External_Name => "esp_rom_printf";
```

Being called — `pragma Export` publishes an Ada subprogram or object under a C name so the boot assembly or glue can reach it (the runtime exports its interrupt-dispatch entries and the per-core flags this way). `Interfaces.C` (and `.Strings`, `.Pointers`) provides the C-compatible scalar and array types for the data that crosses.

Inline assembly — for the handful of operations with no Ada spelling (a special register read, a cache or barrier instruction), `System.Machine_Code.Asm` embeds Xtensa instructions with typed operands; the GDMA driver uses it for a memory barrier around descriptor hand-off. It is a scalpel, not a habit: everything that can be Ada, is.

8.8 The thread through it all

One idea runs through this chapter and the last: in Ada the *type* — with its layout, its range, its volatility, its contracts — carries the intent, so the compiler (or a cheap, switch-off-able run-time check) catches the mistake. Describe a register, a value, or an interface correctly once, and every use of it afterwards is safe by construction.

Chapter 9

Encapsulation, Ownership and Concurrency

Four Ada features let an interface control, in the type system itself, what client code is allowed to do — and together they make a peripheral driver that is misuse-resistant by construction. `private` hides a type’s representation; `limited` forbids copying it; `controlled` gives it automatic setup and teardown; `protected` serialises access to it. None costs more at run time than the discipline you would otherwise have to write and remember by hand, and each turns a class of “be careful to...” rules into a compiler-enforced guarantee. This chapter explains each keyword with small examples and its embedded payoff; the *Writing a Task-Safe Peripheral Driver* chapter shows them combined in the real HAL.

9.1 `private` types: hiding the representation

A private type is declared in two halves. The visible part of a package gives the type a name and the operations on it; the *private part* — everything after the word `private` — gives its real definition. Client code may use the name and call the operations, but cannot see or touch the components.

```
package Counters is
  type Counter is private;           -- clients see only this and the ops
  procedure Bump (C : in out Counter);
  function Value (C : Counter) return Natural;
private
  type Counter is record             -- the real definition, hidden
    N : Natural := 0;
  end record;
end Counters;
```

A user can declare a `Counter`, `Bump` it and read its `Value`, but cannot write `C.N`, cannot rely on it being a record, and will not break if the representation later changes. This is the difference between a `struct` in a public C header — whose fields anyone may scribble on — and a genuinely opaque handle.

Embedded payoff. The representation of a driver handle (which host it owns, a DMA-descriptor pointer, a hardware index) is hidden, so application code cannot corrupt it and the driver author stays free to change it.

9.2 private child packages: sealing the unsafe layer

Visibility extends to whole packages. A child package declared *private* is visible only inside its own family’s bodies; application code cannot even *name* it in a *with* clause. The HAL uses this to seal the only code that touches registers:

```
private package ESP32S3.SPI.Engine is    -- un-with-able from an application
    -- the raw, lock-free register driver: the ONLY code that pokes SPI.
    ...
end ESP32S3.SPI.Engine;
```

The public `ESP32S3.SPI` package withs the engine in its body and mediates every operation. **Embedded payoff.** “Poke the registers directly” is not merely discouraged — it is *unwritable*: a `with ESP32S3.SPI.Engine;` in an application fails to compile, so the unsafe core is reachable only through the safe façade.

9.3 limited types: values that cannot be copied

A limited type has no assignment (`:=`) and no predefined equality (`=`). You can declare one, pass it (by reference) and operate on it, but you cannot copy it. Make a type limited when a value stands for a *unique* thing that must not be duplicated — a handle to a single hardware resource, a lock, an open session:

```
type Session is limited private;    -- a SPI bus session
...
A, B : Session;
B := A;    -- COMPILE ERROR: Session is limited, it cannot be copied
```

If you *could* copy a handle, two copies would each believe they own the bus; releasing one would leave the other dangling. *limited* makes that bug impossible to write. It is the real shape of the SPI Session, the GDMA Channel, the ADC Reader, the LEDC/PCNT Channel and the rest of the HAL’s ownership handles.

9.4 controlled types and RAII: deterministic teardown

`Ada.Finalization.Controlled` — and its non-copyable sibling `Limited_Controlled` — give a type three hooks the runtime calls *automatically*: `Initialize` when an object comes into being, `Finalize` when it leaves scope (*including* when an exception unwinds the scope), and `Adjust` after a copy (for the non-limited kind). That is RAII: a resource’s lifetime is tied to an object’s scope.

```
type Session is new Ada.Finalization.Limited_Controlled with record
    Host : ...;
end record;
overriding procedure Finalize (S : in out Session);    -- auto-release the bus
```

```
declare
    S : Session;    -- Initialize
begin
    Acquire (S, Host => SPI2);    -- take the bus
    Transfer (S, ...);
end;    -- Finalize runs HERE: the bus is released,
        -- even if Transfer raised an exception
```

Embedded payoff. You *cannot forget* to release a DMA channel, unlock a bus, or power down a peripheral — the release is bound to the scope and happens on every exit path, with no leak, deterministic timing, and no goto cleanup ladder. `limited + controlled` is the project’s standard handle: `limited` stops you copying the owner, `controlled` guarantees it is released. (The real SPI Session’s overriding `Finalize` is commented “auto-release on scope exit” for exactly this.)

From C / ESP-IDF: ESP-IDF hands you an opaque handle and a matching `*_delete` / `*_free` you must remember to call on every path (including the error ones). A `controlled Session` releases itself when it leaves scope — even when an exception unwinds past it — so the leak and the missed-cleanup bug cannot happen.

9.5 protected types and objects: race-free sharing

A protected object is data plus operations with mutual exclusion built in — a monitor. Only one task at a time runs a protected *procedure* or *entry*; the compiler generates the locking, and on this runtime the lock is a *priority ceiling*, so worst-case blocking is bounded (real-time friendly). There are three kinds of operation:

- a protected **function** reads the state and may run concurrently with other functions (a shared read);
- a protected **procedure** reads and writes with exclusive access (one caller at a time);
- a protected **entry** is a procedure with a Boolean *barrier*; a caller blocks until the barrier is open (condition synchronisation).

```
protected Counter is
  procedure Bump;           -- exclusive write
  function Value return Natural; -- shared read
private
  N : Natural := 0;
end Counter;

protected body Counter is
  procedure Bump is begin N := N + 1; end Bump;
  function Value return Natural is (N);
end Counter;
```

`Counter.Bump` from one task and `Counter.Value` from another never race — no `Volatile`, no hand-rolled critical section. The HAL uses exactly this to arbitrate a shared peripheral: a per-port protected `Guard` (or `Pool`, or `Lock`) hands out ownership so two tasks cannot drive the same controller at once, and an interrupt handler is itself a protected procedure (the Interrupts chapter). **Embedded payoff.** Shared state across tasks, cores and interrupt handlers is correct by construction, with bounded, analysable blocking.

9.6 The four together: a misuse-proof driver

Combined, these keywords make a peripheral that is hard to misuse even on purpose:

- the registers live in a `private` child — you cannot see or poke them;
- the public handle is `limited` — you cannot copy ownership;
- it is `controlled` — you cannot forget to release it;
- access is `protected` — you cannot race it.

That is precisely the HAL's task-safe driver skeleton; the *Writing a Task-Safe Peripheral Driver* chapter builds one end to end (Engine + Session + Guard). The payoff is that a whole class of embedded bugs — a dangling handle, a forgotten unlock, a double-driven bus, a torn shared counter — becomes a compile error or a structural impossibility instead of a field failure.

9.7 Related keywords

A few neighbours round out the family. Each is shown with a small example.

constant — a value that never changes. A constant is given its value once and can never be assigned again; the compiler may place it in read-only memory (flash), saving RAM. A constant with no type — a *named number* — is a pure compile-time value.

```
Max_Pins : constant := 48;           -- named number (no type, no storage)
Sine      : constant array (0 .. 255) of Integer := ( ... ); -- table, in flash
-- Sine (K) := 0; -- COMPILE ERROR: Sine is constant
```

In embedded code lookup tables, calibration data and configuration are constant so they live in flash, not in scarce RAM.

aliased — an object you may point at. By default you cannot take 'Access of a declared object (the accessibility rules guard against dangling pointers). Marking it aliased permits it — needed when, say, a DMA buffer must be handed to a descriptor by reference.

```
Buf  : aliased DMA_Buffer;           -- 'Access may now be taken
Desc.Src := Buf'Access;              -- give the DMA engine a pointer to it
```

renames — a new name for an existing thing. A renaming declaration introduces an *alias*: a second name that denotes the *same* entity — no copy, no new object is made. It works for objects, components, subprograms, packages and exceptions.

```
-- alias a deeply-nested register field: short name, same storage
Ctrl : Unsigned_32 renames Periph.SPI2.User.Control;
Ctrl := 16#1#;           -- writes Periph.SPI2.User.Control

-- alias a package to shorten a long path (just this one name, unlike 'use')
package GE renames ESP32S3.GPIO;
GE.Toggle (2);

-- alias a subprogram -- give an operation a local or operator name
function "+" (L, R : Vector) return Vector renames Vectors.Add;
```

For an object or component the alias shares the original's storage — writing through Ctrl above writes the real register field — so *renames* is the way to name one specific deeply-qualified thing once and use the short name afterwards. Note how it differs from *use* (§8.1): *use* makes *all* of a package's names directly visible, whereas *renames* introduces exactly *one* new name and says precisely what it stands for — so it shortens a single long reference without the wholesale visibility (and possible name clashes) that *use* brings.

tagged — records with inheritance and dispatch. A tagged type can be *extended* (a derived type adds components) and its primitive operations can *dispatch* at run time: a call through a class-wide value T'Class selects the right override for the actual object.

```
type Sensor is tagged record Id : Natural; end record;
function Read (S : Sensor) return Integer;

type Temp_Sensor is new Sensor with record Pin : Pin_Id; end record; -- extends
overriding function Read (S : Temp_Sensor) return Integer;           -- override
```

A call `Read (S)` where `S` is `Sensor`'Class dispatches to `Temp_Sensor`'s `Read` if that is the actual type. It is used sparingly in embedded code (the embedded-profile demo shows library-level tagged dispatch) because a dispatching call is an indirect call.

abstract — **a type or operation with no implementation.** An abstract type cannot be instantiated, and an abstract operation has no body — every concrete descendant *must* provide one. This is how Ada expresses an interface: a contract that says “every sensor shall have a `Read`.”

```
type Sensor is abstract tagged null record;
function Read (S : Sensor) return Integer is abstract; -- no body; must override
-- S : Sensor; -- COMPILE ERROR: Sensor is abstract
```

These are reached for less often than `private` / `limited` / `protected` / `controlled`, but knowing them completes the picture of how Ada lets the *declaration* of a type state what may — and may not — be done with it.

Chapter 10

The Peripheral HAL

The hand-written drivers live under `libs/esp32s3_hal` as children of a single root package, `ESP32S3`: `ESP32S3.GPIO`, `ESP32S3.SPI`, and so on. A consumer adds one line to its project file,

```
with "esp32s3_hal.gpr";    -- or a relative path in-repo
```

and then withs the drivers it needs. Every driver in this book has been exercised on real silicon.

10.1 Conventions used by every driver

Valid-pin types.

A pin is named with `ESP32S3.GPIO.Pin_Id`, a subtype whose *static predicate* admits only the pads the ESP32-S3 exposes (0–21 and 38–48); the flash/PSRAM pads are excluded. A bad constant pin is a compile-time error.

Errors.

Simple drivers report failure through a `Status` enum or an out `Boolean`; the filesystem and richer layers raise exceptions. Either way, failure is explicit.

Task-safety, two shapes.

A peripheral that is a single shared controller (SPI host, I2C host, UART port, I2S port, the TWAI/LCD controller) is mediated by a protected object plus a limited, non-copyable `Session` handle: you `Acquire` it, use it, and it is released automatically when it leaves scope. A peripheral that is a *pool* of identical units (GDMA channels, LEDC/RMT/PCNT/SDM channels, timers, ADC units, MCPWM channels) hands out a limited, controlled handle you `Claim`; ownership makes per-handle operations safe without a global lock, and the unit is reclaimed on scope exit.

RAII.

Those `Session/Channel/Reader` handles are controlled types — they release on scope exit, including during exception unwinding — so they require the embedded or full profile. The lock-free drivers (GPIO, RNG, Temperature, RTC, RTC_IO, Touch, SHA, AES, SDMMC) work under every profile.

Init-before-use guard.

For the drivers that configure a controller once at startup and only later hand out ownership (SPI, I2C, UART, I2S, TWAI, LCD, and MCPWM), `Acquire/Claim` raises `Not_Initialized` if the port/host/unit was never `Setup`. Configuration must precede ownership; the contract is enforced, not merely documented.

Ownership gateway.

Across all six Session drivers (SPI, I2C, UART, I2S, TWAI, LCD), all register access lives in a private Engine child, and the configured handle it returns is hidden in a nested package body. The only way to reach it is one ownership-checked gateway, `Owned (S)`, which raises `Not_Owned` unless the caller holds the port. Every operation that touches a live controller — transfers *and* every post-Setup reconfiguration (pins, baud, inversion, loopback) — goes through it. So a transfer or a setting change without ownership fails loudly, and a new operation cannot be added that skips the check: there is no other door to the hardware. This is a project rule for any exclusive driver that can be reconfigured after setup (see the UART section for the worked pattern).

Recurring parameter types. Each driver below lists its parameters with their allowed values and defaults. A few types recur throughout and are described once here rather than on every driver:

`Pin_Id`

a usable GPIO pad — 0 .. 21 or 38 .. 48 (the gaps are flash/PSRAM/absent pads). A bad constant is a compile error.

`Optional_Pin`

a `Pin_Id` or `No_Pin` (the value `-1`) to leave that line unrouted.

Ownership handles (`Session`, `Channel`, `Reader`, `Unit`, `Timer`)

the limited, controlled handles described above: you declare one, `Acquire/Claim` it, pass it to each operation, and it releases on scope exit.

`..._Hz`, `Freq`, `Baud`, `Bit_Rate`

frequencies / rates in hertz (or bits per second); `Positive` unless a narrower subtype is noted.

`..._Percent`

a `Float` in 0.0 .. 100.0.

Buffers

a transfer takes a buffer 'Address and a byte Length; the DMA-backed drivers cap a single transfer at 4095 bytes.

10.2 GPIO — digital input and output

ESP32S3.GPIO configures a pad's direction, pull and drive strength and then drives or reads it. `Set/Clear/Write/Read` are hardware-atomic (lock-free); `Configure/Toggle` are serialised by a protected object.

```
procedure Configure (Pin : Pin_Id; Mode : Pin_Mode;
                    Pull  : Pull_Mode      := Floating;
                    Drive : Drive_Strength := Drive_Medium);
procedure Set      (Pin : Pin_Id);  procedure Clear (Pin : Pin_Id);
procedure Toggle  (Pin : Pin_Id);  procedure Write  (Pin : Pin_Id; On : Boolean);
function Read     (Pin : Pin_Id) return Boolean;
```

Parameters.

`Pin` the pad to act on (`Pin_Id`).

`Mode`

direction: `Input` or `Output`.

`Pull`

internal resistor: `Floating` (default), `Pull_Up` or `Pull_Down`.

Drive

output current: Drive_Weak, Drive_Medium (default), Drive_Strong, Drive_Strongest (roughly 5 / 10 / 20 / 40 mA).

On (Write) the level to drive — True = high, False = low.

Configure always sets the pad up as a software-controlled GPIO (routed through the GPIO matrix). Handing a pad to a *peripheral* instead is the job of that peripheral's own Configure_Pins, which programs the matrix directly — the ESP32-S3 GPIO matrix is a full crossbar, so almost any signal can reach almost any pad.

Example 1 — blink an LED.

```
with Ada.Real_Time; use Ada.Real_Time;
with ESP32S3.GPIO; use ESP32S3.GPIO;
procedure Blink is
  Led : constant Pin_Id := 2;
begin
  Configure (Led, Mode => Output);
  loop
    Toggle (Led);
    delay until Clock + Milliseconds (250);
  end loop;
end Blink;
```

Example 2 — a button drives an LED.

```
with ESP32S3.GPIO; use ESP32S3.GPIO;
procedure Button is
  Button_Pin : constant Pin_Id := 9;    -- active-low, internal pull-up
  Led        : constant Pin_Id := 2;
begin
  Configure (Button_Pin, Mode => Input, Pull => Pull_Up);
  Configure (Led, Mode => Output);
  loop
    Write (Led, not Read (Button_Pin)); -- pressed (low) -> LED on
  end loop;
end Button;
```

10.3 RNG — hardware random numbers

```
function Read return Word;           -- a 32-bit random word
procedure Fill (Buffer : out Byte_Array); -- fill any length
```

Parameters. Read takes none and returns a Word (a 32-bit Interfaces.Unsigned_32). Fill's Buffer is an out Byte_Array of *any* length — the call fills every element with random bytes.

Example 1 — a random word.

```
with Ada.Text_IO; use Ada.Text_IO;
with ESP32S3.RNG;
procedure RNG_Word is
  X : constant ESP32S3.RNG.Word := ESP32S3.RNG.Read;
begin
  Put_Line (ESP32S3.RNG.Word'Image (X));
end RNG_Word;
```

Example 2 — a random nonce.

```
with ESP32S3.RNG;
procedure RNG_Nonce is
  Nonce : ESP32S3.RNG.Byte_Array (0 .. 15);
begin
  ESP32S3.RNG.Fill (Nonce);  -- 16 random bytes
end RNG_Nonce;
```

(For true entropy the radio/ADC clocks should be running; see the package spec.)

10.4 Temperature — the on-die sensor

```
procedure Initialize (Span : Measure_Range := Range_Minus10_80);
function Read_Celsius return Integer;  -- whole degrees
function Read_Centi_Celsius return Integer;  -- hundredths of a degree
```

Parameters. Initialize's Span (a Measure_Range) picks the calibrated window — accuracy is best when it brackets the real temperature. The five values are Range_Minus10_80 (default, $-10..80^{\circ}\text{C}$), Range_20_100, Range_50_125, Range_Minus30_50 ($-30..50$) and Range_Minus40_20 ($-40..20$). The two Read_* functions take no parameters; one returns whole degrees, the other hundredths.

Example 1 — read degrees.

```
with Ada.Text_IO;      use Ada.Text_IO;
with ESP32S3.Temperature; use ESP32S3.Temperature;
procedure Temp_Degrees is
begin
  Initialize;
  Put_Line ("die temp =" & Integer'Image (Read_Celsius) & " C");
end Temp_Degrees;
```

Example 2 — higher resolution and a different span.

```
with Ada.Text_IO;      use Ada.Text_IO;
with ESP32S3.Temperature; use ESP32S3.Temperature;
procedure Temp_Centi is
begin
  Initialize (Span => Range_50_125);  -- a hotter measurement range
  declare
    Centi : constant Integer := Read_Centi_Celsius;
  begin
    Put_Line ("temp =" & Integer'Image (Centi / 100) & "." &
              Integer'Image (Centi mod 100) & " C");
  end;
end Temp_Centi;
```

10.5 SPI — full-duplex master

ESP32S3.SPI drives either general-purpose host (SPI2/SPI3) over GDMA. Configure the host once, then Acquire a Session for a transaction. Transfer shifts Tx out while capturing Rx.

```
procedure Setup (Host : SPI_Host; Mode : SPI_Mode := 0;
                Clock_Hz : Positive := 1_000_000);
procedure Configure_Pins (Host : SPI_Host; Sclk, Mosi, Miso : Optional_Pin;
```

```

        Cs : Optional_Pin := No_Pin);
procedure Acquire (S : in out Session; Host : SPI_Host);
procedure Transfer (S : Session; Tx, Rx : System.Address; Length : Natural);
procedure Release (S : in out Session);

```

Parameters.**Host**

which general-purpose host: SPI2 or SPI3.

Mode

SPI clock mode 0 . . 3 (the CPOL/CPHA combinations); default 0.

Clock_Hz

bit clock in Hz; default 1_000_000, clamped to roughly 80 kHz .. 80 MHz.

Sclk, Mosi, Miso, Cs

the four lines as Optional_Pin; Cs defaults to No_Pin (drive it as a plain GPIO when you have more than one device).

Tx, Rx, Length

transmit/receive buffer 'Addresses and a byte count, 1 . . 4095; full-duplex, so both move Length bytes together.

Pad (Enable_Loopback) one pad that folds MOSI back to MISO for the wiring-free self-test.**Example 1 — write a command to a device.**

```

with Interfaces;
with ESP32S3.SPI; use ESP32S3.SPI;
procedure SPI_Write is
  S : Session;
  Tx : aliased array (0 .. 2) of Interfaces.Unsigned_8 := (16#9F#, 0, 0);
  Rx : aliased array (0 .. 2) of Interfaces.Unsigned_8;
begin
  Setup (SPI2, Mode => 0, Clock_Hz => 8_000_000);
  Configure_Pins (SPI2, Sclk => 12, Mosi => 11, Miso => 13, Cs => 10);
  Acquire (S, SPI2);
  Transfer (S, Tx'Address, Rx'Address, 3);    -- Rx now holds the response
end SPI_Write;                               -- S releases the host here

```

Example 2 — wiring-free self-test via internal loopback.

```

with Interfaces;
with ESP32S3.SPI; use ESP32S3.SPI;
procedure SPI_Loopback is
  S : Session;
  Tx : aliased array (0 .. 31) of Interfaces.Unsigned_8 := (others => 16#A5#);
  Rx : aliased array (0 .. 31) of Interfaces.Unsigned_8 := (others => 0);
begin
  Setup (SPI2);
  Enable_Loopback (SPI2, Pad => 5);    -- MOSI->MISO through one pad
  Acquire (S, SPI2);
  Transfer (S, Tx'Address, Rx'Address, 32);
  -- Rx = Tx
end SPI_Loopback;

```

10.6 I2C — master

```

procedure Setup (Host : I2C_Host; Clock_Hz : Positive := 100_000);
procedure Configure_Pins (Host : I2C_Host; Scl, Sda : Pin_Id);
procedure Acquire (S : in out Session; Host : I2C_Host);
procedure Write (S : Session; Addr : Slave_Address; Data : Byte_Array;
                Success : out Boolean; Check_Ack : Boolean := True);
procedure Read (S : Session; Addr : Slave_Address; Data : out Byte_Array;
                Success : out Boolean);

```

Parameters.

Host

I2C0 or I2C1.

Clock_Hz

SCL frequency; default 100_000 (standard mode), 400_000 for fast mode.

Scl, Sda

the two bus lines (Pin_Id); the bus is open-drain, so fit external pull-ups.

Addr

7-bit slave address, 0 .. 16#7F# (Slave_Address).

Data

the bytes to write, or the out buffer to read into (Byte_Array — its length sets the count).

Success

out Boolean — True if the slave acknowledged.

Check_Ack

(Write) require an ACK on each byte; default True.

Example 1 — write a register on a sensor at address 16#68#.

```

with ESP32S3.I2C; use ESP32S3.I2C;
procedure I2C_Write_Reg is
  S : Session;
  Ok : Boolean;
begin
  Setup (I2C0, Clock_Hz => 400_000);
  Configure_Pins (I2C0, Scl => 9, Sda => 8);
  Acquire (S, I2C0);
  Write (S, Addr => 16#68#, Data => (16#6B#, 16#00#), Success => Ok);  -- reg<=0
end I2C_Write_Reg;

```

Example 2 — register read (write the address, then read).

```

with ESP32S3.I2C; use ESP32S3.I2C;
procedure I2C_Read_Reg is
  S : Session;
  Ok : Boolean;
  Who : Byte_Array (0 .. 0);
begin
  Setup (I2C0, Clock_Hz => 400_000);
  Configure_Pins (I2C0, Scl => 9, Sda => 8);
  Acquire (S, I2C0);
  Write (S, 16#68#, (1 => 16#75#), Ok);  -- point at WHO_AM_I register
  Read (S, 16#68#, Who, Ok);  -- read it back

```

```
end I2C_Read_Reg;
```

10.7 UART — asynchronous serial

```
-- Startup (single-threaded): the ONLY port-based configuration call.
procedure Setup (Port : UART_Port; Baud : Baud_Rate := 115_200;
                 Bits : Data_Bits := 8; Parity : Parity_Mode := None;
                 Stop : Stop_Bits := One;
                 Tx, Rx, Rts, Cts : Optional_Pin := No_Pin; -- routes the pads too
                 Rx_Flow_Threshold : Natural := 100);
procedure Acquire (S : in out Session; Port : UART_Port);
procedure Write (S : Session; Data : Byte_Array);
procedure Read (S : Session; Data : out Byte_Array; Count : out Natural);
function Available (S : Session) return Natural;
-- Post-Setup reconfiguration -- each takes the HELD Session, not a Port:
procedure Set_Baud (S : Session; Baud : Baud_Rate);
procedure Set_Data_Bits (S : Session; Bits : Data_Bits);
procedure Set_Parity (S : Session; Parity : Parity_Mode);
procedure Set_Stop_Bits (S : Session; Stop : Stop_Bits);
procedure Configure_Pins (S : Session; ...); -- re-route pads
procedure Enable_Loopback (S : Session; On : Boolean := True);
```

Parameters.

Port

UART0, UART1 or UART2.

Baud

bit rate (Baud_Rate, 300 . . 5_000_000); default 115_200.

Bits

data bits per frame (Data_Bits, 5 . . 8); default 8.

Parity

None (default), Even or Odd.

Stop

stop bits: One (default) or Two.

Tx, Rx, Rts, Cts

the four pads as Optional_Pin; all default to No_Pin (route only the lines the link uses).

Rx_Flow_Threshold

RX FIFO fill, in bytes, at which RTS de-asserts; default 100 (the FIFO holds 128).

On

(Enable_Loopback) True (default) folds TX back to RX internally.

the Set_* setters

each take the held Session and one new value of the matching Setup type (e.g. Set_Parity takes a Parity_Mode); Configure_Pins additionally accepts per-line *_Invert booleans (default False).

Setup both configures the frame format *and* routes the four pins (all optional — pass only what the link uses, or none for an internal loopback self-test). It is the one port-based call, meant to run once at startup before any task contends for the port. *Every* later change — baud, data bits, parity, stop bits, pin re-routing, loopback — takes a Session instead of a Port, so you must Acquire the port before you can reconfigure it. Changing a parameter therefore requires owning the port and can never race another

task's transfer. Each frame setter is an independent read-modify-write that leaves the other attributes and the pin routing untouched.

This rule is enforced *structurally*, not by convention. Inside the body, the per-port register handle lives in a nested package whose body hides it; the only way to obtain it is one ownership-checked gateway that raises `Not_Owned` unless the passed `Session` holds the port. Every transfer and every reconfiguration goes through that gateway, so a call without ownership fails loudly — and, just as importantly, a *new* operation added to the driver cannot reach the registers without passing a `Session` through the same check. The guard is impossible to forget because there is no other door to the hardware.

Example 1 — transmit a line.

```
with ESP32S3.UART; use ESP32S3.UART;
procedure UART_Tx is
  S : Session;
begin
  Setup (UART1, Baud => 115_200, Tx => 17, Rx => 16);  -- format + pins in one
  Acquire (S, UART1);
  Write (S, (Character'Pos ('h'), Character'Pos ('i'), 10));
end UART_Tx;
```

Example 2 — echo received bytes (wiring-free, internal loopback).

```
with ESP32S3.UART; use ESP32S3.UART;
procedure UART_Echo is
  S : Session;
  Buf : Byte_Array (0 .. 63);
  N : Natural;
begin
  Setup (UART1);
  Acquire (S, UART1);
  Enable_Loopback (S);  -- TX folded back to RX (held port)
  Write (S, (1 => 16#42#));
  loop
    exit when Available (S) > 0;
  end loop;
  Read (S, Buf, N);  -- Buf (0 .. N-1) holds the echoed bytes
end UART_Echo;
```

Example 2, step by step. This example is worth reading line by line, because it exercises every part of the `Session` pattern.

The declarations (before begin).

- `S : Session` — `Session` is a *limited controlled* type. *Limited* means it cannot be copied or assigned (two tasks can never share one hold); *controlled* means it has a `Finalize` that runs automatically when `S` leaves scope. It starts inactive — it owns nothing yet.
- `Buf : Byte_Array (0 .. 63)` — a 64-element array of `Byte` (which is new `Interfaces.Unsigned_8`); the destination for the bytes we read back.
- `N : Natural` — receives the count of bytes actually read.

The statements, in execution order.

1. `Setup (UART1)` brings `UART1` up with the default frame format (115200 baud, 8 data bits, no parity, one stop bit — the defaults on `Setup`). It opens the controller and records that `UART1` is configured. No pins are routed: we pass no pin arguments to `Setup` (they all default to `No_Pin`) because this is a wiring-free self-test.

2. `Acquire (S, UART1)` takes exclusive ownership of UART1. It first checks the init-before-use guard: had UART1 never been Setup, this would raise `Not_Initialized`. It then blocks on UART1's protected guard until the port is free (here, immediately), marks `S` active, and records that it holds UART1. Until `S` is released no other task can acquire UART1 — and, because every reconfiguration call below takes the held `Session`, no other task can change UART1's settings either.
3. `Enable_Loopback (S)` sets the controller's internal TX→RX loopback bit *on the held port*. Everything the transmitter shifts out is fed straight back into the receiver *inside the chip*. Because UART is push-pull (unlike I2C's open-drain bus) this loopback is faithful and needs *no external wiring and no GPIO pads* — the whole point of the example. It takes `S` (not UART1): you can only flip loopback on a port you own.
4. `Write (S, (1 => 16#42#))` — `(1 => 16#42#)` is a one-element array aggregate, a `Byte_Array` of length 1 holding the byte 16#42# (66, ASCII 'B'). `Write` pushes it into the TX FIFO and returns once it is queued. This runs *outside* the protected lock — the lock only arbitrated *ownership* in the previous step; the blocking I/O is never done while holding it. With loopback on, that byte will reappear in the RX FIFO shortly.
5. The poll loop `loop exit when Available (S) > 0; end loop;` spins, re-reading how many bytes sit in the RX FIFO, until at least one has arrived. The looped-back byte takes roughly one character-time ($\sim 87\mu\text{s}$ at 115200 baud), so the loop spins briefly then exits. (A self-test idiom; production code would use a timeout or an interrupt instead of a bare spin.)
6. `Read (S, Buf, N)` reads up to `Buf'Length` (64) bytes into `Buf` and sets `N` to how many it actually got — here `N = 1`, with `Buf (0) = 16#42#`. Elements `Buf (N .. 63)` are left untouched. `Read` waits only briefly per byte and does a short read on timeout, so it never hangs on bytes that never come.
7. At `end UART_Echo`, `S` goes out of scope. Being controlled, its `Finalize` runs automatically and calls `Release`, handing UART1 back to its guard. This happens *even if an exception had been raised* between `Acquire` and here — that is why the RAII handle exists: the lock cannot leak.

The two phases are deliberately separate. `Setup` (step 1) is the one port-based call, meant to run single-threaded at startup before any task contends for the port. `Acquire` (step 2) is the per-transaction ownership gate that hands you a `Session` — and everything after it, including `Enable_Loopback` here, acts through that `Session`, so a reconfiguration is only possible while you hold the port. Two guards tie the phases together: you cannot `Acquire` a port you never `Setup`, and you cannot change a port's parameters without holding it.

Example 3 — change baud and frame format independently. Each frame attribute has its own setter, so a protocol that renegotiates the link (or a bootloader that bumps the speed after handshaking) can change just one without re-running `Setup` or touching the pin routing. The setters take the held `Session`, so you `Acquire` the port first — the reconfiguration runs under the same ownership as the transfers:

```
with ESP32S3.UART; use ESP32S3.UART;
procedure UART_Reconfig is
  S : Session;
begin
  Setup (UART1, Tx => 17, Rx => 16);    -- 115200 8-N-1 to start
  Acquire (S, UART1);                 -- own the port before reconfiguring
  Set_Baud      (S, 9_600);             -- then change each attribute alone
  Set_Data_Bits (S, 7);
  Set_Parity    (S, Even);
  Set_Stop_Bits (S, Two);               -- now 9600 7-E-2
  Write (S, (1 => 16#55#));
```

```
end UART_Reconfig;
```

Chapter 11

DMA, PWM, Audio, Pulse and Analog Peripherals

These drivers follow the *claim-a-handle* pattern: a peripheral with several identical units hands out a limited, controlled handle through a protected pool. You Claim a unit, use it through the handle, and it is reclaimed when the handle leaves scope.

11.1 GDMA — general DMA

```
procedure Claim (C : in out Channel; Peri : Peripheral);
procedure Copy  (C : Channel; Dst, Src : System.Address; Length : Natural);
procedure Start (C : Channel; Dir : Direction; ...); -- peripheral transfers
function Done   (C : Channel; Dir : Direction) return Boolean;
procedure Wait  (C : Channel; Dir : Direction);
```

Parameters.

Peri

which peripheral the channel serves — Mem2Mem (a plain memory copy) or a real peripheral: SPI2, SPI3, UHCIO, I2S0, I2S1, LCD_CAM, AES, SHA, ADC_DAC, RMT. The pool has five channels.

Dir transfer direction: Mem_To_Periph or Periph_To_Mem.

Dst, Src, Length

destination/source 'Addresses and a byte count, 1 .. 4095.

Example 1 — memory-to-memory copy.

```
with Interfaces;
with ESP32S3.GDMA; use ESP32S3.GDMA;
procedure GDMA_Copy is
  C : Channel;
  Src : aliased array (0 .. 255) of Interfaces.Unsigned_8 := (others => 16#5A#);
  Dst : aliased array (0 .. 255) of Interfaces.Unsigned_8 := (others => 0);
begin
  Claim (C, Peri => Mem2Mem); -- a free channel for mem2mem
  Copy (C, Dst'Address, Src'Address, 256); -- blocking DMA copy; Dst = Src
end GDMA_Copy; -- C is released on scope exit
```

Example 2 — the RAII guarantee. Claiming all five channels and then one more raises (the pool is exhausted); each goes back automatically:

```

with ESP32S3.GDMA; use ESP32S3.GDMA;
procedure GDMA_RAII is
  A, B, C2, D, E : Channel;
begin
  Claim (A, Mem2Mem); Claim (B, Mem2Mem); Claim (C2, Mem2Mem);
  Claim (D, Mem2Mem); Claim (E, Mem2Mem);           -- all five claimed
end GDMA_RAII;                                     -- all five reclaimed here

```

(The SPI and I2S drivers use the peripheral Start/Wait path internally, so most applications use GDMA only for memory copies.)

11.2 MCPWM — motor-control PWM

```

procedure Setup (Unit : MCPWM_Unit);
procedure Claim (C : in out Channel; Unit : MCPWM_Unit; Index : Channel_Index);
procedure Configure_Channel (C : Channel; Freq : Positive; Pin : Pin_Id;
                             Complement_Pin : Optional_Pin := No_Pin;
                             Dead_Time_Ns   : Natural := 0);
procedure Start (C : Channel);   procedure Set_Duty (C : Channel; Percent :
Duty_Percent);

```

Parameters.

Unit

MCPWM0 or MCPWM1.

Index

generator channel within the unit: Ch0, Ch1 or Ch2.

Freq

PWM frequency in Hz.

Pin the output pad.

Complement_Pin

optional inverted output for a half-bridge (Optional_Pin, default No_Pin).

Dead_Time_Ns

gap inserted between the complementary edges, in nanoseconds; default 0.

Percent

duty cycle, 0.0 .. 100.0.

Example 1 — 20 kHz at 25 % on one pin.

```

with ESP32S3.MCPWM; use ESP32S3.MCPWM;
procedure MCPWM_Simple is
  C : Channel;
begin
  Setup (MCPWM0);
  Claim (C, MCPWM0, Ch0);
  Configure_Channel (C, Freq => 20_000, Pin => 4);
  Set_Duty (C, 25.0);
  Start (C);
end MCPWM_Simple;

```

Example 2 — a complementary pair with 500 ns dead-time (half-bridge).

```

with ESP32S3.MCPWM; use ESP32S3.MCPWM;
procedure MCPWM_Pair is
  C : Channel;
begin
  Setup (MCPWM0);
  Claim (C, MCPWM0, Ch0);
  Configure_Channel (C, Freq => 20_000, Pin => 4,
                    Complement_Pin => 5, Dead_Time_Ns => 500);
  Set_Duty (C, 60.0);
  Start (C);
end MCPWM_Pair;

```

The driver also offers a carrier (Set_Carrier — Prescale 0 .. 15, Duty_Eighths 1 .. 7, First_Pulse 0 .. 15) and fault/trip-zone protection: Configure_Fault binds a Fault_Input (Fault0 .. 2) to a pad, and Protect_Channel sets the Fault_Mode (One_Shot or Cycle_By_Cycle) and Trip_Action (No_Change, Force_Low or Force_High).

11.3 I2S — digital audio over GDMA

```

type I2S_Mode is (Standard, PDM);
procedure Setup (Port : I2S_Port; Sample_Rate : Positive := 16_000;
                Bits : Sample_Bits := Bits_16; Mode : I2S_Mode := Standard);
procedure Configure_Pins (Port : I2S_Port;
                        Bclk, Ws, Dout, Din : Optional_Pin := No_Pin);
procedure Acquire (S : in out Session; Port : I2S_Port);
procedure Write (S : Session; Tx : System.Address; Length : Natural);
procedure Read (S : Session; Rx : System.Address; Length : Natural);
procedure Transfer (S : Session; Tx, Rx : System.Address; Length : Natural);

```

Parameters.

Port

I2S0 or I2S1.

Sample_Rate

frames per second (Positive); default 16_000.

Bits

sample width: Bits_8, Bits_16 (default), Bits_24 or Bits_32.

Mode

data path: Standard (default — plain I2S/TDM) or PDM (the sigma-delta converters; see below).

Bclk, Ws, Dout, Din

the bit-clock, word-select, data-out and data-in lines (Optional_Pin; Dout/Din default No_Pin).

Tx, Rx, Length

buffer 'Addresses and a byte count, 1 .. 4095.

Example 1 — play a buffer of samples.

```

with Interfaces;
with ESP32S3.I2S; use ESP32S3.I2S;
procedure I2S_Play is
  S : Session;
  Tone : aliased array (0 .. 255) of Interfaces.Integer_16 := (others => 0);
begin

```

```

Setup (I2S0, Sample_Rate => 44_100, Bits => Bits_16);
Configure_Pins (I2S0, Bclk => 4, Ws => 5, Dout => 6);
Acquire (S, I2S0);
Write (S, Tone'Address, Tone'Length * 2); -- 2 bytes per 16-bit sample
end I2S_Play;

```

Example 2 — wiring-free full-duplex loopback self-test.

```

with Interfaces;
with ESP32S3.I2S; use ESP32S3.I2S;
procedure I2S_Loopback is
  S : Session;
  Tx : aliased array (0 .. 63) of Interfaces.Unsigned_8 := (others => 16#5A#);
  Rx : aliased array (0 .. 63) of Interfaces.Unsigned_8 := (others => 0);
begin
  Setup (I2S0);
  Enable_Loopback (I2S0, Pad => 7); -- TX folded to RX on one pad
  Acquire (S, I2S0);
  Transfer (S, Tx'Address, Rx'Address, 64); -- Rx = Tx
end I2S_Loopback;

```

11.3.1 PDM mode: the hardware PCM ↔ PDM converters

Mode => PDM routes the data path through the sigma-delta converters instead of the plain serial framer. PDM (pulse-density modulation) is the sigma-delta sibling of PWM: a 1-bit oversampled stream whose *density* of ones encodes the analog level. The S3 I2S converts in both directions, and the DMA still moves ordinary PCM, so only the on-wire format changes:

- **TX** (PCM2PDM) turns each PCM sample into a 1-bit stream — feed it to a class-D amplifier or an RC low-pass for analog audio out (the S3 has no analog DAC, so this is how you get one).
- **RX** (PDM2PCM) decimates a 1-bit PDM input back to PCM. PDM is the format *emitted* by digital **PDM microphones** and some PDM-output **ADCs**; the converter turns their bitstream into the PCM samples your program reads. In PDM the receiver is the clock master: it drives the device clock out on Ws and reads the bitstream in on Din.

```

with Interfaces;
with ESP32S3.I2S; use ESP32S3.I2S;
procedure PDM_Mic is
  S : Session;
  Rx : aliased array (0 .. 511) of Interfaces.Integer_16 := (others => 0);
begin
  Setup (I2S0, Sample_Rate => 16_000, Bits => Bits_16, Mode => PDM);
  Configure_Pins (I2S0, Bclk => No_Pin, Ws => 5, Dout => No_Pin, Din => 6);
  Acquire (S, I2S0);
  Read (S, Rx'Address, Rx'Length * 2); -- PDM mic -> PCM
end PDM_Mic;

```

A note on verifying PDM. Every other HAL driver self-tests with no external hardware; PDM is the exception, and the reason is instructive. There is no on-chip loopback for the converters — the SIG_LOOPBACK bit only shares the *standard*-I2S WS and BCK, not the bit timing between PCM2PDM and PDM2PCM — so an internal PCM → PDM → PCM loop recovers nothing. Nor can a static level synthesised from a GPIO (a pull resistor or a driven pin) stand in for a mic: the decimator’s mandatory high-pass filter strips DC, so a constant 1 or 0 is rejected. Genuine PDM verification needs a real device producing a toggling bitstream, so the esp32s3_i2s_pdm example is a microphone-capture demo rather than a self-test.

11.4 LEDC — LED / general PWM

```
procedure Claim (C : in out Channel; Index : Channel_Index);
procedure Configure (C : in out Channel; Freq : Positive; Pin : Pin_Id;
                    Bits : Resolution := 10);
procedure Set_Duty (C : Channel; Percent : Duty_Percent);
```

Parameters.

Index

channel 0 .. 7 (Channel_Index).

Freq

PWM frequency in Hz.

Pin the output pad.

Bits

duty resolution in bits, 1 .. 14 (Resolution); default 10. More bits give finer duty steps but a lower maximum frequency.

Percent

duty cycle, 0.0 .. 100.0.

Example 1 — 5 kHz at 50 %.

```
with ESP32S3.LEDC; use ESP32S3.LEDC;
procedure LEDC_Simple is
  C : Channel;
begin
  Claim (C, 0);
  Configure (C, Freq => 5_000, Pin => 2);
  Set_Duty (C, 50.0);
end LEDC_Simple;
```

Example 2 — fade an LED.

```
with Ada.Real_Time; use Ada.Real_Time;
with ESP32S3.LEDC; use ESP32S3.LEDC;
procedure LEDC_Fade is
  C : Channel;
begin
  Claim (C, 0);
  Configure (C, Freq => 5_000, Pin => 2);
  for D in 0 .. 100 loop
    Set_Duty (C, Duty_Percent (D));
    delay until Clock + Milliseconds (10);
  end loop;
end LEDC_Fade;
```

11.5 RMT — remote-control pulse TX/RX (IR, WS2812)

```
type RMT_Symbol is record
  Level0, Level1 : Boolean := False; Duration0, Duration1 : Tick_Count := 0;
end record;
procedure Configure (C : in out TX_Channel; Resolution_Hz : Positive; Pin : Pin_Id);
procedure Transmit (C : TX_Channel; Symbols : Symbol_Array);
```

```

procedure Configure (C : in out RX_Channel; Resolution_Hz : Positive; Pin : Pin_Id);
procedure Start (C : RX_Channel);
procedure Receive (C : RX_Channel; Into : out Symbol_Array; Count : out Natural);

```

Parameters.

Index

channel 0 .. 3 — four TX channels (TX_Index) and four RX (RX_Index), separate pools.

Resolution_Hz

the tick clock: one Tick_Count unit lasts 1/Resolution_Hz seconds.

Pin the carrier pad.

Idle_Ticks

(RX) the silence that ends a frame, 0 .. 32_767; default 2_000.

Symbols / Into

a Symbol_Array of RMT_Symbols, each holding two level+duration halves; Duration0/Duration1 are Tick_Count (0 .. 32_767). One TX burst is at most 47 symbols.

Count

(Receive) out count of symbols captured.

Example 1 — transmit a pulse burst (1 μ s resolution).

```

with ESP32S3.RMT; use ESP32S3.RMT;
procedure RMT_Tx is
  Tx      : TX_Channel;
  Burst : constant Symbol_Array :=
    ((Level0 => True,  Duration0 => 10, Level1 => False, Duration1 => 10),
     (Level0 => True,  Duration0 => 20, Level1 => False, Duration1 => 20));
begin
  Claim (Tx, 0);
  Configure (Tx, Resolution_Hz => 1_000_000, Pin => 4);
  Transmit (Tx, Burst);
end RMT_Tx;

```

Example 2 — receive into a buffer (e.g. an IR remote).

```

with ESP32S3.RMT; use ESP32S3.RMT;
procedure RMT_Rx is
  Rx      : RX_Channel;
  Syms : Symbol_Array (0 .. 63);
  N      : Natural;
begin
  Claim (Rx, 0);
  Configure (Rx, Resolution_Hz => 1_000_000, Pin => 5);
  Start (Rx);
  Receive (Rx, Syms, N);  -- Syms (0 .. N-1) are the captured pulses
end RMT_Rx;

```

11.6 PCNT — pulse / edge counter

```

procedure Configure (U : in out Unit; Pin : Pin_Id; Both_Edges : Boolean := False);
function Count (U : Unit) return Integer;  procedure Clear (U : Unit);

```

Parameters.

Index

unit 0 .. 3 (Unit_Index).

Pin the input pad whose edges are counted.

Both_Edges

False (default) counts rising edges only; True counts both (double resolution).

Count

returns the signed counter (16-bit, wraps at ± 32768).**Example 1 — count rising edges on a pin.**

```

with Ada.Text_IO; use Ada.Text_IO;
with ESP32S3.PCNT; use ESP32S3.PCNT;
procedure PCNT_Count is
  U : Unit;
begin
  Claim (U, 0);
  Configure (U, Pin => 6);
  Clear (U);
  -- ... pulses arrive ...
  Put_Line ("edges =" & Integer'Image (Count (U)));
end PCNT_Count;

```

Example 2 — count both edges (double the resolution).

```

with ESP32S3.PCNT; use ESP32S3.PCNT;
procedure PCNT_Both is
  U : Unit;
begin
  Claim (U, 0);
  Configure (U, Pin => 6, Both_Edges => True);
end PCNT_Both;

```

11.7 SDM — sigma-delta density output

```

procedure Configure (C : in out Channel; Pin : Pin_Id;
                    Carrier_Hz : Positive := 1_000_000);
procedure Set_Density (C : Channel; Percent : Density_Percent);

```

Parameters.

Index

channel 0 .. 7 (Channel_Index).

Pin the output pad.

Carrier_Hz

the desired pulse-stream (carrier) frequency in Hz; default 1_000_000. The modulator divides the APB clock (≈ 80 MHz) by an integer 1 .. 256, so the achieved rate is the nearest APB/N — roughly 312_500 .. 80_000_000 Hz, rounded for you. A higher carrier sits further above your RC low-pass cutoff and smooths to a cleaner level; the exact value rarely matters.

Percent

target density, 0.0 .. 100.0.

Example 1 — a 25 % density bitstream (analog-ish level via an RC filter).

```

with ESP32S3.SDM; use ESP32S3.SDM;
procedure SDM_Density is
  C : Channel;
begin
  Claim (C, 0);
  Configure (C, Pin => 7, Carrier_Hz => 1_000_000);
  Set_Density (C, 25.0);
end SDM_Density;

```

Example 2 — breathe an LED by sweeping density.

```

with Ada.Real_Time; use Ada.Real_Time;
with ESP32S3.SDM; use ESP32S3.SDM;
procedure SDM_Breathe is
  C : Channel;
begin
  Claim (C, 0);
  Configure (C, Pin => 7);
  for P in 0 .. 100 loop
    Set_Density (C, Density_Percent (P));
    delay until Clock + Milliseconds (10);
  end loop;
end SDM_Breathe;

```

11.8 TWAI — CAN 2.0

```

subtype Standard_Id is Interfaces.Unsigned_32 range 0 .. 16#7FF#;      -- 11-bit
subtype Extended_Id is Interfaces.Unsigned_32 range 0 .. 16#1FFF_FFFF#; -- 29-bit
type Standard_Frame is record -- 11-bit id (CAN 2.0A)
  Id : Standard_Id := 0; Remote : Boolean := False; -- Remote => RTR request
  Length : Data_Length := 0; Data : Data_Bytes := (others => 0);
end record;
type Extended_Frame is record -- 29-bit id (CAN 2.0B)
  Id : Extended_Id := 0; Remote : Boolean := False;
  Length : Data_Length := 0; Data : Data_Bytes := (others => 0);
end record;
procedure Setup (Mode : Bus_Mode := Normal; Bit_Rate : Positive := 125_000);
procedure Acquire (S : in out Session);
procedure Send (S : Session; F : Standard_Frame); -- overloaded on the frame type
procedure Send (S : Session; F : Extended_Frame);
function Available (S : Session) return Boolean; -- a frame is waiting
function Is_Extended (S : Session) return Boolean; -- ... and it is 29-bit
procedure Receive (S : Session; F : out Standard_Frame; Got : out Boolean);
procedure Receive (S : Session; F : out Extended_Frame; Got : out Boolean);

```

Parameters.**Mode**

Normal (default), Listen_Only (receive without ACKing) or Self_Test (ACK its own frames — the wiring-free loopback).

Bit_Rate

bus speed in bits/s; default 125_000 (e.g. 250_000, 500_000, 1_000_000).

Tx, Rx

the bus pads (Optional_Pin).

F (Standard_Frame / Extended_Frame)

the frame: Id (range-checked to its width — Standard_Id 0 .. 16#7FF# or Extended_Id 0 .. 16#1FFF_FFFF#), Length (0 .. 8 bytes), Data (an 8-byte array), and Remote — False (default) for a data frame, True for a remote-transmission request (RTR), which carries Id and Length but no Data. Send is overloaded on the frame type.

Got (Receive) out Boolean — True if a frame of the requested width was read.

Pad (Enable_Loopback) the single self-test pad.

Standard vs. extended frames. A CAN identifier is either 11-bit (standard) or 29-bit (extended, CAN 2.0B). The two are distinct frame types, so the id is range-checked to its width and Send is overloaded — you cannot put a 29-bit id in a standard frame. On receive the *sender* picks the width, so you peek first: Available reports a waiting frame, Is_Extended reports its width, and you call the matching Receive overload (Example 2). Orthogonal to the width, Remote marks a *remote-transmission request* (RTR): a frame that carries an id and a length but no data, asking whichever node owns that id to reply with the data. Set it on either frame type; a received frame reports it.

Example 1 — send a standard and an extended frame.

```
with ESP32S3.TWAI; use ESP32S3.TWAI;
procedure TWAI_Send is
  S : Session;
begin
  Setup (Mode => Normal, Bit_Rate => 500_000);
  Acquire (S); -- own the controller, then route pins
  Configure_Pins (S, Tx => 4, Rx => 5);
  Send (S, Standard_Frame'(Id => 16#123#, Remote => False, Length => 2,
    Data => (16#DE#, 16#AD#, others => 0)));
  Send (S, Extended_Frame'(Id => 16#14AB_CDE#, Remote => False, Length => 1,
    Data => (0 => 16#42#, others => 0))); -- type picks

  Send
  Send (S, Standard_Frame'(Id => 16#7A5#, Remote => True, -- RTR: request
    Length => 8, Data => (others => 0))); -- 8 bytes, none

  sent
end TWAI_Send;
```

Example 2 — self-test loopback (no transceiver, no wiring).

```
with ESP32S3.TWAI; use ESP32S3.TWAI;
procedure TWAI_Selftest is
  S : Session;
  RE : Extended_Frame;
  Got : Boolean := False;
begin
  Setup (Mode => Self_Test);
  Acquire (S);
  Enable_Loopback (S, Pad => 4);
  Send (S, Extended_Frame'(Id => 16#14AB_CDE#, Remote => False, Length => 1,
    Data => (0 => 16#42#, others => 0)));
  if Available (S) and then Is_Extended (S) then
    Receive (S, RE, Got); -- Got, RE.Id = 16#14AB_CDE#, RE.Remote = False
  end if;
end TWAI_Selftest;
```

11.9 Timer — general-purpose timers

```

procedure Configure (T : in out Timer; Tick_Hz : Positive := 1_000_000);
procedure Start (T : Timer); function Value (T : Timer) return Ticks;
procedure Set_Alarm (T : Timer; At_Ticks : Ticks);
function Alarm_Fired (T : Timer) return Boolean; procedure Clear_Alarm (T : Timer);

```

Parameters.

Index

0 or 1 (Timer_Index — the two timer groups TIMG0/TIMG1).

Tick_Hz

counter rate in Hz; default 1_000_000 (1 tick = 1 μ s).

At_Ticks / Value

a 54-bit count (Ticks, an Unsigned_64); Set_Alarm fires when the counter reaches At_Ticks.

Example 1 — measure an elapsed interval in microseconds.

```

with Ada.Text_IO; use Ada.Text_IO;
with ESP32S3.Timer; use ESP32S3.Timer;
procedure Timer_Measure is
  T : Timer;
  Start_T : Ticks;
begin
  Claim (T, 0);
  Configure (T, Tick_Hz => 1_000_000); -- 1 tick = 1 us
  Start (T);
  Start_T := Value (T);
  -- ... work ...
  Put_Line ("elapsed us =" & Ticks'Image (Value (T) - Start_T));
end Timer_Measure;

```

Example 2 — a one-shot alarm 50 ms out.

```

with ESP32S3.Timer; use ESP32S3.Timer;
procedure Timer_Alarm is
  T : Timer;
begin
  Claim (T, 0);
  Configure (T, Tick_Hz => 1_000_000);
  Start (T);
  Set_Alarm (T, Value (T) + 50_000);
  loop exit when Alarm_Fired (T); end loop;
  Clear_Alarm (T);
end Timer_Alarm;

```

11.10 LCD — 8-bit i80 parallel master

```

procedure Setup (Pclk_Hz : Positive := 1_000_000);
procedure Configure_Pins (Data : Data_Pins; Pclk : Optional_Pin);
procedure Acquire (S : in out Session);
procedure Transmit (S : Session; Tx : System.Address; Length : Natural;
                   Ok : out Boolean);

```

Parameters.**Pclk_Hz**

pixel clock in Hz; default 1_000_000 (the achieved value is 20 MHz divided by an integer).

Data

a Data_Pins value — an 8-element array of Optional_Pin for the data lines D0..D7.

Pclk / Pclk_Pad

the pixel-clock pad.

Tx, Length

frame-buffer 'Address and a byte count, 1 .. 4095.

Ok out Boolean — True on a completed transfer.**Example 1 — push a frame buffer to a parallel display.**

```

with Interfaces;
with ESP32S3.LCD; use ESP32S3.LCD;
procedure LCD_Frame is
  S : Session;
  Ok : Boolean;
  FB : aliased array (0 .. 3999) of Interfaces.Unsigned_8 := (others => 16#FF#);
begin
  Setup (Pclk_Hz => 200_000);
  Configure_Pins (Data => (1, 2, 3, 4, 5, 6, 7, 8), Pclk => 9);
  Acquire (S);
  Transmit (S, FB'Address, FB'Length, Ok);
end LCD_Frame;

```

Example 2 — expose the pixel clock on a pad (for a scope).

```

with ESP32S3.LCD; use ESP32S3.LCD;
procedure LCD_Clk is
begin
  Enable_Clock_Out (Pclk_Pad => 9);
end LCD_Clk;

```

11.11 ADC — SAR analog input

```

procedure Claim (R : in out Reader; Unit : ADC_Unit);
function Read (R : Reader; Ch : Channel_Index; Atten : Attenuation := Db_12)
  return Raw_Value;
-- 0 .. 4095 (12-bit)

```

Parameters.**Unit**

ADC1 or ADC2.

Ch channel 0 .. 9 (Channel_Index); each maps to a fixed pad.**Atten**

input attenuation, trading range for resolution: Db_0 (~1.1 V full scale), Db_2_5 (~1.5 V), Db_6 (~2.2 V) or Db_12 (~3.3 V, default).

returns

a Raw_Value, 0 .. 4095 (12-bit).

Example 1 — read a 12-bit sample.

```
with Ada.Text_IO; use Ada.Text_IO;
with ESP32S3.ADC; use ESP32S3.ADC;
procedure ADC_Read_Sample is
  R : Reader;
  V : Raw_Value;
begin
  Claim (R, ADC1);
  V := Read (R, Ch => 0);      -- 0 .. 4095 at full (12 dB) attenuation
  Put_Line (Raw_Value'Image (V));
end ADC_Read_Sample;
```

Example 2 — a lower attenuation for a small signal.

```
with Ada.Text_IO; use Ada.Text_IO;
with ESP32S3.ADC; use ESP32S3.ADC;
procedure ADC_Atten is
  R : Reader;
  V : Raw_Value;
begin
  Claim (R, ADC1);
  V := Read (R, Ch => 3, Atten => Db_0);  -- ~0..1.1 V full scale
  Put_Line (Raw_Value'Image (V));
end ADC_Atten;
```

Chapter 12

Low-Power, Touch and Crypto

These drivers need no finalization, so they build under *every* runtime profile.

12.1 RTC — deep sleep and retained memory

The RTC domain keeps a small slow-memory block alive across deep sleep, and drives the wake timer. Read/Write access retained 32-bit words; Deep_Sleep_For/Until enter deep sleep (the chip resets on wake, so execution resumes at the top of Main); Last_Wake reports why it woke.

```
function Read (Index : Word_Index) return Interfaces.Unsigned_32;  
procedure Write (Index : Word_Index; Value : Interfaces.Unsigned_32);  
function Last_Wake return Wake_Cause;    -- Power_On / Deep_Sleep_Timer / ...  
procedure Deep_Sleep_For (Wake_After : Duration);  
procedure Deep_Sleep_Until (Pin : Pin_Id; High : Boolean := True);
```

Parameters.

Index

retained-word slot, 0 .. 2047 (Word_Index — the 8 KB RTC slow memory as 32-bit words).

Value

the 32-bit word to store (Unsigned_32).

Wake_After

(Deep_Sleep_For) the sleep length as an Ada Duration in *seconds*; approximate, since the RTC slow clock is uncalibrated.

Pin, High

(Deep_Sleep_Until) the wake pad and the level to wake on; High => True (default) wakes on a high level.

Last_Wake

returns Power_On, Deep_Sleep_Timer, Deep_Sleep_GPIO or Other_Reset.

Example 1 — a boot counter that survives deep sleep.

```
with Interfaces;      use Interfaces;  
with Ada.Text_IO;    use Ada.Text_IO;  
with ESP32S3.RTC;    use ESP32S3.RTC;  
procedure RTC_Boot is  
  N : Interfaces.Unsigned_32;  
begin
```

```

if Last_Wake = Deep_Sleep_Timer then
    N := Read (0) + 1;      -- woke from our own timer
else
    N := 1;                -- cold boot
end if;
Write (0, N);
Put_Line ("boot #" & N'Image);
Deep_Sleep_For (5.0);      -- Duration in seconds; wakes and re-enters Main
end RTC_Boot;

```

Example 2 — wake on a button.

```

with ESP32S3.RTC; use ESP32S3.RTC;
procedure RTC_Wake is
begin
    Deep_Sleep_Until (Pin => 0, High => False); -- sleep until GPIO0 goes low
end RTC_Wake;

```

(After a deep-sleep wake the driver also disarms the super-watchdog — `Disable_Super_Watchdog` — so a staying-awake app is not reset.)

12.2 RTC_IO — pad hold and RTC pulls

Latch a pad's level so it survives GPIO writes and deep sleep, and apply RTC-domain pulls.

```

procedure Hold (Pin : RTC_Pin);   procedure Release (Pin : RTC_Pin);
function Is_Held (Pin : RTC_Pin) return Boolean;
procedure Set_Pull (Pin : RTC_Pin; Mode : Pull_Mode); -- No_Pull / Up / Down

```

Parameters.

`Pin` an RTC-capable pad — `RTC_Pin`, restricted to 0 . . 21 (only those reach the RTC domain).

`Mode`

(`Set_Pull`) the RTC-domain pull: `No_Pull`, `Up` or `Down`.

Example 1 — hold a relay line high through sleep.

```

with ESP32S3.GPIO;
with ESP32S3.RTC_IO; use ESP32S3.RTC_IO;
procedure RTCIO_Hold is
begin
    ESP32S3.GPIO.Configure (5, Mode => ESP32S3.GPIO.Output);
    ESP32S3.GPIO.Set (5);
    Hold (5);                -- GPIO writes now ignored; level frozen
    -- ... deep sleep ...    -- pad stays high across sleep
    Release (5);             -- GPIO control restored
end RTCIO_Hold;

```

Example 2 — an RTC pull-up on a wake pin.

```

with ESP32S3.RTC_IO; use ESP32S3.RTC_IO;
procedure RTCIO_Pull is
begin
    Set_Pull (0, Up);
end RTCIO_Pull;

```

12.3 Touch — capacitive sensing

```

procedure Setup;                                -- bring up the touch FSM (call once)
procedure Enable (Ch : Channel);                -- channel n is on GPIO n (n = 1 .. 14)
function Read (Ch : Channel) return Natural;    -- raw self-capacitance count
function Touched (Ch : Channel; Reference : Natural; Margin : Natural := 20_000)
  return Boolean;

```

Parameters.

Ch touch channel 1 .. 14 (Channel); channel *n* is wired to GPIO *n*.

Read

returns a raw self-capacitance count (Natural — larger means more capacitance; 0 means not measured yet).

Reference

(Touched) the untouched baseline count you captured earlier with Read.

Margin

how far the live count must deviate from Reference to register a touch; default 20_000.

Example 1 — read two pads' baselines.

```

with Ada.Text_IO; use Ada.Text_IO;
with ESP32S3.Touch; use ESP32S3.Touch;
procedure Touch_Read_Base is
begin
  Setup; Enable (1); Enable (3);
  Put_Line ("ch1 =" & Natural'Image (Read (1)) &
            " ch3 =" & Natural'Image (Read (3)));
end Touch_Read_Base;

```

Example 2 — detect a finger against a captured baseline.

```

with Ada.Text_IO; use Ada.Text_IO;
with ESP32S3.Touch; use ESP32S3.Touch;
procedure Touch_Detect is
  Base : constant Natural := Read (1); -- with nothing touching
begin
  loop
    if Touched (1, Base, Margin => 50_000) then
      Put_Line ("touched!");
    end if;
  end loop;
end Touch_Detect;

```

12.4 SHA — hardware hashing

```

function Hash_1 (Data : Byte_Array) return SHA1_Digest; -- 20 bytes
function Hash_224 (Data : Byte_Array) return SHA224_Digest; -- 28 bytes
function Hash_256 (Data : Byte_Array) return SHA256_Digest; -- 32 bytes

```

Parameters. Each Hash_* takes one Data : Byte_Array of any length and returns a fixed-size digest — SHA1_Digest (20 bytes), SHA224_Digest (28) or SHA256_Digest (32).

Example 1 — SHA-256 of a message.

```

with ESP32S3.SHA; use ESP32S3.SHA;
procedure SHA_256_Ex is
    Msg      : constant Byte_Array := (16#61#, 16#62#, 16#63#); -- "abc"
    Digest   : constant SHA256_Digest := Hash_256 (Msg);
begin
    null; -- Digest is the FIPS-180 vector for "abc"
end SHA_256_Ex;

```

Example 2 — SHA-1 (e.g. for a legacy checksum).

```

with ESP32S3.SHA; use ESP32S3.SHA;
procedure SHA_1_Ex is
    Msg : constant Byte_Array := (16#61#, 16#62#, 16#63#);
    D1  : constant SHA1_Digest := Hash_1 (Msg);
begin
    null;
end SHA_1_Ex;

```

The accelerator is a shared resource serialised by a protected object, so concurrent Hash calls from different tasks are safe.

12.5 AES — hardware block cipher

```

subtype Key_128 is Key_Bytes (0 .. 15); subtype Key_256 is Key_Bytes (0 .. 31);
function Encrypt_ECB (Key : Key_Bytes; Plain : Block) return Block
    with Pre => Supported_Key (Key); -- 16- or 32-byte keys only
function Decrypt_ECB (Key : Key_Bytes; Cipher : Block) return Block
    with Pre => Supported_Key (Key);

```

Parameters.

Key the cipher key (Key_Bytes); its *length* selects the cipher — 16 bytes (Key_128) for AES-128, 32 bytes (Key_256) for AES-256. The Supported_Key precondition rejects any other length (there is no AES-192 on this silicon).

Plain / Cipher

one 16-byte Block to transform; the result is a Block.

Example 1 — AES-128 encrypt then decrypt one block.

```

with ESP32S3.AES; use ESP32S3.AES;
procedure AES_128_Ex is
    Key      : constant Key_128 := (16#00#, 16#01#, 16#02#, 16#03#, 16#04#, 16#05#,
                                     16#06#, 16#07#, 16#08#, 16#09#, 16#0A#, 16#0B#,
                                     16#0C#, 16#0D#, 16#0E#, 16#0F#);
    Plain    : constant Block := (others => 16#11#);
    Cipher   : constant Block := Encrypt_ECB (Key, Plain);
    Back     : constant Block := Decrypt_ECB (Key, Cipher); -- Back = Plain
begin
    null;
end AES_128_Ex;

```

Example 2 — AES-256 (a 32-byte key selects the cipher automatically).

```

with ESP32S3.AES; use ESP32S3.AES;
procedure AES_256_Ex is

```

```
Key256 : constant Key_256 := (others => 16#2B#);
Plain  : constant Block := (others => 16#11#);
C      : constant Block := Encrypt_ECB (Key256, Plain);
begin
  null;
end AES_256_Ex;
```

The key parameter is one unconstrained type; its *length* (16 or 32 bytes) chooses AES-128 or AES-256, and the `Supported_Key` precondition rejects anything else — the contract makes the unsupported AES-192 (absent on this silicon) a compile/run-time error rather than a silent fallback.

Chapter 13

Writing a Task-Safe Peripheral Driver

Chapters 10–12 *used* the drivers. This chapter shows how to *write* one, and — just as important — why it is built the way it is. The shape described here is a project rule: every driver in the HAL for a peripheral that is **exclusive** (cannot be shared between tasks) and can be **reconfigured after setup** follows it exactly. By the end you should be able to add a new one and get the same guarantees for free.

13.1 The problem

Consider a typical controller — a SPI host, a UART, a CAN controller. Three facts make it dangerous to share:

1. **It is one physical resource.** Two tasks driving the same FIFO or the same configuration registers at once corrupt each other's transfers.
2. **It is configured in stages.** You bring it up once (clocks, frame format), then later you may re-route pins, change the baud rate, flip a line's polarity — often interleaved with real traffic.
3. **The registers are global.** On a bare-metal target the peripheral is a fixed memory-mapped block. Any code that can name it can poke it.

A naive driver exports `Configure`, `Write`, `Read` as plain procedures over a global. Nothing stops task B from calling `Set_Baud` while task A is mid-`Write`, or from starting a transfer it has no business starting. The corruption is intermittent and miserable to debug. We want a design where those mistakes are *impossible to write*, not merely discouraged.

13.2 The ownership model

The cure is to make *ownership* a value you hold, and to make every operation that touches the live controller require it.

A Session is a capability.

Acquire hands a task a *Session* — a limited, controlled handle that represents exclusive ownership of the controller. “Limited” means it cannot be copied (two tasks can never share one), and “controlled” means it releases automatically when it leaves scope, even on an exception. While you hold it, no other task can acquire the controller.

Three things become impossible.

You cannot acquire a controller that was never set up (`Not_Initialized`); you cannot transfer or reconfigure without holding it (`Not_Owned`); and you cannot leak the lock, because the handle's finalizer always releases.

The `Session` is the public face. The interesting part is the machinery behind it that makes “you cannot reconfigure without holding it” a structural fact rather than a rule a future maintainer must remember.

13.3 Three layers

A compliant driver is three packages:

1. **The Engine** (`ESP32S3.<Drv>.Engine`, a *private child*). It owns *all* register access and exposes a `Bus` handle. It is the only unit in the whole program that names the peripheral’s registers.
2. **The State gateway** (a package nested in the parent’s *body*). It holds the one configured `Bus` handle and hands it out through a single function, `Owned (S)`, which raises unless `S` holds the controller.
3. **The public package** (`ESP32S3.<Drv>`). It declares the `Session`, the exceptions, `Setup`, `Acquire`, and the operations — each of which is a one-liner that reaches the hardware only through `Owned`.

The rest of this chapter builds a small driver, `ESP32S3.Widget`, one layer at a time.

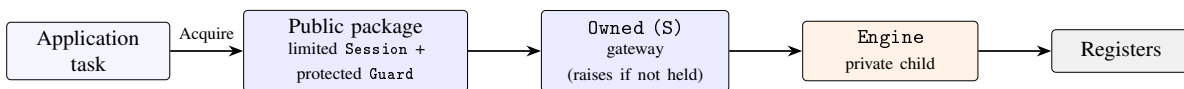


Figure 13.1: The task-safe driver skeleton: a transfer reaches the registers only via the ownership-checked `Owned` gateway, and only the private `Engine` names them. Forgetting the check is structurally impossible.

13.4 Layer 1 — the Engine (the only code that sees the registers)

The engine is a *private child*: a unit named `ESP32S3.Widget.Engine` can be with-ed only from inside the `ESP32S3.Widget` subtree. Application code — and even other drivers — physically cannot reach it.

```

private package ESP32S3.Widget.Engine is
  type Bus is private;                                -- an opaque, configured controller

  function Open (Clock_Hz : Positive) return Bus;
  procedure Configure_Pins (B : Bus; Tx, Rx : ESP32S3.GPIO.Optional_Pin);
  procedure Transfer (B : Bus; Tx, Rx : System.Address; Length : Natural);
private
  type Bus is record
    Regs : Periph_Ref := null;                        -- pointer to the register block
    Valid : Boolean := False;
  end record;
end ESP32S3.Widget.Engine;
  
```

The engine *body* is the one place that withs the SVD register package. It does the bit-twiddling — the same code you would have written anyway — but now it is sealed behind the `Bus` handle. `Open` brings the controller up and returns a `Bus`; every other operation takes a `Bus` and works through it. Crucially, **the parent package never withs the SVD registers at all** — so once the engine exists, the global controller is unreachable from anywhere except through a `Bus` handle.

13.5 Layer 2 — ownership: the Session and the Guard

The public package declares the handle and a per-controller lock. The `Session` is limited and controlled:

```

package ESP32S3.Widget is
  type Session is limited private;

  Not_Initialized : exception;  -- Acquire before Setup
  Not_Owned       : exception;  -- use without holding the controller

  procedure Setup (Clock_Hz : Positive := 1_000_000);           -- startup only
  procedure Acquire (S : in out Session);
  procedure Configure_Pins (S : Session; Tx, Rx : ESP32S3.GPIO.Optional_Pin);
  procedure Transfer (S : Session; Tx, Rx : System.Address; Length : Natural);
  procedure Release (S : in out Session);
private
  type Session is new Ada.Finalization.Limited_Controlled with record
    Active : Boolean := False;
  end record;
  overriding procedure Finalize (S : in out Session);           -- auto-release
end ESP32S3.Widget;

```

In the body, a protected object arbitrates ownership. Its guarded section is tiny — it only flips a flag — so a task is never blocked inside it while a slow transfer runs:

```

protected Guard is
  entry Acquire;
  procedure Release;
private
  Held : Boolean := False;
end Guard;

protected body Guard is
  entry Acquire when not Held is begin Held := True; end Acquire;
  procedure Release is begin Held := False; end Release;
end Guard;

```

entry Acquire when not Held is the whole mutual-exclusion story: a second task calling Acquire *suspends* on the barrier until the first releases. (For a peripheral with several identical instances — e.g. two SPI hosts — this is an array (Host) of Guard and a matching array of handles; the singleton case is shown here for clarity.)

13.6 Layer 3 — the gateway that cannot be bypassed

Here is the heart of the design. The configured Bus lives inside a package nested in the *body*, so no other part of the driver can name it. The only way out is Owned:

```

package State is
  procedure Open (Clock_Hz : Positive);           -- the Setup path
  function Ready return Boolean;
  function Owned (S : Session) return E.Bus; -- THE gateway
end State;

package body State is
  The_Bus : E.Bus;                               -- hidden here; unreachable
  elsewhere
  Is_Set : Boolean := False;

  procedure Open (Clock_Hz : Positive) is
  begin

```

```

    The_Bus := E.Open (Clock_Hz);
    Is_Set  := True;
end Open;

function Ready return Boolean is (Is_Set);

function Owned (S : Session) return E.Bus is
begin
    if not S.Active then
        raise Not_Owned with "Widget used without holding it -- Acquire first";
    end if;
    return The_Bus;
end Owned;
end State;

```

Now every public operation is a single line that funnels through `Owned`, and the ownership check happens in exactly one place:

```

procedure Setup (Clock_Hz : Positive := 1_000_000) is
begin
    State.Open (Clock_Hz);
end Setup;

procedure Acquire (S : in out Session) is
begin
    if not State.Ready then
        raise Not_Initialized with "Widget acquired before Setup";
    end if;
    Guard.Acquire;                                     -- suspends until free
    S.Active := True;
end Acquire;

procedure Configure_Pins (S : Session; Tx, Rx : ESP32S3.GPIO.Optional_Pin) is
begin
    E.Configure_Pins (State.Owned (S), Tx, Rx); -- Owned raises unless held
end Configure_Pins;

procedure Transfer (S : Session; Tx, Rx : System.Address; Length : Natural) is
begin
    E.Transfer (State.Owned (S), Tx, Rx, Length);
end Transfer;

procedure Release (S : in out Session) is
begin
    if S.Active then
        S.Active := False;
        Guard.Release;
    end if;
end Release;

overriding procedure Finalize (S : in out Session) is
begin
    Release (S);                                       -- scope exit / exception unwind
end Finalize;

```

Look at `Configure_Pins` and `Transfer`: there is *no* `if S.Active then` in either. They cannot reach the hardware without calling `State.Owned (S)`, and `Owned` is the one place that checks. A maintainer

adding a new operation tomorrow has no choice but to go through the same door — there is no Bus in scope to grab any other way.

13.7 The limited-handle variant

If the engine’s Bus is a *limited* type — typically because it holds something limited, like a finalization-controlled DMA channel — it cannot be returned by value. Make the hidden instance aliased and have Owned return an access to it:

```
The_Bus : aliased E.Bus;
function Owned (S : Session) return access E.Bus is
begin
  if not S.Active then raise Not_Owned with "..."; end if;
  return The_Bus'Access;
end Owned;
-- callers: E.Transfer (State.Owned (S).all, Tx, Rx, Length);
```

In the HAL, SPI and I2S (whose engine Bus is limited) and LCD (whose handle owns a GDMA channel) use this form; I2C, UART and TWAI return the handle by value. Nothing else about the pattern changes.

13.8 Why it is built this way

Each layer earns its place by removing a specific class of mistake:

Private-child engine → the registers have one owner.

Because the parent never withs the SVD package, the controller global is unreachable outside the engine. The “any code can poke it” hazard is gone at the visibility level, not by convention.

Hidden handle + single Owned gateway → the guard cannot be forgotten.

The earlier, weaker approach put an `if S.Active` check at the top of each operation. That works until someone adds an operation and forgets the check. Hiding the only Bus behind Owned makes the check structural: forgetting it is not a bug you can write, because there is no other way to obtain a Bus.

Limited Session → ownership cannot be duplicated.

A non-copyable handle means two tasks can never hold the same controller, and a handle can never outlive its scope as a dangling copy.

Controlled Session (RAII) → the lock cannot leak.

The finalizer releases on every exit path, including exception unwinding. A fault between Acquire and Release hands the controller back instead of deadlocking the next task forever.

Tiny protected section → ownership without serialising work.

The lock is held only long enough to flip a flag; the blocking transfer runs outside it. Tasks contend for *ownership*, never for the bus itself.

Two named exceptions → misuse fails loudly.

`Not_Initialized` and `Not_Owned` turn “used in the wrong state” into an immediate, explanatory error instead of a silent no-op or a corrupted transfer.

13.9 The benefits

- **Mistakes become unrepresentable.** “Configure a port you don’t own,” “transmit from the wrong task,” “poke the registers directly” — none of these can be written. The compiler and the visibility rules enforce the discipline, so code review does not have to.

- **One pattern, every peripheral.** Because all six `Session` drivers share this skeleton, learning one teaches all of them, and a new driver is a fill-in-the-blanks exercise rather than a fresh concurrency design.
- **The unsafe core is testable in isolation.** The engine is plain, lock-free register logic; the parent is plain ownership logic. Each can be reasoned about — and the engine bench-tested — on its own.
- **RAII hygiene for free.** Scope-based release means the common “acquire, do work, release” path needs no explicit `Release` and is exception-safe by construction.
- **It costs almost nothing.** The handle is a couple of words, the gateway is one comparison, and the protected section is a flag flip. The safety is structural, not runtime overhead.

13.10 When the rule applies (and when it does not)

Use this pattern for a driver that is **both** exclusive (cannot be shared between tasks) **and** reconfigurable after setup. That is the combination that makes uncoordinated access dangerous.

It does *not* apply to lock-free or shareable peripherals. `GPIO`’s `set/clear/read` are hardware-atomic; `RNG` is a stateless read; the `Temperature`, `RTC` and `crypto` blocks either are internally serialised by a small protected object or have nothing a second task can corrupt. Those drivers stay simple on purpose and run under every runtime profile, including light-tasking, where the controlled `Session` types (which need finalization) are not available. Reach for the full ownership machinery only when the hazard is real; for everything else, the simplest safe thing is better.

Chapter 14

Tasking and Concurrency

Concurrency in Ada is part of the language, not a bolted-on RTOS API: *tasks* for independent threads of control, *protected objects* for race-free sharing, interrupt handlers as protected procedures, and a monotonic clock for real-time work. This chapter shows the practical mechanics on the dual-core ESP32-S3 — creating tasks and interrupt handlers, periodic scheduling, waiting with timeouts, the recurring concurrency patterns, watchdogs, and putting both cores to work. It draws on the encapsulation features of Chapter 9 and the drivers of the HAL chapters.

14.1 Tasks and interrupt handlers

Concurrency in Ada is in the language, not an RTOS API: you declare *tasks* for independent threads of control and *interrupt handlers* as protected procedures. This section shows the practical mechanics; the Interrupts chapter has the full story on how attachment works underneath.

14.1.1 Creating and starting a task

You do not “start” an Ada task with a call — you *declare* it, and it activates automatically. A single task is declared with its body; it begins running when its master (the enclosing scope) reaches the code after the declarations. For a *library-level* task that master is the program itself, so the binder activates it before Main even begins.

From C / ESP-IDF: there is no `xTaskCreate`, `pthread_create`, or `vTaskStartScheduler`: you declare the task and it activates itself, and the Ada runtime *is* the scheduler — already running when Main begins.

```
task Blinker;                                -- declaration: this creates the task
task body Blinker is
begin                                         -- runs concurrently once activated
  loop
    ESP32S3.GPIO.Toggle (2);
    delay until Clock + Milliseconds (250);
  end loop;
end Blinker;
```

Add a pragma `Priority` (or the `Priority` aspect) to place it in the priority order, and on this dual-core target pin it to a core with `CPU => 1` or `CPU => 2`:

```
task Sampler with Priority => 5, CPU => 2;    -- runs on the second core
```

For a *family* of identical workers use a task *type* and declare objects of it. One restriction matters here: under the light-tasking and embedded (Jorvik) profiles a task may not be nested inside a

subprogram (`No_Task_Hierarchy`), so application tasks live at *library level* — in a package — not inside `Main`.

14.1.2 Creating and attaching an interrupt handler

An interrupt handler is a *protected procedure* bound to an interrupt. The protected object's `Interrupt_Priority` is the hardware level it runs at (and the ceiling that serialises it), and `pragma Attach_Handler` wires it up at elaboration — no run-time call (§8.1 and the Interrupts chapter):

```
protected Handler with Interrupt_Priority => Device_L3_Priority is
private
  procedure ISR;
  pragma Attach_Handler (ISR, Device_L3_0);    -- bound to the L3 device slot
end Handler;
```

For GPIO pins the HAL wraps that boilerplate: `ESP32S3.GPIO.Interrupts` owns the one GPIO interrupt source, routes it to the level-3 slot, and demuxes by pin to per-pin callbacks. You just register a callback with a trigger:

```
Enable (Pin => 0, On => Falling_Edge, Action => On_Edge'Access);
```

A handler — protected ISR or HAL callback — runs in *interrupt context* at the slot's ceiling. Keep it tiny, and do not call a lower-ceiling protected object or block. The idiom is to record the event (bump an `Atomic`, or Set a `Suspension_Object`) and let an ordinary task do the real work.

14.1.3 Worked example: counting falling edges on GPIO0

Putting both together: a counter that increments on every falling edge of GPIO0, with a task that reports it once a second. The interrupt callback is the only writer of `Edges` (the level-3 ISR cannot re-enter itself), so an `Atomic Natural` written there and read by the task needs no lock. Everything is library-level, so it is legal on the embedded profile.

```
with Ada.Real_Time;           use Ada.Real_Time;
with Ada.Text_IO;            use Ada.Text_IO;
with ESP32S3.GPIO;          use ESP32S3.GPIO;
with ESP32S3.GPIO.Interrupts; use ESP32S3.GPIO.Interrupts;

package body Edge_Demo is

  Edges : Natural := 0 with Atomic;    -- written by the ISR, read by the task

  -- Runs in interrupt context (level-3 ceiling): do the minimum.
  procedure On_Edge is
  begin
    Edges := Edges + 1;                -- sole writer -> the inc is safe
  end On_Edge;

  -- A normal task, activated automatically at start-up.
  task Reporter;
  task body Reporter is
  begin
    loop
      delay until Clock + Seconds (1);
      Put_Line ("GPIO0 falling edges:" & Natural'Image (Edges));
    end loop;
  end Reporter;
```

```

begin
  -- Package elaboration: configure the pin and attach the interrupt.
  Configure (0, Mode => Input, Pull => Pull_Up);           -- pulled-up input
  Enable (Pin => 0, On => Falling_Edge, Action => On_Edge'Access);
end Edge_Demo;

```

Reading it line by line. For a reader new to Ada, here is every construct in the example:

`with Ada.Real_Time;`

names a unit this code *depends on* and makes its declarations available — like an `#include`, but semantic: the compiler records the dependency rather than pasting text. The four `with`s pull in timing (`Ada.Real_Time`), console output (`Ada.Text_IO`), the GPIO driver, and its interrupt child.

`use Ada.Real_Time;`

makes that unit's names usable *without* the package prefix (see below). `with` grants access; `use` grants short names.

`package body Edge_Demo is`

begins the implementation of a package called `Edge_Demo`. A package groups related declarations; everything up to `end Edge_Demo;` belongs to it. Because it is a top-level (*library-level*) package, the task it declares is legal under the Jorvik profile.

`Edges : Natural := 0 with Atomic;`

declares a variable. The colon separates the name from its type; `Natural` is the predefined type of integers ≥ 0 ; `:= 0` sets its initial value (`:=` is *assignment*; `=` is the equality *test*). `with Atomic` is an aspect making each read or write a single indivisible access.

`- ...`

a comment: `-` runs to the end of the line.

`procedure On_Edge is ... begin ... end On_Edge;`

defines a *procedure* — a subprogram that returns no value — taking no parameters. The statements between `begin` and `end` run when it is called. Repeating the name after `end` is optional but conventional.

`Edges := Edges + 1;`

an assignment statement: evaluate `Edges + 1` and store it back. Statements end with a semicolon.

`task Reporter;`

declares a *task* — an independent thread of control. This line creates it; it will start on its own (no “start” call).

`task body Reporter is ... loop ... end loop; ...`

the task's code. `loop ... end loop;` repeats indefinitely.

`delay until Clock + Seconds (1);`

suspends the task until an *absolute* instant: the current time (`Clock`) plus one second (`Seconds (1)` is a time span). `delay until` waits to a point in time; plain `delay X` would wait a duration.

`Put_Line ("..." & Natural'Image (Edges));`

prints a line. `&` concatenates strings, and `Natural'Image (Edges)` converts the number to text. The apostrophe introduces an *attribute* — a built-in query on a type or object; `'Image` is “the textual image of this value.”

`begin ... end Edge_Demo;`

a package body may end with an executable part. It runs *once, automatically*, when the package

is elaborated at start-up (before Main). Here it brings up the hardware.

`Configure (0, Mode => Input, Pull => Pull_Up);`
 calls the GPIO driver. `0` is a positional argument (the pin); `Mode => Input` and `Pull => Pull_Up` are *named* associations (name => value), so the call documents which parameter is which.

`Enable (... , Action => On_Edge'Access);`
 registers the interrupt. `On_Edge'Access` is the `'Access` attribute — a reference to the `On_Edge` procedure — which the HAL stores and calls from the ISR on each falling edge.

Why use `makes it readable`. A use clause makes a package's names directly visible, so you write `Configure`, `Falling_Edge` and `Put_Line` instead of their full paths. Without it, the same two setup lines would read:

```
ESP32S3.GPIO.Configure (0, Mode => ESP32S3.GPIO.Input,
                        Pull => ESP32S3.GPIO.Pull_Up);
ESP32S3.GPIO.Interrupts.Enable
(Pin => 0, On => ESP32S3.GPIO.Interrupts.Falling_Edge,
 Action => On_Edge'Access);
```

The package path repeats on nearly every name and buries the intent. With use the code reads in the language of the problem — “configure pin 0 as a pulled-up input; enable a falling-edge interrupt” — while the with clauses still record exactly where those names come from. The trade-off is that a name's origin is less explicit, and two used packages could in principle declare the same name; in a small, focused unit like this the clarity gain wins, and you can always write a single name in full where it aids understanding.

The application's Main need only `with Edge_Demo;` and `stay alive;` `Reporter` and the interrupt are running before Main's first statement. Each time GPIO0 goes high→low (release of a pull-up button, say) the ISR bumps `Edges`, and the reporter prints the running total — a complete interrupt-driven counter in a couple of dozen lines, with no RTOS, no manual ISR vectoring, and no explicit task start.

14.2 Real-time tasks: periodic and time-driven

For periodic work use `Ada.Real_Time` — a *monotonic* clock that never jumps — not `Ada.Calendar`. The canonical cyclic task computes its *next* release as an absolute instant and `delay` untils it, so timing error does not accumulate:

```
task body Sampler is
  Period : constant Time_Span := Milliseconds (10);
  Next    : Time := Clock;      -- first release = now
begin
  loop
    Read_And_Process;
    Next := Next + Period;      -- the next absolute instant
    delay until Next;           -- no drift, however long the body took
  end loop;
end Sampler;
```

Because `Next` advances by exactly `Period` regardless of how long `Read_And_Process` ran, the task holds its average rate with no cumulative jitter — the crucial difference from a naive `delay Period`, which slips by the body's run time every cycle. Two more tools are available on all profiles:

- `Time_Span` arithmetic (`Milliseconds`, `Microseconds`, `To_Duration`) for expressing intervals exactly;

- `Ada.Real_Time.Timing_Events` — register a procedure to run at a future instant without dedicating a task, handy for a one-shot timeout.

`Ada.Calendar` exists too, for wall-clock dates, but its absolute form is unsuited as a scheduling base (see the Limitations chapter) and on the bare runtime it is anchored only to boot. Schedule with `Ada.Real_Time`.

14.3 Waiting for an event — and timing out

Two questions come up constantly in firmware: how do I *wait* for something to happen — a GPIO pin to change, a byte to arrive on the UART — and how do I *give up* after a while? There are two ways to wait (poll, or block until an interrupt wakes you), and the clean way to bound the wait with a timeout differs across the three profiles, because the leaner two forbid the `select` statement.

14.3.1 Polling (any profile)

The simplest wait re-checks a condition in a loop, backing off with `delay until` so other tasks run; a deadline turns it into a timeout. To wait for a UART byte:

```
with Ada.Real_Time; use Ada.Real_Time;
with ESP32S3.UART; use ESP32S3.UART;

function Wait_For_Byte (S : Session; Timeout : Time_Span) return Boolean is
  Deadline : constant Time := Clock + Timeout;
begin
  while Available (S) = 0 loop
    if Clock >= Deadline then
      return False; -- gave up
    end if;
    delay until Clock + Milliseconds (1); -- back off; let other tasks run
  end loop;
  return True; -- a byte is waiting
end Wait_For_Byte;
```

The same shape waits on a GPIO level — replace `Available (S) = 0` with `not ESP32S3.GPIO.Read (Pin)`. Polling works on *every* profile and needs only `Ada.Real_Time`; the cost is the CPU spent re-checking.

Rather than hand-roll the deadline loop each time, the HAL bundles it as `ESP32S3.Wait.Until_True` (lock-free, all profiles): give it a condition function and a timeout and it returns `True` on success or `False` on timeout.

```
with ESP32S3.Wait;
function Byte_Ready return Boolean is (Available (S) > 0);
...
if ESP32S3.Wait.Until_True (Byte_Ready'Access, Milliseconds (200)) then
  Read (S, Buf, N); -- a byte arrived in time
end if;
```

14.3.2 Blocking until an interrupt signals you (any profile)

Better, when the wait may be long or power matters: sleep until the hardware wakes you. Attach an interrupt handler that signals a `Suspension_Object`, and suspend the task on it:

```
with Ada.Synchronous_Task_Control; use Ada.Synchronous_Task_Control;
with ESP32S3.GPIO.Interrupts;
```

```

Edge_Seen : Suspension_Object;

procedure On_Edge is          -- runs in interrupt context: keep it tiny
begin
    Set_True (Edge_Seen);
end On_Edge;
...
ESP32S3.GPIO.Interrupts.Enable
    (Pin => 0, On => ESP32S3.GPIO.Interrupts.Falling_Edge, Action => On_Edge'Access);
...
Suspend_Until_True (Edge_Seen);    -- the task sleeps here until the edge fires

```

This burns no CPU while waiting. But `Suspend_Until_True` has no timeout of its own — and *that* is where the profiles diverge.

14.3.3 Timing out: three profiles, three approaches

Jorvik (light-tasking). No select and no exception propagation, so the timeout is a *value* and the wait blocks on a protected entry whose barrier opens on either the event or a deadline. The deadline is an `Ada.Real_Time.Timing_Events` handler; the task genuinely sleeps, and on return a flag says which fired:

```

with Ada.Real_Time.Timing_Events; use Ada.Real_Time.Timing_Events;

protected Gate is
    procedure Signal;          -- from the interrupt callback
    procedure Expire (E : in out Timing_Event); -- the deadline handler
    entry Wait (Timed_Out : out Boolean);
private
    Occurred, Expired : Boolean := False;
end Gate;

protected body Gate is
    procedure Signal is begin Occurred := True; end Signal;
    procedure Expire (E : in out Timing_Event) is begin Expired := True; end Expire;
    entry Wait (Timed_Out : out Boolean) when Occurred or Expired is
    begin
        Timed_Out := not Occurred;
        Occurred := False; Expired := False;    -- re-arm for next time
    end Wait;
end Gate;

-- Use: the interrupt callback calls Gate.Signal; the caller arms a deadline.
declare
    Deadline : Timing_Event;
    Late     : Boolean;
begin
    Set_Handler (Deadline, Clock + Milliseconds (500), Gate.Expire'Access);
    Gate.Wait (Late);    -- sleeps until the edge OR 500 ms, whichever first
    if Late then ... else ... end if;
end;

```

embedded. The same Jorvik primitives (still no select), but exceptions now propagate — so a timeout reads naturally as a *catchable exception*, giving linear code, with any controlled handles released on the way out:

```

Timeout_Error : exception;

procedure Wait_For_Byte (S : Session; Timeout : Time_Span) is
  Deadline : constant Time := Clock + Timeout;
begin
  while Available (S) = 0 loop
    if Clock >= Deadline then
      raise Timeout_Error with "no byte within the deadline";
    end if;
    delay until Clock + Milliseconds (1);
  end loop;
end Wait_For_Byte;
...
begin
  Wait_For_Byte (S, Milliseconds (200));
  Read (S, Buf, N);           -- a byte is here
exception
  when Timeout_Error =>      -- handle the timeout in one place
    ...
end;

```

(You can raise the same exception from the `Gate.Wait` barrier above instead of polling, if you would rather sleep than spin.)

full. Full Ada gives the timeout directly, as a *timed entry call*: the task blocks on the entry and a delay races it — no helper deadline, no polling. The protected object only needs the event half:

```

protected Gate is
  procedure Signal;           -- from the interrupt callback
  entry Wait;
private
  Occurred : Boolean := False;
end Gate;

protected body Gate is
  procedure Signal is begin Occurred := True; end Signal;
  entry Wait when Occurred is begin Occurred := False; end Wait;
end Gate;

-- Use: whichever happens first wins.
select
  Gate.Wait;                  -- the event arrived
  ...
or
  delay until Clock + Milliseconds (500); -- the timeout
  ...
end select;

```

(Use `delay until` — an *absolute* time — here: on this port a *relative* delay as a select trigger has a known gap, see the Limitations chapter.) `full` also offers `select ... then abort` to put a deadline around an *arbitrary* computation, not just an entry call — abandoning it if it overruns.

Choosing. Poll for a short, infrequent wait or the leanest code; interrupt-signalled blocking is better when the wait may be long or power matters. For the timeout: a status value on Jorvik, a catchable exception on embedded, and the `select/delay` timed entry call on `full`.

14.4 Concurrency patterns

A few protected-object shapes recur often enough to be worth naming. All work on every tasking profile; the Jorvik restrictions (one entry per protected object, one caller queued at a time) shape them but do not prevent them.

Producer/consumer over a bounded buffer. The classic decoupling: one task (or an interrupt) produces items, another consumes them, with a protected ring buffer between. The consumer’s Get is an *entry* whose barrier holds it until something is available — no polling.

```
protected Queue is
  procedure Put (X : Item);           -- producer side (also callable from an ISR)
  entry      Get (X : out Item);      -- consumer side: blocks while empty
private
  Buf       : Item_Array (0 .. 15);
  Count     : Natural := 0;
  Head, Tail : Natural := 0;
end Queue;

protected body Queue is
  procedure Put (X : Item) is
  begin
    if Count < Buf'Length then        -- (full -> drop; or count an overflow)
      Buf (Tail) := X;
      Tail := (Tail + 1) mod Buf'Length;
      Count := Count + 1;
    end if;
  end Put;

  entry Get (X : out Item) when Count > 0 is
  begin
    X := Buf (Head);
    Head := (Head + 1) mod Buf'Length;
    Count := Count - 1;
  end Get;
end Queue;
```

The consumer task just loops `Queue.Get (X)`; it sleeps on the barrier when the queue is empty and wakes the instant `Put` adds an item. (Under Jorvik, exactly one consumer task may queue on `Get` at a time — which is the usual one-producer/one-consumer arrangement.)

A signal / mailbox. The degenerate one-slot case is a handoff: an interrupt deposits a value and a task collects it. It is the “protected entry” half of the waiting pattern (§14.3) carrying a payload — a procedure `Post (V)` from the ISR and an entry `Take (V : out ...)` barriered on “something posted.”

DMA double-buffering (ping-pong). For a continuous stream — audio in over I2S, samples to a DAC — you cannot stop to process between transfers. Use *two* buffers: while the GDMA fills (or drains) buffer A, the CPU processes buffer B; when the transfer completes you swap, so the peripheral is never idle. The completion interrupt signals the processing task (a suspension object or a queue of “buffer ready” indices) and points the next transfer at the other buffer. The protected object owns the “which buffer is the hardware’s” state; the task owns the other. This keeps a real-time stream gap-free with bounded, analysable hand-off.

Why protected objects, not flags. Each of these could be hacked with a shared `Volatile` flag, but the protected object gives you the barrier (no polling), mutual exclusion (no torn reads across cores), and a

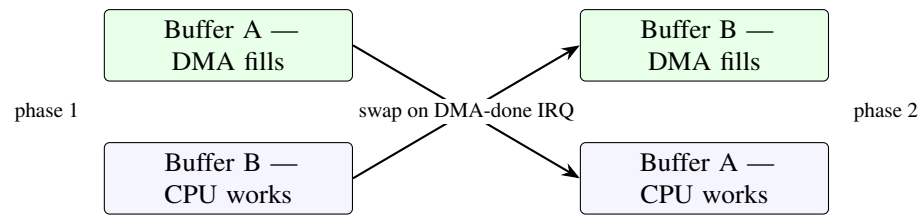


Figure 14.1: DMA double-buffering: the peripheral always has one buffer while the CPU works the other; the completion interrupt swaps their roles (the crossing), so the stream never stalls.

priority ceiling (bounded blocking) for free — the points the Encapsulation chapter (Chapter 9) makes about protected.

14.5 Watchdogs and recovery

A watchdog is a hardware timer that resets the chip unless the software keeps “feeding” it — the last line of defence against a hang that no exception or check can catch (a wild loop, a deadlock, a stuck peripheral). The ESP32-S3 has several: the two timer-group main watchdogs (TG0/TG1 MWDT), the RTC watchdog (RWDT), and the RTC *super*-watchdog (a hardware backstop).

The bare boot leaves them off. The ROM and the second-stage bootloader arm watchdogs during boot, but this runtime’s startup *disables* them (so a long elaboration or a debugger pause is not cut short). So by default a bare-metal app runs with *no* active watchdog — a hang will sit forever rather than recover. That is fine while developing (and is why a wedged example does not reset on its own), but a deployed device usually wants protection.

Arming the RTC watchdog. To get auto-recovery, enable the RWDT with a timeout and feed it from a place that only runs while the system is healthy. The registers are key-locked; the sequence is unlock, configure, feed, re-lock:

```

-- RTC watchdog registers (overlaid as plain words).
Key      : constant := 16#50D8_3AA1#;
Config0  : Unsigned_32 with Volatile, Address => To_Address (16#6000_8098#), Import;
Config1  : Unsigned_32 with Volatile, Address => To_Address (16#6000_809C#), Import;
Feed     : Unsigned_32 with Volatile, Address => To_Address (16#6000_80AC#), Import;
Wprot    : Unsigned_32 with Volatile, Address => To_Address (16#6000_80B0#), Import;

procedure Arm (Timeout_Cycles : Unsigned_32) is -- RTC-slow-clk (~63 kHz) cycles
begin
    Wprot := Key; -- unlock
    Config1 := Timeout_Cycles;
    Config0 := 2**31 -- WDT_EN
                or (3 * 2**28) -- stage 0 action = reset the main system
                or (7 * 2**16) or (7 * 2**13); -- reset-pulse lengths
    Feed := 2**31;
    Wprot := 0; -- re-lock
end Arm;

procedure Pet is begin Wprot := Key; Feed := 2**31; Wprot := 0; end Pet;
  
```

Feed on progress, not on a timer. The robust idiom is to pet the dog from points that only execute when work is actually advancing — the main loop body, a completed transaction — not from a periodic task that would keep petting even if the real work has stalled. The ACATS test harness in this project does exactly this: it arms the RWDT (≈ 19 s) and feeds it on every progress line, so a test that deadlocks stops feeding and is reset, while a slow-but-progressing test survives.

The **super-watchdog** is a separate hardware backstop that a deep-sleep wake can leave armed; ESP32S3. `RTC.Disable_Super_Watchdog` turns it off when an app means to stay awake (the RTC driver also re-feeds it automatically after a deep-sleep wake). Between an armed RWDT for hang-recovery, the precise stack-overflow watchpoint (the Debugging chapter), and a last-chance handler for the unrecoverable case, a deployed app has a layered safety net.

14.6 Dual-core (SMP) programming

The ESP32-S3 has two cores and the runtime schedules across both. Two ordinary-Ada facilities put them to use.

Pin a task to a core with the CPU aspect (CPU => 1 is the first core, 2 the second); omit it and the task may run on either:

```
task Beat0 with Priority => Priority'Last - 1, CPU => 1;
task Beat1 with Priority => Priority'Last - 1, CPU => 2;  -- the other core
```

Share data with a protected object — it works *across* cores, not just between tasks on one core. A protected *entry* can even hand a caller from one core to the other: in the `esp32s3_smp` example a producer on core 1 blocks in entry `Get` when `Full`, and serving the entry wakes it through the inter-core interrupt. The locking is the same priority-ceiling monitor (Chapter 9); the runtime adds the cross-core wake-up underneath.

```
protected Mailbox is
  procedure Put (V : Integer);
  entry      Get (V : out Integer);  -- blocks until Full; may serve cross-core
private
  Item : Integer;
  Full : Boolean := False;
end Mailbox;
```

Two cautions specific to this target: time comes from a *shared* hardware timer (the SYSTIMER), not each core's own cycle counter, so `Clock` agrees across cores; and the USB-serial-JTAG console is not robust for two tasks printing at once — funnel diagnostics through one task. With those in mind, an SMP application is just ordinary tasks, pinned where you want them, sharing through protected objects.

Chapter 15

Mass Storage: SD Cards and the ext4 Filesystem

The final layer is mass storage: two block drivers for SD cards, and a complete pure-Ada ext2/3/4 filesystem (a reimplementation of `lwext4`) that sits on top of them and mounts real `mkfs.ext4` cards.

15.1 SD_SPI — an SD card over the SPI master

`ESP32S3.SD_SPI` talks to an SD/SDHC card in SPI mode, layered on the task-safe SPI driver. The chip-select is an ordinary GPIO so it can be held across a whole command. Blocks are 512-byte logical sectors (LBA addressing); SDSC and SDHC are detected and handled internally.

```
type Block is array (0 .. 511) of Interfaces.Unsigned_8;
type Status is (OK, No_Card, Unusable, Init_Timeout, Read_Error, Write_Error);
procedure Setup (C : out Card; Host : ESP32S3.SPI.SPI_Host;
                 Sclk, Mosi, Miso, Cs : Pin_Id;
                 Init_Clock_Hz : Positive := 400_000;
                 Data_Clock_Hz : Positive := 8_000_000);
procedure Initialize (C : in out Card; Result : out Status);
procedure Read_Block (C : in out Card; LBA : Block_Address;
                     Data : out Block; Result : out Status);
procedure Write_Block (C : in out Card; LBA : Block_Address;
                      Data : Block; Result : out Status);
```

Example 1 — bring up a card and read sector 0.

```
with ESP32S3.SD_SPI; use ESP32S3.SD_SPI;
with ESP32S3.SPI;
procedure SD_Init is
  C : Card;
  St : Status;
  Sec : Block;
begin
  Setup (C, ESP32S3.SPI.SPI2, Sclk => 12, Mosi => 11, Miso => 13, Cs => 10);
  Initialize (C, St);
  if St = OK then
    Read_Block (C, LBA => 0, Data => Sec, Result => St); -- the MBR sector
  end if;
end SD_Init;
```

Example 2 — a non-destructive read / write-back / re-read round-trip.

```

with ESP32S3.SD_SPI; use ESP32S3.SD_SPI;
with ESP32S3.SPI;
procedure SD_Roundtrip is
  C      : Card;
  St     : Status;
  Orig, Back : Block;
begin
  Setup (C, ESP32S3.SPI.SPI2, Sclk => 12, Mosi => 11, Miso => 13, Cs => 10);
  Initialize (C, St);
  Read_Block (C, 16#2000#, Orig, St);      -- read a scratch sector
  Write_Block (C, 16#2000#, Orig, St);     -- write the same bytes back
  Read_Block (C, 16#2000#, Back, St);     -- re-read; Back = Orig, no data lost
end SD_Roundtrip;

```

15.2 SDMMC — the native SD/MMC host

ESP32S3.SDMMC drives the dedicated SDHOST controller (clock + command + 1- or 4-bit data) in PIO mode — faster than SPI mode, and with no DMA or finalization it works under every profile. The API mirrors SD_SPI.

```

procedure Setup (C : out Card; On : Slot; Clk, Cmd, D0 : Pin_Id;
  D1, D2, D3 : Optional_Pin := No_Pin;
  Width : Bus_Width := Width_1; ...);
procedure Initialize (C : in out Card; Result : out Status);
procedure Read_Block (C : in out Card; LBA : Block_Address; Data : out Block;
  Result : out Status);

```

Example — 4-bit bus, read a sector.

```

with ESP32S3.SDMMC; use ESP32S3.SDMMC;
procedure SDMMC_Read is
  C : Card;
  St : Status;
  Sec : Block;
begin
  Setup (C, On => Slot1, Clk => 14, Cmd => 15, D0 => 2, D1 => 4, D2 => 12,
    D3 => 13, Width => Width_4);
  Initialize (C, St);
  if St = OK then
    Read_Block (C, 0, Sec, St);
  end if;
end SDMMC_Read;

```

15.3 The ext4 filesystem

ESP32S3.Ext4 is a pure-Ada ext2/3/4 implementation with a JBD2 journal (Chapter overview in `libs/esp32s3_hal/src/ext4/README.md`). It reads default `mkfs.ext4` cards (extents, 64-bit, HTree directories, `metadata_csum` verification), writes ext2/3/non-csum filesystems, and recovers a dirty journal on mount. It needs the embedded or full profile.

15.3.1 Connecting a block device

The filesystem talks to a small block-device abstraction (`ESP32S3.Block_Dev.Device`). An adapter turns an initialised SD card into one with a single call — the Make below appears in each complete

example that follows:

```
with ESP32S3.Block_Dev;
with ESP32S3.Block_Dev.SD_SPI_Source;
-- ... given an initialised "Card : aliased ESP32S3.SD_SPI.Card" ...
Dev : constant ESP32S3.Block_Dev.Device :=
    ESP32S3.Block_Dev.SD_SPI_Source.Make (Card'Access);
```

15.3.2 Mounting and reading

A Mount is a controlled handle: it flushes and closes on scope exit, and Open automatically replays a dirty journal on a writable mount. The complete example brings up the card, wraps it, mounts read-only, then reads a file:

```
with ESP32S3.SPI;
with ESP32S3.SD_SPI;
with ESP32S3.Block_Dev;
with ESP32S3.Block_Dev.SD_SPI_Source;
with ESP32S3.Ext4;
with ESP32S3.Ext4.FS;
with ESP32S3.Ext4.Inode;
procedure EXT4_Read is
    use type ESP32S3.SD_SPI.Status;
    Card : aliased ESP32S3.SD_SPI.Card;
    St    : ESP32S3.SD_SPI.Status;
begin
    ESP32S3.SD_SPI.Setup (Card, ESP32S3.SPI.SPI2,
        Sclk => 12, Mosi => 11, Miso => 13, Cs => 10);
    ESP32S3.SD_SPI.Initialize (Card, St);
    if St /= ESP32S3.SD_SPI.OK then
        return;
    end if;
    declare
        Dev : constant ESP32S3.Block_Dev.Device :=
            ESP32S3.Block_Dev.SD_SPI_Source.Make (Card'Access);
        M    : ESP32S3.Ext4.FS.Mount;
        I    : ESP32S3.Ext4.Inode.Info;
        Buf  : ESP32S3.Ext4.Byte_Array (0 .. 255);
        Last : Natural;
    begin
        M.Open (Dev, Read_Only => True);
        M.Stat (M.Lookup ("/etc/hello.txt"), I); -- resolve a path -> its inode
        M.Read_File (I, Offset => 0, Into => Buf, Last => Last);
        -- Buf (0 .. Last - 1) holds the first Last bytes of the file
    end; -- M finalizes: flush + close
end EXT4_Read;
```

Listing a directory. Iterate calls a visitor for each entry. The card bring-up is identical to the example above; the body is:

```
declare
    Dev : constant ESP32S3.Block_Dev.Device :=
        ESP32S3.Block_Dev.SD_SPI_Source.Make (Card'Access);
    M    : ESP32S3.Ext4.FS.Mount;
    Root : ESP32S3.Ext4.Inode.Info;
    procedure Show (Name : String;
        Ino : ESP32S3.Ext4.Inode_Number;
```

```

File_Type : ESP32S3.Ext4.U8) is
begin
  Put_Line (" " & Name);
end Show;
begin
  M.Open (Dev, Read_Only => True);
  M.Stat (M.Lookup ("/"), Root);
  M.Iterate (Root, Show'Access);
end;

```

15.3.3 Writing

On a non-metadata_csum filesystem (ext2/3, or mke2fs -O ^metadata_csum) the filesystem can be modified. The full POSIX-like set is available: Create_File, Write_File, Truncate, Mkdir, Rmdir, Unlink, Rename and hard Link.

```

declare
  Dev : constant ESP32S3.Block_Dev.Device :=
    ESP32S3.Block_Dev.SD_SPI_Source.Make (Card'Access);
  M   : ESP32S3.Ext4.FS.Mount;
  N   : ESP32S3.Ext4.Inode_Number;
begin
  M.Open (Dev, Read_Only => False);
  N := M.Create_File ("/", "log.txt");
  M.Write_File (N, (Character'Pos ('h'), Character'Pos ('i'), 10));
  M.Mkdir ("/", "data");
  M.Close;           -- flush + close (or let M finalize)
end;

```

15.3.4 Crash-safe transactions

For atomicity, group an operation behind Commit: the dirty metadata and the superblock are written to the journal (durably), the RECOVER flag is set — the commit barrier — then the metadata is checkpointed to its final locations and the flag cleared. A power loss after the barrier is recovered forward by the next Open (or by any other ext4 implementation, including the Linux kernel); a loss before it has no effect.

```

-- Payload : constant ESP32S3.Ext4.Byte_Array := (16#41#, 16#42#, 16#43#);
N := M.Create_File ("/", "important.txt");
M.Write_File (N, Payload);
M.Commit;           -- one atomic, journaled, crash-safe transaction

```

15.3.5 Errors

Filesystem operations raise: the standard Ada.IO_Exceptions names (Name_Error for a missing path, Use_Error, Device_Error, End_Error) plus filesystem-specific Unsupported_Feature, Bad_Checksum, Corrupt, No_Space, Not_Empty and Read_Only. Wrap calls in a handler so a bad card or a missing file never escapes a task:

```

begin
  M.Stat (M.Lookup ("/maybe/missing"), I);
exception
  when ESP32S3.Ext4.Name_Error => Put_Line ("not found");
  when ESP32S3.Ext4.Bad_Checksum => Put_Line ("corrupt metadata");
end;

```

15.3.6 How it is validated

Because the filesystem is pure logic over the block interface, it is developed and tested host-native (x86) against real `mke2fs` images, with the Linux kernel and `e2fsck` as the oracle: every read is byte-compared to the source file, every mutation is checked with `e2fsck -fn`, and journal replay and commit are cross-checked against the kernel's own recovery. The same code cross-compiled for the ESP32-S3 and runs over the SD drivers above. See `tests/ext4_host/` and `libs/esp32s3_hal/src/ext4/REFERENCES.md`.

The whole stack — runtime, peripheral HAL, and filesystem — is one coherent body of Ada: strong types from the register up, the same RAII and task-safety patterns in every driver, and a filesystem validated against the Linux kernel, all running on both cores of the ESP32-S3 on its own bare-metal runtime.

Chapter 16

Conformance at Scale: A Standalone ACATS Sweep

The runtime is only as trustworthy as the conformance evidence behind it. This chapter describes how the project runs the official ACATS 4.2 suite *one test per image* across a pool of ESP32-S3 boards in parallel, and why that arrangement recovers hundreds of passes that a bundled sweep loses.

Note. The ACATS test suite and its sweep harness (`acats_build.py`, `acats_run.py`, the `ACATS/` tree) are not shipped in this distribution — they belong to the development repository. This chapter is kept as a record of the conformance methodology and results; the commands below are shown to document *how* the sweep was run, not as steps to reproduce from this repository.

16.1 Two ways to run the suite

The earlier sweep (`parallel_sweep.py`) is *bundled*: dozens of tests share one flashed image and advance through it with a panic-resume cursor, so a crash or hang resets the board and the next boot continues past the offending test. That is fast and cheap on flash cycles, but it couples the tests. A neighbour’s heap churn or secondary-stack residue can exhaust memory mid-batch and reset a later test *before* it prints its grade — the test is recorded as ungraded even though it passes in isolation. On the full profile the bundled run grades 914 PASS, with a large ungraded tail dominated by these batch artifacts.

The *standalone* sweep removes the coupling completely: every test is compiled as its own single-test program, flashed alone, run alone, and graded alone. There is no shared cursor, no shared heap, no neighbour. The cost — roughly 1,500 separate cross-compiles and flashes — is absorbed by running the work across many boards and many build trees at once.

16.2 Getting the grade off the chip

A subtlety decides whether the standalone approach is even possible. The bare runtime sends its console — including the ACATS Report PASSED/FAILED lines — out the ESP32-S3’s *built-in USB-Serial-JTAG*, not the UART pins. The board pool here is wired with ordinary external USB-serial adapters (FTDI and CH343) on UART0, which never see that JTAG console.

The fix is small. `ACATS/target/report.adb` routes each grade line through the ROM’s `esp_rom_printf` in addition to `Ada.Text_IO`:

```
procedure Line (Msg : String) is
```

```

procedure Rom_Printf (Fmt, Arg : System.Address)
  with Import, Convention => C, External_Name => "esp_rom_printf";
  Fmt : constant String := "%s" & ASCII.LF & ASCII.NUL;
  Buf : String (1 .. Msg'Length + 1);
begin
  Buf (1 .. Msg'Length) := Msg;
  Buf (Buf'Last) := ASCII.NUL;
  Rom_Printf (Fmt'Address, Buf'Address);
end Line;

```

The ROM console reaches UART0, so the grades land on the external bridge regardless of how the board is wired.

16.3 The grader must not trust the test id

The first version of the harness mis-scored most tests as hung. The cause was the grading regex: it looked for the list name followed by PASSED, but the list says A22006 while Report prints the *unit* name A22006B — the suffix letter breaks an exact-id match. Because each image contains exactly one test, the grader instead matches the Report terminal line *structurally*, ignoring the name:

```

FAILED          ->  **** <anything> FAILED          (checked first)
NOT-APPLICABLE  ->  ++++ <anything> NOT-APPLICABLE
PASSED          ->  ==== <anything> PASSED
TENTATIVELY     ->  !!!! <anything> TENTATIVELY PASSED

```

A second early bug scored passing tests as crashes: the flash step reset the board to run, then the capture step reset it again, producing two ESP-ROM: banners — and the crash heuristic keyed on “two or more boot banners.” Flashing with `--no-reset` leaves the chip halted so the capture’s own reset is the single deterministic boot; a second banner then genuinely means a crash-reboot.

16.4 Phase 1 — parallel build

`acats_build.py` compiles each test as a one-test image. The build is the bottleneck (a single board sits idle for tens of seconds per cross-compile), so it is parallelised. The complication is that the build performs several *shared* writes — it regenerates the runtime (`gen_runtime.sh`), rebuilds `common/bare/boot/obj`, and stages test sources under `ACATS/work/run` — so naive parallel builds in one tree corrupt each other.

Each builder therefore gets its own isolated “mini-repo”: a copy of `crates`, `libs`, `ACATS`, `ACATS42`, and the two needed `examples` subtrees, laid out at the same depth so the build scripts’ `REPO=$EX/./...` arithmetic still resolves. The copy carries the warm caches, so per-test builds stay incremental — no cold runtime rebuild per worker.

```

# OUT/<name>.bin          built OK (the flashable image)
# OUT/<name>.nobuild      could not build / all tests dropped (empty marker)
# OUT/BUILD_DONE         written once every test has been attempted
./acats_build.py ACATS/work/full_applicable.txt OUT 10

```

A test that drops to zero buildable units (it needs a library unit the bare runtime omits) produces `tests` in `image: 0` and becomes a `.nobuild`. The `.bin` is written atomically (a temp file renamed into place) so Phase 2 never sees a partial image, and the phase is resumable — a test that already has a `.bin` or `.nobuild` is skipped.

16.5 Phase 2 — parallel run and grade

`acats_run.py` starts one worker per board. Each worker claims the next un-run `.bin`, flashes its board with `--no-reset`, pulses a reset, captures the console until it sees a grade (or a timeout), classifies the run, and writes a single marker file holding the raw log:

```
# OUT/<name>.passed .failed .na .crashed .hung .nooutput .flasherr
./acats_run.py OUT 20 # 20-second capture window
```

The phases may overlap: Phase 2 drains `.bin` files as they appear and exits once `BUILD_DONE` exists and no un-run image remains. The whole sweep is resumable at the file level — delete a marker and re-run, and only that test is re-flashed. A handful of tests that briefly produced no console (a capture race during parallel flashing) and several genuinely slow tasking tests were re-run with a 60-second window simply by removing their markers.

16.6 Results

Full profile, `full_applicable.txt` (1,518 tests), one image each:

Outcome	Count	Meaning
PASS	1,286	graded PASSED / TENTATIVELY PASSED
NOT-APPLICABLE	10	test ruled itself out (correctly)
FAIL	0	
HUNG	4	documented limitations (see below)
CRASH	0	
won't-build	218	needs a library unit the bare runtime omits

That is **1,296 conformant** (PASS plus N/A), **zero failures**, and **zero crashes** — and **+372 PASS** over the bundled full run (914), recovered purely by removing batch interaction. Building each test alone is what turns a co-bundled “ungraded” tail back into passes.

Three apparent FAIL were harness artifacts, now resolved. CXD1002 and CXD1007 check the *main task's* pragma `Priority`, live only when the test is built *as the program main*; the harness now honours `standalone_tests.txt`'s main mode and they PASS. CZ1101 is the *ACATS Report self-test*, which deliberately prints `**** FAIL_TEST FAILED` sub-test sentinels; the grader now matches the result line for the test's *own* name and takes its final verdict (!!!! CZ1101A TENTATIVELY PASSED).

Five apparent HUNG were under-resourced, now passing. CXD1008 and CXD4007 are silent slow tests (`ImpDef.Switch_To_New_Task` is 1.0 s, so they need minutes, not the default 60 s window); C433A01/02 and C953002 OOM'd at the 64 KB secondary-stack default. The harness now honours two tuning lists — `long_wdt_tests.txt` (a 240 s capture window) and `small_sec_stack_tests.txt` (the 4 KB default for many-task tests) — and all five PASS.

Two more were program-termination cases, now fixed. C761001's grade is emitted from a *library-level* controlled object's `Finalize`, which runs only at program termination — but `Configurable_Run_Time` makes `gnatbind` omit the `finalize_library` chain, so it never ran. Flipping that flag is not viable (it needs a full runtime rebuild and is entangled with the runtime's codegen; the `.ali` flags are identical native-vs-target, so it is purely the binder mode). Since the per-unit finalizers (`<unit>__finalize_spec/body`) are still generated, the build synthesizes a finalizer-runner that calls the user units' finalizers in reverse elaboration order (parsed from the binder's `adainit`; weak + guarded), merges it into the image, and the bare `env` body calls it after `adafinal` — reached only by a program-main image after its main returns, so the wrapped sweep images are unaffected. C761001 now PASSES.

C94004 is a multi-file test whose real main is C94004A, a library task “examined to see whether it terminates.” As a wrapped batch test its non-terminating library tasks pile up and hang; built as the program main, only C94004A runs and prints PASSED from its own body. Its main unit (c94004a) differs from the list key (C94004), so the build resolves the boot entry from `batch_runnable.txt` (`build_ada.sh` writes `.noidf/ada_main.sym`, which `bare_build` honours instead of the guessed `_ada_c94004`). C94004 now PASSES.

The remaining four HUNG are documented limitations. CXD1006 (an `Interrupt_Priority` task collides with the kernel’s own Xtensa level 5), CB1010 (a large-frame recursive stack overflow the 64-byte watchpoint cannot catch), C480001 (unbounded recursion / GNAT codegen), and CXC7004 — *not* a deadlock but an environment-task *termination* test (JTAG dual-core post-mortem shows `cpu0` spun past `ADA_MAIN`, `cpu1` idle) whose grade depends on shutdown-path behavior the Configurable runtime does not provide.

The ten N/A are correct. CXD2001-2004 and CXD6001-6003 test *uniprocessor* dispatching — a single ready queue with deterministic ordering — which does not apply to a genuine two-core SMP target, so each prints + Multi-Processor Configuration and grades NOT-APPLICABLE. C45322, C45523, and C4A012 rule themselves out for their own reasons.

16.6.1 A wider net: the maximal list across seven boards

A later run pushed the net wider on purpose. Rather than the curated `full_applicable.txt`, it swept the *maximal union* — the applicable list plus everything previously parked in `jorvik_skipped.txt` and `ungraded_tests.txt`, 1,536 tests — one image each, across *seven* boards at once (the standalone harness is one worker per serial console, so seven consoles run seven tests in parallel). The point of including the already-skipped set is that capabilities added since the lists were drawn (the Calendar children, and the interrupt work) might turn an old `nobuild` into a pass; running the superset catches that automatically.

It did, and it also earned its keep by finding a real bug. The dynamic `Ada.Interrupts` tests CXC3007/CXC3008 *failed* — which is how the restricted-a-interr-stub problem (every dynamic operation raise `Program_Error`; see the Interrupts chapter) came to light. With the delegating a-interr body in place, CXC3007 now PASSES and CXC3008 advances into its interactive “generate the interrupt” phase (which the unattended rig cannot satisfy). The wider net found a second real bug too: C954025/C954026 (a *requeue-with-abort* whose timeout is a delay until an `Ada.Calendar` time) expired at the wrong moment, because the kernel’s timed-sleep collapsed the `Absolute_Calendar` and `Absolute_RT` delay modes and fed a Calendar-base deadline to the raw counter. Shifting a Calendar deadline back into the raw base (adding the BB-time Epoch) in `Timed_Sleep` fixes both — they now PASS (they are slow and silent, so they join the wide-watchdog list). After these fixes the residue on the wider list is what you would expect: the documented frontiers (CXD1006, CB1010, C480001, CXC7004), the interactive interrupt and suspension tests that need a bench-generated interrupt, and C96004’s post-2099 Calendar range — no crashes, and no surprise failures in the conformant core.

16.7 The embedded profile

The same harness runs the embedded (ZCX) profile against the `jorvik_hw_runnable.txt` list (846 tests). It needed one runner fix first. For a single-test image the generated driver adds a do-nothing task to force the environment task’s secondary stack; that task was declared *nested inside* the `Main` procedure, which is fine under `full` but violates the embedded / light-tasking Jorvik restriction `No_Task_Hierarchy` (a task nested in a subprogram is illegal). Every size-1 image therefore failed to build — the standalone sweep produced all-no-build. The bundled `parallel_sweep` never hit this because it pads isolated images with filler tests to keep the batch larger than one, so the helper task was never

emitted. The fix is to emit the helper as a *library-level* task instead, which is legal under Jorvik and accepted by full alike.

Outcome	Count	Meaning
PASS	840	graded PASSED / TENTATIVELY PASSED
NOT-APPLICABLE	3	test ruled itself out (correctly)
FAIL	1	CXC3004, the interactive test (see below)
HUNG	0	
CRASH	0	
won't-build	2	partition-wide build constraints (see below)

843 conformant, zero HUNG, zero CRASH, zero genuine FAIL — above the bundled embedded baseline (836). The one FAIL, CXC3004, is the *interactive* test: it waits for a manually-generated external interrupt, which nothing raises on the unattended rig — a documented limitation, not a runtime bug.

The two won't-build are notable because neither is a missing Ada library unit — each is a *partition-wide* build constraint. CXH1001 (Annex H) uses the configuration pragma `Normalize_Scalars`, which must be applied to every unit in the partition including the runtime; the prebuilt embedded runtime is not compiled with it, so the binder rejects the program (“*some but not all files compiled with Normalize_Scalars*”). CA5006 (a pragma `Elaborate` separate-compilation test) binds to an *elaboration circularity* under the embedded runtime's static elaboration model. Both are correctly counted as won't-build rather than failures.

16.8 Closing a won't-build category: the Ada.Calendar family

Some of the “won't-build” tail is not a fundamental limitation but a simply-missing library unit, and the sweep is what makes that visible. The bare runtime originally shipped only `base Ada.Calendar` (plus `.Conversions`); the RM 9.6.1 *child* units were absent, so every Calendar formatting/arithmetic test dropped at compile. Those children are now provided on the embedded and full profiles, from a shared `calendar_overlay/` applied by `gen_runtime.sh`:

- `Ada.Calendar.Arithmetic` — day arithmetic on the parent's Modified-Julian-Day representation (exact; the bare runtime has no leap-second table, so `Leap_Seconds` is always zero).
- `Ada.Calendar.Formatting` — `Image/Value/Split/ Time_Of/Day_Of_Week`, written against the parent's public API with hand-rolled decimal formatting (the runtime excludes the `'Image/'Value` helper units, so the displayed fraction is truncated and `Value` raises on out-of-range fields, both as the language requires).
- `Ada.Calendar.Time_Zones` — bare metal has no time-zone database, so the Calendar keeps UTC and `Local_Time_Offset` returns zero.
- `Ada.Calendar.Delays` — `delay until` on a `Calendar.Time`, delegated to the SMP-hardened `Ada.Real_Time.Delays`.
- the obsolescent J.1 top-level Calendar rename, so Ada-83 with `Calendar`; tests build.

Hardware-verified: C961001 (Formatting `Image/Value` conformance) and C960002 (`Delay_Until`) PASS on both profiles, and the `with Calendar`; tests C96005/C96006 PASS. The one Calendar test that does *not* pass is C96004, which requires `Year_Number'Last = 2399` (Ada 2005); the bare runtime's Julian-day Calendar is bounded to the Ada 95 2099, and lifting it needs a new time base rather than a subtype edit (see the Limitations chapter). `light-tasking` is deliberately *excluded*: it is a `No_Exception_Propagation` profile, so every `Time_Error/Constraint_Error` becomes a board

reset — `Ada.Calendar` is unusable and grades nothing there, which is exactly why `bb-runtimes` omits it from the Jorvik profile. By contrast `Ada.Numerics` needed no work at all: the parent, elementary functions, the random generators, complex types, real arrays and big integers already ship and compile into both profiles (HW-verified via CXG2003, CXG2009, CXG1004).

16.9 Why this matters

The standalone sweep is slower per test, but it produces an unambiguous, file-per-test record: every outcome is one marker file containing the exact console transcript that justified it. Cross-checking the two methods is itself a result — the bundled run’s ungraded tail is explained, test by test, by the standalone run’s passes, and the only remaining non-passes are a short, named list of artifacts and documented frontiers.

Chapter 17

Building and Porting the Runtime

The earlier *Building and Running* chapter (§5) showed the runtime from the outside: pick a profile, run `./x`, get an image. This chapter opens the box. It explains how the three runtimes are actually *constructed* — where the Ada runtime sources come from, what makes one profile differ from another, how the experimental `full` profile is synthesised, and how the register packages are generated. If you ever need to change a runtime unit, add a peripheral, or port the whole stack to another Xtensa chip, this is the map.

17.1 The idea: a board on top of bb-runtimes

The runtime is not written from scratch. It is grown from AdaCore’s `bb-runtimes` — the same generator that produces the bareboard GNAT runtimes for Cortex-M, LEON, RISC-V and the rest. `bb-runtimes` knows how to assemble a runtime from a large pool of shared Ada units; what it needs from *you* is a *board definition*: the target triple, the clock, the interrupt model, and a handful of board-specific bodies (the console, the timer, machine reset).

`bb-runtimes` lives in the repository as a git submodule — a fork (`rowsail/bb-runtimes`, branch `xtensa-esp32s3-port`) that carries the ESP32-S3 board on top of the upstream sources. Keeping it a submodule means the board port is version-pinned and the upstream history stays visible.

From C / ESP-IDF: There is no equivalent step in an ESP-IDF project: the C toolchain ships a fixed newlib and FreeRTOS. Here the language runtime itself is a build product you can read, patch and regenerate — the scheduler, the tick, the secondary stack and the exception machinery are all Ada source in the tree.

17.2 `gen_runtime.sh`: from board to archive

One script, `crates/esp32s3_rts/gen_runtime.sh`, turns the board definition into a ready-to-link runtime. It is driven by a single environment variable:

```
export ESP32S3_RTS_PROFILE=embedded      # light-tasking (default) | embedded | full
crates/esp32s3_rts/gen_runtime.sh
```

The script does four things:

1. **Generate the source tree.** It runs `bb-runtimes’ build_rts.py` against the `esp32s3` board, which copies the selected pool units plus the board’s own sources into a profile-specific directory:

```
python3 build_rts.py -f -o "$HERE" \
  --rts-src-descriptor=gnat_rts_sources/lib/gnat/rts-sources.json esp32s3
```

The output lands in `crates/esp32s3_rts/<profile>-esp32s3/` — a complete runtime with `gnat/` (sequential units) and `gnarl/` (tasking units) subdirectories. These directories are *git-ignored*: they are build products, regenerated on demand.

2. (full only) **Apply the overlay and patches** — the next section.
3. **Compile** the tree with `gprbuild` against a small `raven-scar_build.gpr`.
4. **Archive** the objects into `adalib/libgnat.a` (sequential) and `adalib/libgnarl.a` (tasking) with `xtensa-esp32-elf-ar`. The `esp32s3_rts.gpr` that applications with points its Runtime at `<profile>-esp32s3/`, so consuming the runtime is just naming the crate.

Crucially the compile-and-archive step is guarded: it runs only when `adalib/libgnat.a` is *absent*. A second invocation with the tree already built is therefore nearly instant — but it is also the source of a trap (§17.6).

17.3 Three profiles, three `system.ads`

What actually distinguishes the profiles is mostly one file: `system.ads`, the package that declares the target’s restrictions and capabilities. The board class maps each profile name to a variant:

Profile	<code>system.ads</code> variant
light-tasking	<code>system-xi-xtensa-light-tasking.ads</code>
embedded	<code>system-xi-xtensa-embedded.ads</code>
full	synthesised locally (see below)

The light-tasking variant carries the restrictive pragmas — `No_Exception_Propagation`, `No_Finalization`, `No_Secondary_Stack`, all under `pragma Profile` (Jorvik). The embedded variant is the *same Jorvik profile* with those three restrictions lifted: exceptions propagate (zero-cost, ZCX), controlled types finalize, and a secondary stack exists. Changing profile is, at heart, swapping this declaration — everything downstream (which pool units `build_rts` selects, which code the compiler generates) follows from it.

The board’s own bodies are shared across profiles and named with the `__esp32s3` convention that `bb-runtimes` uses for board files:

File	Role
<code>s-bbpara__esp32s3.ads</code>	board parameters: 240 MHz clock, interrupt range, stacks
<code>s-bbbosu__esp32s3.adb</code>	board support: the tick (<code>CCOMPARE</code>), interrupt dispatch
<code>s-textio__esp32s3.adb</code>	console Put via the ROM <code>esp_rom_printf</code>
<code>s-macres__esp32s3.adb</code>	machine reset
<code>a-intnam__esp32s3.ads</code>	<code>Ada.Interrupts.Names</code> for the CPU’s interrupts

17.4 The full profile: overlay plus patches

There is no upstream Jorvik-free Xtensa runtime, so `full` is built on top of `embedded` rather than generated directly. `gen_runtime.sh` generates the embedded tree, then layers two things on it:

- an **overlay** (`full_overlay/`) — a de-restricted `system.ads` and a set of GNARL units (interrupt management, the OS primitives, the tasking primitives) that replace the Jorvik-limited versions with the full ones; and
- a small set of **source patches**, kept as ordinary `.patch` files and applied with GNU `patch`:

```
patch -p1 -d "$RTS" < full_overlay/patches/01-...-secondary-stacks.patch
```

Each patch fixes one concrete defect that only surfaces under full tasking, and each is documented at its head:

Patch	What it fixes
secondary-stack reclaim trampoline IRAM alias	free a task's heap secondary stack when it terminates point nested-subprogram trampolines at the executable (IRAM) alias of SRAM, not the data view — otherwise the first call faults
To_Time_Span rounding	a nonzero Duration must round to ≥ 1 tick (the 4.17 ns tick is coarser than Duration'Small)
dynamic-priority re-apply	re-apply a Set_Priority after the lock is released, which the bareboard unlock would otherwise undo
FIFO-within-priority	implement RM D.2.3 so a base-priority change moves a task to the tail of its ready queue

Keeping these as diffs rather than wholesale copied files means each change is small, reviewable, and obviously attributable to a specific runtime behaviour — and it keeps `gen_runtime.sh` readable.

17.5 Register packages from the SVD

The HAL never hard-codes a peripheral address. Every register is a typed Ada record drawn from `ESP32S3_Registers.*`, and those packages are *generated* from Espressif's machine-readable SVD (System View Description) by `svd2ada`. The generator script, `libs/esp32s3_hal/regenerate.sh`:

1. fetches a pinned `esp32s3.svd` (a specific `espressif/svd` commit, overridable with `ESP32S3_SVD=...`);
2. builds `svd2ada` from *AdaCore* source — the Alire-indexed build is too old and mishandles dimensioned register arrays;
3. runs it into `libs/esp32s3_hal/svd/`:

```
svd2ada esp32s3.svd -o svd -p ESP32S3_Registers --boolean
```

The root package is named `ESP32S3_Registers` on purpose: GNAT forbids user-defined children of `Interfaces`, so the natural `Interfaces.ESP32S3` is not allowed. One wrinkle needs a post-process: the SVD has a peripheral literally called `SYSTEM`, which becomes `ESP32S3_Registers.SYSTEM` — and because Ada is case-insensitive, a bare `System...` reference inside that unit resolves to the wrong package. A one-line `sed` rewrites those references to the fully qualified `Standard.System...`. Add a peripheral and the recipe is the same: add `ESP32S3.<Peri>` to the HAL, backed by the generated register record.

17.6 The stale-cache trap

Because the heavy compile/archive step runs only when `adalib/libgnat.a` is missing, editing a runtime source *in the generated tree* does not, on its own, rebuild anything — the next build links the old archive and your change silently does nothing. This has burned us more than once, so it is worth stating plainly:

After editing a generated runtime `.adb`, delete the archives before rebuilding:

```
rm crates/esp32s3_rts/<profile>-esp32s3/adalib/libgnat.a \
    crates/esp32s3_rts/<profile>-esp32s3/adalib/libgnarl.a
```

(Editing a source under `bb-runtimes/` or `full_overlay/` instead regenerates correctly, because the whole tree is rebuilt; the trap is specifically editing the *regenerated copy*.) The same caution applies to verifying any runtime fix: confirm the change is in the linked archive, not just on disk, before drawing a conclusion.

17.7 Porting to another board

The structure makes the porting checklist short. To bring the runtime to another Xtensa SoC you would: add a board class under `bb-runtimes/xtensa/` (target triple, clock, interrupt range, SMP flag); supply the five `__<board>` bodies (parameters, board support, console, reset, interrupt names); write the two `system-xi-xtensa-*.ads` variants (or reuse these if the restriction set is the same); regenerate the register packages from that chip's SVD; and point `gen_runtime.sh` at the new board name. The boot path — the 2nd-stage loader, the clock and cache bring-up, PSRAM — is the subject of the next chapter.

Chapter 18

The Boot Path: Bootloader and PSRAM

When the chip comes out of reset, a great deal has to happen before the first line of Ada in your `Main` runs: the application has to be read out of flash and placed in RAM, the instruction and data caches have to be pointed at the right flash windows, the CPU has to be sped up from its sleepy reset clock, the register windows have to be made sane for Ada's calling convention, and — if the board has it — the external PSRAM has to be brought up and mapped. This chapter follows that path. It is the part of the system that is *not* Espressif's: the project replaced the last vendored binary, the 2nd-stage bootloader, with its own — written, like almost everything else here, in Ada.

18.1 Three things run before `Main`

Boot happens in three hand-offs (the boot figure in Chapter 3, Fig. 3.2, sketches the same sequence):

1. **The mask ROM** (silicon, unchangeable) wakes on reset, reads the image at flash offset `16#0#`, and loads it into SRAM. That image is our *2nd-stage bootloader*.
2. **Our 2nd-stage bootloader** reads the *application* image (at flash offset `16#1_0000#`), copies its RAM segments into place, maps its flash-resident code and data through the cache MMU, brings up PSRAM, and jumps to the application's entry point.
3. **The application's `start.S`** normalises the CPU state, configures the instruction cache, switches to the full clock, and calls into the Ada world (eventually the environment task that runs `Main`).

18.2 The 2nd-stage bootloader

The loader is ZFP-style Ada — no runtime under it — in `examples/common/bare/bootloader/` (the core is `boot_main.adb`). It is deliberately small. Its job, in order:

- **Silence the watchdogs.** The boot ROM arms the timer-group and RTC watchdogs; the loader unlocks them (the magic key `16#50D8_3AA1#`) and clears them so they cannot reset the chip mid-load.
- **Read and validate the app header.** At flash offset `16#1_0000#` sits the image header: a `16#E9#` magic byte, a segment count, and the entry-point address. (This is exactly the format the `esp_elf2image` host tool emits — §5.)
- **Place each segment by where it wants to live.** For every segment the loader looks at its load address:

Window	Range	Action
IRAM	16#4037_0000#..16#403E_0000#	copy from flash into SRAM
DRAM	16#3FC8_0000#..16#3FD0_0000#	copy from flash into SRAM
IROM	16#4200_0000#..16#4400_0000#	map (execute-in-place via cache)
DROM	16#3C00_0000#..16#3E00_0000#	map (read-in-place via cache)

RAM segments are copied byte-for-byte with the ROM SPI-flash read routine; the large code and read-only-data segments are left *in* flash and reached through the cache.

- **Map the flash windows through the cache MMU.** The loader programs the instruction-bus and data-bus MMUs (`Cache_Ibus_MMU_Set` / `Cache_Dbus_MMU_Set`, 64 KB pages) so the IROM/DROM virtual addresses resolve to the right flash pages, then clears the cache “shut” bits to switch the buses on.
- **Bring up PSRAM** (next section) and **jump** to the entry point.

The loader’s own code and stack are placed high in SRAM, above the region the application’s segments occupy, so copying the app never overwrites the running loader.

18.3 PSRAM: octal bring-up

External PSRAM is the one place the project still leans on opaque Espressif ROM/blob routines, and for a good reason: bringing up octal PSRAM reconfigures the very MSPI timing that flash execute-in-place depends on, so it has to run from RAM, carefully, at exactly the right moment. The 2nd-stage bootloader is that moment. The bring-up (a small C shim, `psram_boot.c`) does:

1. **Wire the octal pins.** `esp_rom_opiflash_pin_config` and the MSPI drive-strength setup connect the extra data lines and DQS that octal mode needs.
2. **Enable and size the PSRAM** via the octal-PSRAM ROM implementation, reporting the detected size.
3. **Force the cache read mode.** The tuning step leaves the SPI0 data-in mode wrong for OPI-DDR cache reads; a single register poke (16#6000_30C0#) forces the correct mode.
4. **Map it into the address space.** `Cache_Dbus_MMU_Set` with the external-RAM flag maps the PSRAM at virtual base 16#3D00_0000#; the number of 64 KB pages comes from the project’s `board.ads` (`PSRAM_Size` / 64 KB).

Because the bootloader maps PSRAM *before* the application’s `start.S` sets up its stack, the application can even place the environment task’s primary stack in PSRAM (the `ENV_STACK_PSRAM` build option), trading a little latency for a great deal of headroom on deep elaboration or recursion — which is exactly what the full-profile ACATS sweep needs.

From C / ESP-IDF: In ESP-IDF, `CONFIG_SPIRAM=y` hides all of this. Here it is a few dozen lines you can read: the pins, the enable call, the one mode-register fix, and the MMU map. The size comes from `board.ads`, so a different module is a one-line change, not a menuconfig safari.

18.4 `start.S`: from reset state to Ada

Once the loader jumps in, the application’s `start.S` (`examples/common/bare/start.S`) turns the raw post-loader state into something Ada can run on:

- **Vectors and processor state.** It sets `VECBASE` to our own vector table (so any fault from here uses our handlers) and sets `PS` to the windowed-ABI state the runtime expects.

- **Sane register windows.** It rewrites WINDOWSTART to a single frame at the current WINDOWBASE, discarding the loader’s stale call frames so the first window overflow/underflow does not fault. (Getting this ordering right was the cure for an early environment-task double fault.)
- **FPU on.** It sets CPENABLE so the first floating-point instruction does not trap.
- **Stack and .bss.** It loads the stack pointer from the linker symbol and zeroes .bss.
- **The instruction cache.** The loader enabled caching but left the I-cache unconfigured; start.S configures a 16 KB, 8-way, 32-byte-line I-cache and sets the IROM/DROM MMU split, so instruction fetches from the IROM window actually return code rather than zeros.
- **The clock.** This is the big one. Out of reset the CPU runs at 20 MHz (the crystal divided by two); the runtime’s timer math assumes 240 MHz, so short delays would run twelve times too long. start.S selects the 480 MHz PLL divided to 240 MHz (registers 16#600C_0010# and 16#600C_0060#) and tells the ROM timing helpers about the new frequency, all from IRAM before the jump into flash code.

It then jumps (not calls) into the C/Ada glue, which sets up the environment task via `__gnat_enter_env` — a small shim that makes the env-task body the *outermost* window frame, so the first context switch has nothing stale above it to corrupt. On the second core, a parallel cold-start entry (`core1_start`) repeats the window/VECBASE/stack normalisation before the SMP runtime releases it.

18.5 The memory map

All of this is pinned down by the linker scripts under `examples/common/bare/vendor/` (the same regions appear in the memory-map Table 8.1). The ESP32-S3’s internal SRAM is visible at two addresses — an instruction-bus (IRAM) view and a data-bus (DRAM) view of the *same* bytes — which is why code and data segments land in different windows of one physical memory:

Region	Base	What lives there
IRAM code	16#4037_8000#	vectors + hot code copied to SRAM
IROM (flash)	16#4200_0000#	bulk code, execute-in-place via cache
DRAM	16#3FC9_0000#	data, .bss, heap
DROM (flash)	16#3C00_0000#	read-only data via cache
PSRAM	16#3D00_0000#	external RAM (mapped by the loader)
RTC slow mem	16#5000_0000#	retained across deep sleep

Two practical consequences fall out of this map and recur elsewhere in the book. First, executable code must live in (or be reached through) an instruction-bus window: that is why the full profile relocates GCC’s nested-subprogram trampolines to the IRAM alias rather than leaving them on the data-view stack (§17). Second, PSRAM is reachable only after the loader maps it, and it cannot be a DMA source — which is why DMA-capable scratch buffers are carved from internal SRAM even when the task stack itself lives in PSRAM (the SD and filesystem drivers rely on this).

Chapter 19

The Context Switch

Every preemptive runtime turns on one small, brutal piece of code: the routine that freezes one task's CPU state, thaws another's, and jumps. On a flat register machine that routine is tedious but obvious. On the Xtensa LX7, with its *windowed* register file and a hardware FPU bolted on the side, it is neither — and getting it wrong produces failures that look like anything but a broken context switch. This chapter walks through the mechanism end to end: where it lives, the state it must preserve, the two completely different ways it is entered, and how the trickier parts were verified on hardware.

19.1 Where it lives

The switch spans three files, two languages, and the boundary between the generic GNARL kernel and this CPU port:

- `crates/bb-runtimes/xtensa/context_switch.S` — the register-level switch itself, in hand-written Xtensa assembly. It is copied verbatim into each profile's runtime tree (`<profile>-esp32s3/gnarl/context_switch.S`) when `gen_runtime.sh` builds the RTS, so editing the `bb-runtimes` copy and regenerating is what changes the running behaviour.
- `System.BB.CPU_Primitives` (`s-bbcppr.adb`) — the Ada entry point `Context_Switch`, a thin wrapper that picks the per-CPU thread-table slots and calls the assembly. This is the seam the portable kernel sees; every CPU port provides its own.
- `System.BB.CPU_Specific` (`s-bbcpsp.ads`) — the `Context_Buffer` record: the per-task saved CPU state, whose field offsets the assembly hard-codes.

Above all of this sits the portable scheduler, `System.BB.Threads.Queues` (`s-bbthqu.adb`). It never touches a register; it only maintains the ready queue and answers one question — `Context_Switch_Needed`, defined simply as `First_Thread /= Running_Thread`. When the highest ready task is not the one on the CPU, the kernel calls `CPU_Primitives.Context_Switch` and the machinery below takes over.

19.2 The data structures

Two structures carry the state.

The `Context_Buffer`, one per task, is deliberately tiny:

Field	Off	Meaning
SP	0	task stack pointer (top of its spilled frame)

Field	Off	Meaning
PC	4	resume target: a <code>retw</code> stub, or the new-task trampoline
PS	8	processor state (INTLEVEL, window control, ...)
AO	12	windowed return address
THREADPTR	16	thread-local-storage base
CP_State	20	pointer to the FPU/coprocessor save area
SAR	24	shift-amount register
LBEG/LEND/ LCOUNT	28..36	zero-overhead loop registers
Frame_Kind	40	0 = solicited, 1 = interrupt (see below)

What is conspicuously *absent* is the general register file. That is the whole point of the windowed design, and the next section explains why.

The thread tables, `Running_Thread_Table` and `First_Thread_Table`, are indexed by CPU (this is a genuine two-core SMP target). `Running` is who is on the core now; `First` is who the scheduler wants there. `Context_Switch` passes the *addresses* of these two slots to the assembly, which dereferences them to the descriptors whose first field is the `Context_Buffer`. The switch updates `Running := First` as part of resuming, so the table always names the live task.

19.3 The windowed-ABI premise

On the Xtensa windowed ABI a `call` does not push registers; it *rotates* a 64-entry physical register file so the callee sees a fresh window. Live data from outer frames stays in the physical registers until the file fills, at which point the hardware raises window-overflow exceptions that spill the oldest frames to their stacks; returning raises window-underflow exceptions that reload them. The implication for a context switch is liberating: to save a task, you do not copy sixteen or thirty-two registers — you issue `SPILL_ALL_WINDOWS`, which flushes every live frame to that task's own stack, and then almost the entire register state lives on the stack. All you must record separately is the stack pointer and the handful of special registers the window machinery does not cover (SAR, the loop registers, the FPU, `THREADPTR`). Resuming is the mirror image: set the stack pointer, execute one `retw`, and let window-underflow lazily fault the frames back in as the task returns up its own call chain. This is why the `Context_Buffer` is so small, and why this port is closest in spirit to the SPARC (LEON) windowed port rather than the flat ARM and RISC-V ones.

19.4 Two ways in

A task can lose the CPU two very different ways, and the switch handles them with two different entry points.

Cooperative — the running task blocks (a `delay until`, a protected entry that cannot proceed) or a higher task becomes ready at a point where the kernel is already running. The kernel calls `Context_Switch`, which calls `__gnat_context_switch`. This path runs in the task's own context, so it can do the full `SPILL_ALL_WINDOWS` and save the outgoing task in *solicited* form.

Preemptive — a level-5 timer or IPI interrupt fires while a task is running. The L5 vector (`highint5.S`) saves the interruptee with `_xt_context_save` into a full XT_STK interrupt frame, services the tick, and — if that made a higher task ready — tail-jumps `__gnat_preempt_dispatch` to switch away. This path saves the outgoing task in *interrupt* form.

The two must not be mixed, and that is the subtle part. A cooperative SPILL_ALL_WINDOWS executed from inside an interrupt's borrowed window chain corrupts the register windows. So when the kernel decides mid-interrupt that a switch is needed, Context_Switch does *not* switch: it sets a per-CPU Switch_Pending flag and returns. The interrupt epilogue, running in the vector's clean single-window context, notices the flag and performs the dispatch itself via `__gnat_preempt_dispatch`. (Tracking down a bug here — the cooperative spill running in interrupt context, marching the stack pointer down one frame per tick until a window spill double-faulted — is the story behind several of the comments in `highint5.S` and `context_switch.S`, and behind ACATS CXD8002.)

19.5 Dual-format dispatch

Because a task can be saved either way, each `Context_Buffer` records its `Frame_Kind`, and the shared resume code reads it to resume correctly:

- **Solicited** (`Frame_Kind = 0`): the live windows were spilled to the task's stack. Resume by restoring SP, SAR, the loop registers and PS, then a single `retw` — window-underflow reloads the rest.
- **Interrupt** (`Frame_Kind = 1`): the full register state is in an `XT_STK` frame at SP. Resume by calling `_xt_context_restore` and returning with `rfi`.

Both entry points converge on a single label, `.Lresume`: it makes `First` the running task, restores the format-independent extra state (FPU, `THREADPTR`), then branches on `Frame_Kind` to one of the two resume tails. The interrupt tail carries hard-won detail — it forces a single clean window (`WINDOWSTART = 1 « WINDOWBASE`) so a freshly scheduled task does not inherit the dispatcher's stale window mask, reloads SAR (clobbered by the window-start arithmetic), and returns with `rfi 5` so `INTLEVEL` stays high until the very last instruction, closing the race where a fresh tick preempts a half-restored task.

19.6 Walkthrough: a cooperative switch

A task calls `delay_until`. The kernel marks it blocked, finds a different `First_Thread`, and calls `__gnat_context_switch` with the two slot addresses:

1. SPILL_ALL_WINDOWS flushes the task's live frames to its stack.
2. Save into the outgoing buffer: SP, AO, PS, SAR, the loop registers, `THREADPTR`, and (if the task owns an FPU save area) `F0..F15 + FCR + FSR`. `Frame_Kind := 0`.
3. Fall into `.Lresume`: set `Running := First`, restore the incoming task's FPU and `THREADPTR`, and — since the incoming task was itself last saved solicited — restore its SP/SAR/loops/PS and `retw` into it.

19.7 Walkthrough: a preemptive switch

The L5 timer fires while a compute task runs:

1. The vector saves the interruptee with `_xt_context_save` into an `XT_STK` frame and services the tick.
2. If a higher task is now ready, the epilogue tail-jumps `__gnat_preempt_dispatch` from the vector's clean single window (no spill here — the full state is already in `XT_STK`).
3. `__gnat_preempt_dispatch` records the interruptee: SP (= the `XT_STK` frame), `THREADPTR`, the FPU, `Frame_Kind := 1`.

4. `.Lresume` restores the incoming task's FPU/THREADPTR and resumes it in its own format — solicited via `retw`, or interrupt via `_xt_context_restore + rfi`.

19.8 Starting a task that has never run

A brand-new task has no spilled frames to reload, so its first dispatch cannot use `retw`. `Initialize_Context` sets the new buffer's PC field to the trampoline `__gnat_start_thread` and stashes the task's entry point and argument at the top of its fresh stack. The first time the scheduler resumes the task, control lands in the trampoline, which loads the argument and `callx`es the task wrapper — which never returns. The buffer's loop registers are zero-initialised (a non-zero LCOUNT would make the hardware spuriously loop on first entry) and THREADPTR starts at zero, i.e. “no thread-local storage” until the task sets it.

19.9 What state is saved — and what is not

State	Saved?	Notes
SP/PS/A0, register windows	always	spill + frame
SAR, LBEG/LEND/LCOUNT	always	loops: XCHAL_HAVE_LOOPS=1
THREADPTR	always	TLS base; per task
F0..F15 + FCR + FSR	conditional	only when the task owns a CP_State area
MAC16 ACCL0/ACCHI/Mn	no	XCHAL_HAVE_MAC16=0 — the unit is absent

The FPU is handled *eagerly per task*: CPENABLE is enabled globally, every task is given a CP_State save area, and the switch copies all sixteen F registers plus FCR/FSR whenever the buffer's CP_State pointer is non-null. THREADPTR was the last addition to the list; for some time the field existed in the record but the assembly never touched it, leaving all tasks sharing one value — harmless only because nothing yet used per-task TLS.

19.10 Verifying the hard parts

The FPU and THREADPTR paths share a verification hazard: a test that exercises them naïvely passes whether or not the switch actually preserves them, because the value under test was sitting safely in memory the whole time. A green test means nothing unless it is *sensitive* — demonstrably red when the property is broken. Both were therefore verified two-sided on the full profile: a deliberately broken build must fail, and only then does the intact build's success count.

FPU. A low-priority checker holds six floating-point accumulators live in registers across a two-million-step loop (each step the exact identity $X := X * L_m * L_i$, with $L_m=2.0$ and $L_i=0.5$ read once from `Volatile` cells so the compiler can neither fold the pair away nor spill the accumulators — disassembly confirmed zero memory operations in the loop). A higher-priority hog wakes every millisecond and tramples all of F0..F15. With the preemptive FPU save disabled the checker logs corruption continuously; with it intact, zero corruption across hundreds of preemptions per batch.

THREADPTR. A checker task sets THREADPTR to one sentinel; a higher-priority hog sets a different one and preempts it (both confirmed on the same core). The clean discriminator turned out to be the hog's value on re-entry, not the checker's — the checker, as the background task, reads its own value either way. With the restore disabled the hog comes back seeing the checker's sentinel (its own was lost); with the restore intact it keeps its own. That asymmetry is the proof the per-task save/restore works.

Two methodology traps recurred and are worth stating once. First, optimisation: a value only tests the switch if the compiler keeps it *live in a register* across the preemption window — verify that by disassembling the loop, not by reading the source. Second, staleness: editing `bb-runtimes/xtensa/context_switch.S` does not take effect until the generated *base* runtime tree is deleted and regenerated, and the only trustworthy confirmation that a change is in the running image is to disassemble the final linked ELF. Both traps produce a green run from a binary that does not contain your change — the most dangerous kind of pass.

Chapter 20

Interrupts

The context switch in the previous chapter does not run by magic — something has to *call* the scheduler. On this port that something is almost always an interrupt: the level-5 timer tick, or a device interrupt whose handler makes a higher-priority task ready. This chapter follows an interrupt from the silicon vector all the way to an Ada handler, and explains how `pragma Attach_Handler` is supported across all three profiles — including the bare-board full `System.Interrupts` the full profile needs, which no stock bare-board runtime provides.

20.1 The Xtensa interrupt levels

The ESP32-S3’s Xtensa LX7 has six maskable interrupt levels plus a non-maskable one, and two of those levels are special:

Level	Role on this port
1	The general <i>exception</i> vector (synchronous faults). No async interrupt is assigned here.
2–3	Device interrupts (<code>__gnat_level2_vector</code> / <code>__gnat_level3_vector</code>).
4	<i>Unused</i> — there is no level-4 vector; <code>EXCSAVE_4</code> is borrowed as a scratch register by the level-5 vector.
5	Kernel-reserved: the <code>CCOMPARE2</code> timer tick (interrupt 16) and the cross-core IPI “poke” (interrupt 31).
6	<code>DEBUGLEVEL</code> — debug exceptions: breakpoints and the <code>DBREAK</code> data watchpoints (used for stack-overflow detection).
7	<code>NMILEVEL</code> — non-maskable; not used.

Two architectural constants shape everything below. `XCHAL_EXCM_LEVEL` is 3, so `PS.EXCM` masks only levels 1–3; levels 4–6 are “high-level” interrupts with their own dedicated vectors and their own `EPCn/EPSn` save registers, and they keep firing even while a low-level exception is being taken. And the kernel owns level 5 exclusively — no application interrupt priority may mask it, or it would silence the scheduler’s own tick (this is why the Ada interrupt-priority ceilings are capped one below it; see the context-switch and CXD1006 discussions).

20.2 Priorities and the masking model

Ada interrupt priorities map onto Xtensa levels in `System.BB.Board_Support.Priority_Of_Interrupt`: a hardware interrupt’s fixed level becomes the Ada priority its handler runs at, and

conversely a task's ceiling becomes the `INTLEVEL` it raises while it holds a lock (`Enable_Interrupts` in `s-bbcppr`). `Ada.Interrupts.Names` gives symbolic IDs for the slots that are free for application use: `Device_L2_0..2` (CPU interrupts 19/20/21), `Device_L3_0/1` (23/27), with ceilings `Device_L2_Priority / Device_L3_Priority`. The tick (16) and poke (31) are listed as kernel-reserved — do not attach to them.

20.3 From silicon to handler

`start.S` points `VECBASE` at our vendored vector table before anything else runs, so every vector is ours. When a level-5 interrupt fires (say the tick), this is the path:

1. **Vector entry** (`xt_highint5, highint5.S`). The CPU jumps to the level-5 slot. The handler first checks one thing *without touching the stack*: was the interruptee inside a window over/underflow handler (`EPC_5` in the window-vector range)? If so its `a1/SP` is transiently invalid, so the tick is *deferred* — push `CCOMPARE2` forward and `rfi` back, letting the window handler finish (the CXD8002 fix).
2. **Save**. Otherwise it builds an `XT_STK` frame on the interrupted task's stack and calls `_xt_context_save` to spill the full register state into it. (Debug is masked across this frame build so the spill cannot trip the per-task stack-overflow watchpoint — see the context-switch chapter.)
3. **Service**. It drops to `INTLEVEL 5` and calls the Ada dispatch routine — `__gnat_timer_interrupt` for level 5, `__gnat_level3_dispatch / __gnat_level2_dispatch` for the device vectors. Each reads the Xtensa `INTERRUPT` register, and for a pending source it owns it calls `System.BB.Interrupts.Interrupt_Wrapper (Id)`.
4. **The wrapper** (`s-bbinte.adb`) is the GNARL bridge: it records the interrupt being handled, raises the running task's active priority to the interrupt's priority (so the ceiling protocol holds and lower interrupts stay masked), runs the *registered handler*, then restores priority. The handler is an access procedure (`Id : Interrupt_ID`).
5. **Epilogue — the deferred switch**. Back in the vector, if the dispatch made a higher-priority task ready (`Context_Switch_Needed`) the switch is performed *here*, from the vector's clean single-window context, by tail-jumping `__gnat_preempt_dispatch`. The kernel never runs the cooperative spill from interrupt context — it sets a `Switch_Pending` flag and lets the epilogue do it (Option A; the previous chapter).
6. **Resume**. If no switch is due, `_xt_context_restore + rfi 5` returns to the interruptee atomically (`INTLEVEL` stays high until the very last instruction, closing the re-entrancy window).

A device interrupt at level 2 or 3 follows the identical shape one level down (`EPS_3/EPC_3, rfi 3`, etc.), and may itself be preempted by the level-5 tick.

20.4 Attaching a handler: two axes

It helps to keep two *independent* distinctions apart. The first is *which API layer* you attach through; the second — which lives entirely inside the high-level layer — is *static versus dynamic* attachment.

20.4.1 Axis 1 — API layer (high level vs low level)

High level — `Ada.Interrupts / pragma Attach_Handler`. The portable Ada way: the handler is a protected procedure, the enclosing protected object's `Interrupt_Priority` is its ceiling, and the compiler generates the registration and ceiling locking. This is what the GPIO HAL and the `esp32s3_intr_levels` example use. The same counting handler as the low-level version below, written this way, is:

```

with Ada.Interrupts.Names; use Ada.Interrupts.Names;

package body High is
  protected P0 with Interrupt_Priority => Device_L2_Priority is
    function Count return Natural;
  private
    procedure Handle;
    pragma Attach_Handler (Handle, Device_L2_0); -- installed at elaboration
    N : Natural := 0;
  end P0;

  protected body P0 is
    procedure Handle is -- runs at the P0 ceiling; locking is implicit
    begin
      Clear_Source; -- ack the (level-triggered) source
      N := N + 1;
    end Handle;
    function Count return Natural is (N);
  end P0;
end High;

```

There is no Install call: `pragma Attach_Handler` wires the handler up when P0 elaborates. The handler is a protected procedure (no `Id` parameter — it is bound to one interrupt), the count `N` is a protected component rather than a bare `Volatile`, and the ceiling protocol serialises it against other accesses to P0 for free. Contrast the hand-rolled low-level form:

Low level — `System.BB.Interrupts.Attach_Handler`. The bareboard kernel’s own primitive: register a plain access procedure (`Id`) for an `Interrupt_ID`, with an explicit priority. This is what `Interrupt_Wrapper` ultimately calls, it is available on *all* profiles, and it needs no protected object — but also gives you no compiler-enforced ceiling. It is the escape hatch when the portable layer is unavailable or unwanted:

```

with System; use System;
with System.BB.Interrupts; use System.BB.Interrupts;

package body Low is
  Count : Natural := 0 with Volatile;

  procedure Handle (Id : Interrupt_ID) is
  begin
    Clear_Source (Id); -- ack the (level-triggered) source
    Count := Count + 1;
  end Handle;

  procedure Install is
  begin
    -- plain procedure, explicit Id + priority, no protected object
    Attach_Handler (Handle'Access, Id => 19,
                    Prio => Interrupt_Priority'Last);
  end Install;
end Low;

```

20.4.2 Axis 2 — static vs dynamic (within the high-level layer)

Static versus dynamic is the dividing line of the `No_Dynamic_Attachment` restriction; it only concerns the high-level `Ada.Interrupts` layer.

Static — pragma `Attach_Handler`. The handler is fixed at compile time and installed once, automatically, when the protected object is elaborated. There is no run-time attach call; the compiler lowers it straight to `System.Interrupts.Install_Handlers`, *bypassing* `Ada.Interrupts`. This is the form to reach for by default:

```
with Ada.Interrupts.Names; use Ada.Interrupts.Names;

package body Blink is
  protected L2_P0 with Interrupt_Priority => Device_L2_Priority is
  private
    procedure Handle;
    pragma Attach_Handler (Handle, Device_L2_0); -- installed at elaboration
    N : Integer := 0;
  end L2_P0;

  protected body L2_P0 is
    procedure Handle is
    begin
      Clear_L2;                -- ack the source
      N := N + 1;
    end Handle;
  end L2_P0;
end Blink;
```

Dynamic — the `Ada.Interrupts` **operations**. Here the handler is attached, swapped, or detached *at run time*. The protected procedure must first be made eligible with pragma `Interrupt_Handler`; then `Attach_Handler`, `Detach_Handler`, `Exchange_Handler` and `Current_Handler` act on it during execution. These calls go *through* the `Ada.Interrupts` body — the path the bare runtime initially stubbed out (every operation raised `Program_Error`) until a delegating a-interr body was supplied (see *Why the profiles differ* below):

```
with Ada.Interrupts;      use Ada.Interrupts;
with Ada.Interrupts.Names; use Ada.Interrupts.Names;

package body Dyn is
  protected P0 is
    procedure Handle;
    pragma Interrupt_Handler (Handle); -- eligible for dynamic attach
  end P0;
  protected body P0 is
    procedure Handle is begin null; end Handle;
  end P0;

  procedure Run is
  begin
    Attach_Handler (P0.Handle'Access, Device_L2_0); -- attach now
    -- ... later, restore the default treatment ...
    Detach_Handler (Device_L2_0);
  end Run;
end Dyn;
```

So the two axes compose: a high-level handler is either static (the pragma, installed at elaboration) or dynamic (the run-time `Ada.Interrupts` calls), while the low-level `System.BB.Interrupts` primitive sits beneath both — itself a run-time call, but *not* the `Ada.No_Dynamic_Attachment` notion of “dynamic”.

20.5 Why the profiles differ

The high-level form compiles differently depending on the tasking profile, and that is the crux:

- embedded and light-tasking set pragma Profile (Jorvik). Under a Ravenscar-family profile the compiler lowers pragma Attach_Handler to the *restricted* installation path, System.Interrupts.Install_Restricted_Handlers — which the bareboard s-interr provides. So it works.
- full sets *no* profile pragma (its whole point is to lift the Jorvik restrictions). The compiler then lowers interrupt-handler protected objects to the *full dynamic* machinery — Register_Interrupt_Handler, Install_Handlers, and the Static_Interrupt_Protection / Dynamic_Interrupt_ Protection protected types.

(Note this is the *profile* distinction, not Configurable_Run_Time, which is True in both.) The catch is that AdaCore ships a full System.Interrupts only for hosted (POSIX) targets, where it is built on a signal-server task and is unportable to bare metal; *no* bare-board runtime — not even AdaCore’s own ravenscar-full — implements the full surface, because Ravenscar forbids dynamic attachment in the first place. For a long time, then, a full-profile build of an Ada.Interrupts handler simply failed with construct not allowed in this configuration and System.Interrupts.Register_Interrupt_Handler not defined.

20.5.1 The bare-board full System.Interrupts

That gap is now closed. full_overlay/gnarl/s-interr is a bare-board re-implementation of the full System.Interrupts surface — Register_Interrupt_Handler, the Static_ and Dynamic_Interrupt_Protection protected bases, Install_ Handlers and the dynamic Attach_/Detach_/Exchange_ Handler operations — but layered on the machinery that already exists rather than on a signal server. It only had to add the GNARL-facing surface; the hard parts were already done and proven (the previous sections):

- It installs a single *umbrella* handler per interrupt through the low-level System.OS_Interface.Attach_Handler (= System.BB.Interrupts), then redirects it by re-pointing a handler table. The bare-board primitive accepts only *one* attach per source, so the umbrella is installed once and the table does the rest.
- The interrupt’s run priority is fixed by its Xtensa level, so the controller-programming step ignores the priority argument (Install_Interrupt_Handler marks it Unreferenced). That is what makes the dynamic Attach_Handler path — which carries no priority — safe.
- Interrupt *entries* and the POSIX.5 signal services have no bare-board meaning and raise Program_Error.

The overlay file replaces the restricted s-interr for the full profile *only*; embedded and light-tasking keep the restricted one they need for the Jorvik path. So the same protected-handler source now builds on all three profiles.

There is one wrinkle, and it is subtler than it first looks. The full System.Interrupts alone is necessary but not *sufficient*: the *static* pragma Attach_Handler path reaches it directly (the compiler lowers the handler to Install_Handlers), but the *dynamic* operations go through Ada.Interrupts, and the bare runtime inherited the *restricted* (No_Dynamic_Attachment) Ada.Interrupts body from the Jorvik base — in which every operation is simply raise Program_Error. So for a while static attach worked while Ada.Interrupts.Attach_Handler, Detach_Handler, Exchange_Handler, Current_Handler, Is_Attached and Reference all raised Program_Error (the full ACATS sweep caught this — CXC3007). The overlay now also supplies the *delegating* a-interr.adb that for-

wards each operation to `System.Interrupts`, so the full dynamic API works (CXC3007 HW-verified PASSED).

The genuine residual limitation is narrow: because `Configurable_Run_Time` is `True`, the front end does not generate library-level finalization, so a *library-level* interrupt protected object's `Finalize` — which would restore the previously installed handlers at program shutdown — does not run automatically. Explicit `Detach_Handler` works; only the implicit restore-on-finalization of a library-level handler PO is absent. That is harmless on an embedded target that never exits, and the low-level `System.BB.Interrupts.Attach_Handler` primitive remains available on all profiles as the no-finalization escape hatch.

20.6 Exercising the vectors

Because a bug in the vectors (a corrupted context, a dropped interrupt) is exactly the kind of thing that hides until the worst moment, the `esp32s3_intr_levels` example is a standalone regression test for them. It fires the level-2 and level-3 device vectors with *no external wiring* by routing the `FROM_CPU` interrupt-matrix sources (the same mechanism as the cross-core poke) to `Device_L2_0` and `Device_L3_0` and asserting them from software, while a low-priority victim task holds floating-point accumulators and a `THREADPTR` sentinel live in registers — continually preempted by those two vectors *and* the level-5 tick. If any vector failed to preserve the interrupted context, an accumulator or the sentinel comes back wrong. On hardware the handler counts climb and the corruption count stays zero, confirming all three vectors both dispatch correctly and preserve context. (Level 4 has no vector to test; level 1 carries no asynchronous interrupt on this port.)

The companion `esp32s3_full_intr` example is the same test built under the `full` profile. Its job is narrower: to prove that the identical `pragma Attach_Handler` protected objects now build, bind, link *and run* against the bare-board full `System.Interrupts` described above — the exact source that used to fail with `Register_Interrupt_Handler` not defined on `full`. On hardware it boots clean on both cores and the L2 and L3 handler counts climb together with the clean-batch counter, with zero context-loss and zero faults — so on `full`, too, the portable `Ada.Interrupts` protected-handler form dispatches correctly and preserves the preempted context.

Chapter 21

Full-Profile Limitations (and Why They Rarely Bite)

The full profile is a complete GNARL tasking runtime rebuilt on bare metal — no Ravenscar restrictions, with rendezvous, protected objects with entries, dynamic and nested tasks, dynamic priorities, task attributes, exception propagation, abort, and `Ada.Interrupts` handlers all running on hardware. A handful of rough edges remain. This chapter lists them honestly and, for each, explains why it seldom matters for a real embedded program. The short version: the gaps cluster at the margins of the language — constructs a startup-time, fixed-task-set application almost never reaches — and each has a clean workaround or is explicitly permitted by the Reference Manual.

21.1 Toolchain-bound: asynchronous `select` with a relative delay

A `select ... delay D; then abort ... end select` (the relative, Duration-triggered form of asynchronous transfer of control, ACATS C974*) does not compile under full. This is *not* a runtime gap: under `Configurable_Run_Time` the GNAT front end emits a call that passes a `Duration` to `Enqueue_RT (T : Real_Time.Time)`, which, crossed with this port's type `Time`, does not type-check. Fixing it is a compiler change or a deep `Real_Time.Time` redesign, not an RTS unit we can add.

Why it rarely bites. The *absolute* form (`select ... delay until T`) and ATC triggered by an *entry call* both work. A relative timeout is sugar for “compute `Clock + D`, then wait until that instant”, which the absolute form expresses directly. Timed operations are also commonly written with a guard plus `Ada.Real_Time` arithmetic, untouched by this.

21.2 RM-permitted: aborting a pure CPU loop

abort of a task blocked at any *abort-completion point* — a delay, an entry call, a rendezvous — works and is HW-verified (`'Terminated` flips, the task unwinds cleanly). The one case that does *not* take effect is a task spinning in a tight computational loop with no completion point at all: it keeps running until it next reaches one.

Why it rarely bites. This is explicitly allowed — ARM 9.8 only requires abort to take effect at completion points, and a loop with no such point is permitted to run on. Delivering it anyway would mean injecting an exception into a preempted task from the timer ISR, which (as the context-switch and interrupts chapters describe) is deep, windowed-Xtensa, shared-kernel surgery for a behavior the standard does not mandate. In practice any loop that does I/O, delays, or calls an entry is abortable; a genuinely tight numeric kernel that must be cancellable can poll a `Volatile` flag.

21.3 Configurable_Run_Time finalization

Because `system.ads` sets `Configurable_Run_Time => True`, the front end does not generate *library-level* finalization. Two consequences:

- A *library-level* controlled object's `Finalize` runs only if the object is finalized at scope exit, not as part of program shutdown.
- Consequently, the *implicit* restore-on-finalization of a *library-level* interrupt-handler protected object does not run automatically at shutdown. (Both the static pragma `Attach_Handler` path and the full dynamic `Ada.Interrupts` operations — `Attach_Handler`, `Detach_Handler`, `Exchange_Handler`, `Current_Handler` — are now supported and HW-verified; see the `Interrupts` chapter. Only this one implicit-restore case is outstanding.)

Why it rarely bites. Scope-level finalization — the overwhelmingly common case (a controlled object in a subprogram or block) — works normally. Library-level objects live for the entire life of the program anyway, so on an embedded target that never exits there is nothing to finalize. And interrupt handlers are wired up once at elaboration and stay for the life of the system, so the missing implicit restore-at-shutdown never matters in practice; explicit `Detach_Handler` works if you do need it.

21.4 Ada.Calendar year range stops at 2099

The bare `Ada.Calendar` declares `Year_Number` as 1901 .. 2099 (the Ada 95 bound), whereas Ada 2005 onward — and the modern ACATS C96004 — require 2399. This is not a one-line subtype change: the bare runtime's `Calendar` uses a *Julian-day fixed-point* time representation deliberately sized so the 1901–2099 span fits in 64 bits at sub-second precision. Stretching the range to 2399 (roughly five centuries) overflows that representation — which is exactly why mainline GNAT abandoned the Julian-day `Calendar` for an integer-nanosecond one. Reaching 2399 would mean swapping in a new time base across the whole `Calendar`, on a unit shared by the embedded and full profiles, at the risk of the `Calendar` tests that already pass.

Why it rarely bites. The gap is the years 2100–2399. Everything within 1901–2099 — which comfortably covers the service life of any device this runtime will ever run on — is fully conformant: arithmetic, formatting, `Split/Time_Of`, and `delay until` on a `Calendar.Time` are all HW-verified (see the ACATS chapter). The cost of closing it is a `Calendar` rewrite; the benefit is dates after the year 2099.

21.5 Per-task heap reclamation under task churn

A terminated task's primary stack is returned to the heap, but a program that *churns* tasks — creating and destroying them in a tight loop — can still exhaust the heap: the reclamation of the per-task resources is not yet fully race-free against the terminating thread, and the bare allocator fragments. The underlying cause is that the kernel has no explicit “retire this thread” step; the design for fixing it is recorded in the project notes.

Why it rarely bites. Idiomatic embedded Ada creates a *fixed* set of tasks at startup and runs them for the life of the system — the fixed-set case never churns and never leaks. Dynamic create/destroy in a hot loop is an unusual pattern on a microcontroller; where it is needed, a task pool (allocate once, reuse) sidesteps the issue entirely.

21.6 Minor robustness edges

- **Lowering the running task’s priority below a ready peer** can hang the caller (a bare-kernel reschedule edge). Raising priority, or staying the highest, works. *Why it rarely bites:* priority is normally set once at task creation; the dynamic-lowering-below-peers pattern is rare, and priority *ceilings* (the common dynamic case) go up, not down.
- **Ada.Text_IO over the USB-Serial-JTAG console** is not robust for two tasks printing concurrently. *Why it rarely bites:* this is a console-transport detail, not the tasking kernel — route diagnostic output through one task, or use a real UART (the UART driver is task-safe).

21.7 Architectural: Interrupt_Priority’Last and the kernel tick

A task declared at `Interrupt_Priority’Last` masks the kernel’s own Xtensa level-5 tick and cross-core IPI, deadlocking its activation (ACATS CXD1006; see the ACATS chapter). The kernel owns the top hardware level and shares it with the highest Ada interrupt priority.

Why it rarely bites. Only the single *topmost* interrupt priority collides. Any priority below it — including every priority the peripheral HAL drivers use for their device interrupts — is unaffected, and ordinary tasks use `Priority`, not `Interrupt_Priority`, at all. A handler simply should not claim the one ceiling reserved for the scheduler.

21.8 The common case is clean

None of the above touches the path a typical full-tasking program actually walks: declare a fixed set of tasks and protected objects at startup; rendezvous and share data through protected entries; use absolute delays and timing events; attach interrupt handlers statically; finalize controlled objects at scope exit; abort at completion points. That path is implemented and hardware-verified. The limitations are real, but they live at the edges — toolchain corners, an RM-permitted abort case, and patterns (library-level finalization, task churn, dynamic priority lowering) that idiomatic embedded code seldom uses — and every one has either a clean workaround or standing permission from the language.

Appendix A

Quick Reference

A.1 Usable GPIO pins

The HAL’s `Pin_Id` subtype (§7.3) admits only the pads the ESP32-S3 actually exposes. Everything else is a compile-time error.

Pads	Status
0 .. 21	usable (general-purpose)
22 .. 25	absent on the package
26 .. 32	SPI flash — driving them hangs the chip
33 .. 37	octal PSRAM — driving them hangs the chip
38 .. 48	usable (general-purpose)

So `Pin_Id` is `0 .. 21 | 38 .. 48`, and `Optional_Pin` adds the sentinel `No_Pin` (`= -1`) for “leave this line unrouted.”

A.2 Runtime profiles at a glance

Feature	light-tasking	embedded	full
Tasks, protected objects, delay until	yes	yes	yes
Exception propagation (ZCX)	no	yes	yes
Controlled types / finalization (RAII)	no	yes	yes
Secondary stack	no	yes	yes
Heap	bump only	first-fit	first-fit
select / rendezvous / abort	no	no	yes
RAII-handle HAL drivers (SPI, I2S, ...)	no	yes	yes
Smallest footprint	yes	—	—

Choose `light-tasking` for the smallest, most analysable build, `embedded` for most applications (the default), and `full` only when you need dynamic tasking, rendezvous or abort (Chapter 4).

A.3 Example index

Every example builds and runs with `./x run <name> -p /dev/ttyACM0`; the profile column is its default.

Name	Profile	Shows
<code>gpio0_blink</code>	light-tasking	GPIO output; the minimal app
<code>heartbeat</code>	light-tasking	a periodic task

Name	Profile	Shows
psram	light-tasking	external PSRAM bring-up
smp	light-tasking	dual-core scheduling
embedded	embedded	tagged dispatch, finalization, exceptions
exceptions	embedded	local / propagated / unhandled exceptions
crypto	embedded	SHA + AES accelerators
adc_read	embedded	SAR ADC one-shot
touch_read	embedded	capacitive touch
i2c_loopback	embedded	I2C master self-test
uart_loopback	embedded	UART internal loopback
i2s_loopback	embedded	I2S full-duplex DMA loopback
i2s_pdm	embedded	I2S PDM (PCM ↔ PDM)
gdma_copy	embedded	memory-to-memory DMA
ledc_pwm, mcpwm_pwm	embedded	PWM outputs
rmt_loopback	embedded	RMT pulse TX/RX
pcnt_count	embedded	pulse / edge counter
sdm_output	embedded	sigma-delta density output
twai_loopback	embedded	CAN (standard, extended, RTR)
timer_count	embedded	general-purpose timers
lcd_i8080	embedded	8-bit i80 parallel LCD
rtc_sleep, rtcio_hold	embedded	deep sleep, RTC pad hold
sd_spi, sdmmc, ext4	embedded	SD cards + ext4 filesystem
intr_levels	embedded	interrupt levels/priorities
full_tasking, rendezvous, full_intr	full	rendezvous, dynamic tasking

Appendix B

Glossary

A quick reference to the acronyms and terms this book leans on. Where a term has a chapter that treats it properly, the entry points there.

ACATS

Ada Conformity Assessment Test Suite — the official conformance tests for an Ada implementation. This project runs them on real silicon (Chapter 16).

Alire

The Ada package manager (`alr`). It supplies the cross-toolchain and pins the runtime crate; no other SDK is needed.

bb-runtimes

AdaCore’s “bareboard runtimes” generator. The Ada runtime here is a board port grown from it (Chapter 17).

.bss

The zero-initialised data segment; cleared by `start.S` at boot.

CCOUNT / CCOMPARE

Xtensa cycle-counter and compare registers. The runtime uses them to drive the clock tick and timed delays.

ceiling priority

The locking policy for protected objects: a caller is raised to the object’s ceiling priority for the duration, bounding blocking and preventing priority inversion.

context switch

Saving one task’s CPU state and restoring another’s. The Xtensa version (windows, the coprocessor, `THREADPTR`) is Chapter 19.

DRAM / IRAM

The data-bus and instruction-bus *views* of the same internal SRAM. Code runs from the IRAM view; data lives in the DRAM view (§18).

DROM / IROM

The data and instruction *flash* windows, reached *execute-/read-in-place* through the cache MMU.

GNARL

The GNU Ada Runtime Library — the tasking half of the runtime (tasks, protected objects, the scheduler), archived as `libgnarl.a`.

GNAT

The GNU Ada compiler. Its sequential runtime is `libgnat.a`.

GPR / gprbuild

GNAT project files and their builder. Each crate (`esp32s3_rts`, `esp32s3_hal`) is a GPR project.

HAL

Hardware Abstraction Layer — the `ESP32S3.*` peripheral drivers (Chapters 10–12).

IPI

Inter-Processor Interrupt — the poke one core sends another to wake a task or rebalance, central to the SMP scheduler.

Jorvik

A tasking profile (Ada RM D.13) — a less restrictive successor to Ravenscar that still bans the unbounded constructs. Both `light-tasking` and `embedded` build under it.

MMU

Memory Management Unit. Here it is the cache MMU that maps 64 KB flash pages (and PSRAM) into the CPU's address windows.

MSPI

The memory SPI controller that talks to flash and PSRAM; its timing is what makes PSRAM bring-up delicate (§18).

PDM

Pulse-Density Modulation — the 1-bit serial format some microphones and ADCs emit; the I2S driver converts it to/from PCM.

protected object

An Ada monitor: data plus operations with built-in mutual exclusion and condition synchronisation (§9).

PSRAM

Pseudo-static external RAM, mapped at `16#3D00_0000#` by the bootloader. Not DMA-capable.

RAII

“Resource Acquisition Is Initialization” — tying a resource's lifetime to an object's scope. In Ada it is a controlled type whose `Finalize` releases the resource (§9).

Ravenscar

The original restrictive tasking profile for high-integrity real-time systems; Jorvik's stricter predecessor.

rendezvous

Direct task-to-task synchronisation via `entry/accept`. Available only in the `full` profile.

restriction profile

A named bundle of `pragma Restrictions` (e.g. Jorvik) that the compiler enforces, trading language features for analysability and footprint (Chapter 4).

RTS

Run-Time System — the compiled runtime an application links against; here, one of the three profile archives.

secondary stack

Where Ada returns unconstrained results (e.g. a `String` of run-time-determined length). Absent in `light-tasking`.

SMP

Symmetric Multi-Processing — both ESP32-S3 cores under one scheduler, with tasks pinned per core and woken across cores by IPI.

SVD / `svd2ada`

A machine-readable register description (System View Description) and the AdaCore tool that turns it into typed Ada register packages (§17).

THREADPTR

An Xtensa per-thread pointer register, saved/restored on context switch so per-task thread-local data works.

VECBASE

The Xtensa register holding the base address of the exception/ interrupt vector table; `start.S` points it at our vectors.

watchdog (WDT)

A timer that resets the chip unless fed. The bootloader disables the boot watchdogs; the RTC super-watchdog is re-disarmed after a deep-sleep wake.

window (register window)

The Xtensa calling convention rotates a window of physical registers per call instead of spilling to the stack; mismanaged windows are a recurring source of boot and context-switch bugs.

XIP

Execute-In-Place — running code directly from flash through the cache rather than copying it to RAM.

ZCX

Zero-Cost Exceptions — the exception model in `embedded` and `full`: no overhead unless an exception is actually raised.

ZFP

“Zero FootPrint” — a runtime (or coding style) with no tasking, exceptions or secondary stack; the 2nd-stage bootloader is written this way.

Appendix C

Further Reading

Pointers for going deeper, grouped by topic. Online resources were current at the time of writing.

C.1 The Ada language and its profiles

- **Ada Reference Manual** — the standard itself. <http://ada-auth.org/standards/ada22.html>. The most relevant corners for this book: *D.13* (the Ravenscar and Jorvik profiles), *D.2.3* (priority and dispatching policies), *C.6* (Volatile and Atomic), *I3.3/I3.5* (address and record-representation clauses), and *7.6* / *Ada.Finalization* (controlled types).
- **John Barnes, *Programming in Ada 2012*** — the standard textbook for the language as a whole.
- **AdaCore learn.adacore.com** — free courses, including *Introduction to Embedded Systems Programming* and the *Ada for the C++/Java developer* guide. <https://learn.adacore.com>.

C.2 The runtime and the toolchain

- **bb-runtimes** — AdaCore’s bareboard runtime generator that this project’s runtime is ported from. <https://github.com/AdaCore/bb-runtimes>. The ESP32-S3 board lives in the fork <https://github.com/rowsail/bb-runtimes> (branch *xtensa-esp32s3-port*).
- **svd2ada** — generates the typed register packages from an SVD. <https://github.com/AdaCore/svd2ada>.
- **Alire** — the Ada package manager that supplies the toolchain. <https://alire.ada.dev>.
- **GNAT GPL / GNAT Pro documentation** — for pragma *Restrictions*, the bareboard runtime structure, and *gprbuild*.

C.3 The silicon

- **ESP32-S3 Technical Reference Manual** — the authoritative register reference; every peripheral chapter in this book traces back to it. https://www.espressif.com/sites/default/files/documentation/esp32-s3_technical_reference_manual_en.pdf.
- **ESP32-S3 Datasheet** — pin-out, electrical characteristics, the GPIO matrix.
- **Espressif SVD files** — the register descriptions consumed by *svd2ada*. <https://github.com/espressif/svd>.

- **Xtensa Instruction Set Architecture (LX7)** and the *Windowed Register Option* — background for the context-switch and boot chapters (Cadence/Tensilica documentation).

C.4 Conformance and high-integrity Ada

- **ACATS** — the Ada Conformity Assessment Test Suite. <http://www.ada-auth.org/acats.html>.
- **The Ravenscar Profile for Real-Time Systems** (Burns, Dobbing, Vardanega) — the rationale behind the restricted tasking profiles.

C.5 This project

- **Source repository** — https://github.com/rowsail/ada_esp32s3. The `QUICKSTART.md` and `TOOLING.md` there cover setup and the developer tooling; the $\text{\LaTeX}$ source of this book is under `book/`.

Index

- access types, [25](#)
- ADC driver, [63](#)
- AES driver, [68](#)
- array index range, [22](#)
- arrays, [22](#)

- bb-runtimes, [99](#)
- bootloader, [103](#)

- concurrency patterns, [84](#)
- contracts, [26](#)
- controlled types, [38](#)

- elaboration, [29](#)
- exceptions, [27](#)

- fixed-point types, [24](#)
- full profile, [100](#)

- GDMA driver, [53](#)
- gen_runtime.sh, [99](#)
- GPIO driver, [44](#)

- I2C driver, [48](#)
- I2S driver, [55](#)
- interfacing
 - with C, [35](#)

- LCD driver, [62](#)
- LEDC driver, [57](#)
- limited types, [38](#)
- Linker_Section, [34](#)

- MCPWM driver, [54](#)
- memory map, [105](#)

- numeric literals, [21](#)

- others, [23](#)

- packed arrays, [34](#)
- PCNT driver, [58](#)
- private types, [37](#)
- protected objects, [39](#)
- PSRAM, [104](#)

- RAII, [38](#)
- real-time tasks, [80](#)
- register addressing, [31](#)
- representation clause, [32](#)
- Restrictions pragma, [28](#)
- RMT driver, [57](#)
- RNG driver, [45](#)
- RTC driver, [65](#)
- RTC_IO driver, [66](#)
- runtime profiles, [100](#)

- SDM driver, [59](#)
- SHA driver, [67](#)
- SMP, [86](#)
- SPI driver, [46](#)
- stale cache, [101](#)
- start.S, [104](#)
- Storage_Size, [35](#)
- subtype, [21](#)
- SVD, [101](#)
- svd2ada, [101](#)

- tasks, [77](#)
- Temperature driver, [46](#)
- terminology, [19](#)
- Timer driver, [62](#)
- Touch driver, [67](#)
- TWAI driver, [60](#)

- UART driver, [49](#)
- Unchecked_Conversion, [25](#)

- Valid attribute, [25](#)
- Volatile, [33](#)

- waiting for an event, [81](#)
- watchdog, [85](#)